Министерство науки и высшего образования РФ
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
«Владимирский государственный университет имени Александра Григорьевича и Николая Григорьевича Столетовых»
(**МИ ВлГУ**)

Методические указания
по выполнению лабораторных работ

по дисциплине «Инфокоммуникационные системы и сети»

для направления подготовки 09.03.02 Информационные системы и технологии

Составитель:
старший преподаватель кафедры ИС
Булаев А.В.

Муром, 2024

СОДЕРЖАНИЕ

Лабораторная работа №1. Изучение структуры сетевых протоколов..... 3
Лабораторная работа №2. Утилиты по работе и настройке сети в Linux..... 6
Лабораторная работа №3. Виртуальные частные сети..... 26
Лабораторная работа №4. Программирование сетевого протокола TCP. Разработка клиент серверного приложения..... 58
Лабораторная работа №5. Программирование сетевого протокола TCP. Распределенное решение математической задачи 68
Лабораторная работа №6. Программирование сетевого протокола UDP. Разработка клиент серверного приложения..... 70
Лабораторная работа №7. Программирование сетевого протокола UDP. Решение задачи подбора ключа 79
Лабораторная работа №8. Проектирование компьютерных сетей..... 89

Лабораторная работа №1

Тема: Изучение структуры сетевых протоколов

Цель: Изучение структуры пакетов различных сетевых протоколов

Задача: Освоить использование программы-анализатора сетевого трафика WireShark для изучения структуры сетевых пакетов и дейтаграмм различных сетевых протоколов и получения представления о порядке обмена данными между компьютерами, осуществляющими взаимодействие по сети.

Платформа выполнения работы: RedOS

Задание:

1) Определить какие сетевые протоколы были задействованы при выполнении команды ping mivlgu-proxy.lan.mivlgu.ru. Какой протокол используется для выполнения команды ping? Привести примеры пакетов, перехваченных программой WireShark, и их расшифровку. Из каких этапов состоит выполнение команды ping.

2) Определить какие сетевые протоколы были задействованы при выполнении команды traceroute mivlgu.ru. Какие протоколы используется для выполнения указанной команды? Привести примеры пакетов, перехваченных программой WireShark, и их расшифровку. Из каких этапов состоит выполнение команды traceroute?

3) Определить какие сетевые протоколы были задействованы при выполнении взаимодействия компьютеров при обмене данными по ftp. Привести примеры пакетов, перехваченных программой WireShark, и их расшифровку. Из каких этапов состоит обмен данными по ftp?

4) Определить какие сетевые протоколы задействованы при выполнении просмотра web-контента в браузере. Привести примеры пакетов, перехваченных программой WireShark, и их расшифровку.

5) Определить какие сетевые протоколы задействованы при выполнении файловых операций с сетевым ресурсом. Привести примеры пакетов, перехваченных программой WireShark, и их расшифровку.

6) На основании анализа файлов wsh1.pcapng, wsh2.pcapng, wsh3.pcapng определить какие операции выполнялись пользователем. Дать характеристику

структуре пользовательской сети: какие устройства представлены в сети, какие сетевые технологии функционируют в сети.

7) Перечислить все сетевые протоколы, по которым осуществлялось взаимодействие вашего ПК, и привести примеры их пакетов, которые не описаны в ходе выполнения предшествующих этапов.

8) Собрать статистику появления пакетов по каждому протоколу с отражением их доли в общем числе пакетов.

Ход работы:

1) в режиме суперпользователя произвести установку пакета WireShark.

```
dnf install wireshark
```

2) запустить WireShark в режиме суперпользователя.

```
wireshark
```

3) выбрать для захвата пакетов сетевой интерфейс компьютера. По умолчанию **enp0s3**

4) запустить захват сетевого трафика.

5) выполнить пингование сервисов

```
ping google.com
ping ya.ru
ping vk.com
ping mail.ru
```

6) выполнить команду

```
traceroute mivlgu.ru
traceroute google.com
traceroute ya.ru
traceroute vk.com
traceroute mail.ru
```

7) на двух компьютерах организовать обмен данными по протоколу FTP с использованием утилиты filetto

8) в браузере открыть несколько сайтов

9) подключить сетевые каталоги TEMP и SUBJECTS, располагающиеся на сервере кафедры; выполнить операции: скопировать файл с удаленного каталога на локальную машину, скопировать файл с локальной машины в удаленный каталог, создать в удаленном каталоге собственную папку, проделать для обоих указанных каталогов

10) провести анализ прилагаемых файлов wsh1.pcapng, wsh2.pcapng, wsh3.pcapng.

Лабораторная работа №2

Тема: Знакомство с сетевыми утилитами Linux.

Цель: Изучить утилиты Linux, которые могут использоваться для диагностики работы сети.

Задание:

1. Вывести таблицу ARP компьютера.
2. С использованием команды DIG получить информацию DNS о серверах согласно варианта в таблице 1.
3. Изучить назначение команды DSTAT, отобразить 10 строк статистики, дать пояснение к отображаемой информации.
4. Вывести информацию о параметрах работы сетевого интерфейса с использованием ETHTOOL.
5. Освоить работу с сервером FTP через терминальный режим:
    1. установить соединение;
    2. авторизоваться;
    3. вывести список файлов и каталогов корневой директории ftp;
    4. произвести выгрузку с сервера и загрузку на сервер 1 файла;
    5. разорвать соединение.
6. С использованием команды HOST получить информацию об ip-адресе доменов согласно таблицы 1.
7. Получить статистику использования сетевых интерфейсов ПК с использованием IFSTAT.
8. Получить информацию о сетевых интерфейсах с использованием команды IP.
9. Вывести таблицу IPTABLES (выполняется в режиме суперпользователя).
10. Собрать статистику использования сетевого интерфейса с использованием команды IFTOP (выполняется в режиме суперпользователя).
11. Ознакомиться с информацией отражаемой командой NETSTAT, собрать сведения о используемых сокетах и получить таблицу маршрутизации.

12. Получить информацию о DNS-серверах в таблице 1 с использованием команды NSLOOKUP.

13. Получить информацию о доменах в таблице 1 с использованием команды NMAP.

14. Выполнить проверку доступности доменов в таблице 1 с использованием команды PING.

15. Получить информацию о маршрутах ПК.

16. Получить информацию о сокетах с использованием команды SS: обо всех открытых сокетах, о прослушивающих сокетах, о сокетах tcp-соединений.

17. Выполнить перехват сетевого трафика с использованием команды TCPDUMP (выполняется в режиме суперпользователя).

18. Выполнить перехват сетевого трафика с использованием команды TSHARK. Получить информацию о интерфейсах, сетевой поток которых перехватывает программа (выполняется в режиме суперпользователя).

19. Сравнить информативность данных предоставляемых утилитами TCPDUMP и TSHARK.

20. Получить маршрут следования пакетов для доменов в таблице 1 с использованием команды TRACEPATH.

21. Получить маршрут следования пакетов для доменов в таблице 1 с использованием команды TRACEROUTE.

22. Сравнить информативность данных предоставляемых утилитами TRACEPATH и TRACEROUTE.

23. Собрать информацию, предоставляемую программой WHOIS, для доменов в таблице 1.

Таблица 1 – Варианты доменов

Вариант	*Домен 1*	*Домен 2*	*Домен 3*
1	Mail.ru	Birwarden.com	Python.org
2	Gov.ru	Dzen.ru	Scribus.net
3	Darktable.org	Reddatabase.ru	Ya.ru

4	Habr.com	Wikipedia.org	Reg.ru
5	Haskell.org	Sedicomm.com	Sourceforge.net
6	Iprbookshop.ru	Gnu.org	Kremlin.ru
7	Esbvolga.ru	Blender.org	Google.com
8	Red-soft.ru	Gaspay.ru	Opencv.org
9	Youtube.com	Samba.org	Ozon.ru
10	Github.com	Erlang.org	Wildberries.ru
11	Nasm.us	Yourls.org	Squid-cache.org
12	Red-soft.ru	Netfilter.org	Msu.ru
13	Drweb.ru	Finnix.org	Knoppix.nl
15	Gparted.org	Qt.io	Slitaz.org
15	Linux.com	Knopper.net	Kaspersky.ru

Теоретическая часть

Команда arp

Отображает таблицу ARP вашего компьютера.

```
[studentlin@localhost ~]$ arp
Address                          HWtype  HWaddress           Flags Mask
Iface
_gateway                         ether   50:ff:20:19:99:c4   C
enp0s3
210-210-0-6.lan.sify.ne          ether   20:1e:88:07:9f:40   C
enp0s3
```

Команда dig

Используется для простого поиска DNS на DNS-сервере, таком как CName, MX-записи и т.д.

```
[studentlin@localhost ~]$ dig mivlgu.ru

; <<>> DiG 9.16.16-RH <<>> mivlgu.ru
;; global options: +cmd
;; Got answer:
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4914
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;mivlgu.ru.             IN    A

;; ANSWER SECTION:
mivlgu.ru.        300   IN    A     185.61.25.134

;; Query time: 8 msec
;; SERVER: 210.210.0.1#53(210.210.0.1)
;; WHEN: Вс дек 04 14:27:54 MSK 2022
;; MSG SIZE  rcvd: 54
```

Команда dstat

Отображает статистку системных ресурсов

Чтобы увидеть статистику системных ресурсов в реальном времени, вы можете использовать команду ***dstat***:

```
[studentlin@localhost ~]$ dstat
You did not select any stats, using -cdngy by default.
/usr/bin/dstat:515: DeprecationWarning: Using or importing the ABCs from 'collections' instead of from 'collections.abc' is deprecated since Python 3.3, and in 3.9 it will stop working
  if isinstance(self.val[name], collections.Sequence) and not isinstance(self.val[name], six.string_types):
--total-cpu-usage-- -dsk/total- -net/total- ---paging-- ---system--
usr sys idl wai stl| read  writ| recv  send|  in   out | int   csw
  6   1  92   0   0|1132k  100k|   0     0 |   0     0 |2808  1910
  5   1  94   0   0|   0     0 |  60B    0 |   0     0 |2893  1368
  5   0  95   0   0|   0     0 |   0     0 |   0     0 |2921  1423
  8   2  89   0   0|   0     0 | 171B  171B|   0     0 |2865  2553
  9   2  89   0   0|   0     0 |   0     0 |   0     0 |2995  3683
  7   1  92   0   0|   0     0 | 636B  879B|   0     0 |2926  2524
  6   1  93   0   0|   0     0 |   0     0 |   0     0 |2904  1792
  4   1  95   0   0|   0     0 |   0     0 |   0     0 |2680  1484
  3   0  97   0   0|   0     0 |  60B   60B|   0     0 |2722  1355
  4   0  96   0   0|   0     0 |   0     0 |   0     0 |2820  1312
  4   1  95   0   0|   0     0 |   0     0 |   0     0 |2950  1641
```

```
  5   0  95   0   0|   0  8192B|   0     0 |   0     0 |2978  1310
  4   1  95   0   0|   0     0 |  60B    0 |   0     0 |2766  1333
  4   0  95   1   0|   0    48k|  90B   90B|   0     0 |2637  1431
  5   1  94   0   0|   0    24k|   0     0 |   0     0 |2679  1386
  4   0  96   0   0|   0     0 |  90B   90B|   0     0 |2690  1461
  5   0  95   0   0|   0     0 |   0     0 |   0     0 |2890  1368
  4   1  95   0   0|   0     0 |  60B    0 |   0     0 |2896  1352
  6   0  94   0   0|   0    92k|   0     0 |   0     0 |2729  1609
```

Команда ethtool

Отображает настройки сетевой карты (NIC) компьютера

```
[studentlin@localhost ~]$ ethtool enp0s3
Settings for enp0s3:
      Supported ports: [ TP ]
      Supported link modes:   10baseT/Half 10baseT/Full
                              100baseT/Half 100baseT/Full
                              1000baseT/Full
      Supported pause frame use: No
      Supports auto-negotiation: Yes
      Supported FEC modes: Not reported
      Advertised link modes:  10baseT/Half 10baseT/Full
                              100baseT/Half 100baseT/Full
                              1000baseT/Full
      Advertised pause frame use: No
      Advertised auto-negotiation: Yes
      Advertised FEC modes: Not reported
      Speed: 1000Mb/s
      Duplex: Full
      Auto-negotiation: on
      Port: Twisted Pair
      PHYAD: 0
      Transceiver: internal
      MDI-X: off (auto)
netlink error: Operation not permitted
        Current message level: 0x00000007 (7)
                               drv probe link
      Link detected: yes
```

Команда ftp

FTP – наиболее часто используемый протокол передачи файлов в Linux. С помощью команды ***ftp*** вы можете подключиться к сеансу ftp, а затем с помощью различных команд вы можете передавать файлы. Вы также можете напрямую подключиться к удаленному хосту с помощью команды ftp.

```
[studentlin@localhost ~]$ ftp
ftp> open ftp.bav-service.ru
Connected to ftp.bav-service.ru (185.61.24.210).
220---------- Welcome to Pure-FTPd [privsep] ----------
220-You are user number 1 of 50 allowed.
220-Local time is now 14:34. Server port: 21.
220-This is a private system - No anonymous login
220 You will be disconnected after 15 minutes of inactivity.
Name (ftp.bav-service.ru:studentlin): stuser
331 User stuser OK. Password required
Password:
230 OK. Current restricted directory is /
Remote system type is UNIX.
Using binary mode to transfer files.

ftp> dir
227 Entering Passive Mode (185,61,24,210,23,64)
150 Accepted data connection
drwx------    1 root    1000           0 Nov 24 22:35 DualServer
drwx------    1 root    1000           0 Nov 22 21:32 term-ftp
-rwx------    1 root    1000    16375724 Nov  9 14:45 wsh1-bav-old.pcapng
-rwx------    1 root    1000     8665372 Nov  9 14:45 wsh2-bav-old.pcapng
-rwx------    1 root    1000   204518444 Nov 22 22:28 wsh3-vbredos.pcapng
-rwx------    1 root    1000       90808 Nov 23 15:08 ЛБ-4 - WireShark (1).pdf
-rwx------    1 root    1000     7147982 Sep  5  2018 Я свободен.mp3
-rwx------    1 root    1000     4426231 Sep  5  2018 Я теряю корни.mp3
226-Options: -l
226 8 matches total

ftp> get wsh1-bav-old.pcapng /home/studentlin/wireshark.pcapng
local: /home/studentlin/wireshark.pcapng remote: wsh1-bav-old.pcapng
227 Entering Passive Mode (185,61,24,210,12,1)
150-Accepted data connection
150 15991.9 kbytes to download
226-File successfully transferred
226 0.743 seconds (measured here), 21.03 Mbytes per second
```

```
16375724 bytes received in 0,743 secs (22049,20 Kbytes/sec)

ftp> exit
221-Goodbye. You uploaded 0 and downloaded 16128 kbytes.
221 Logout.
```

Команда host

Команда ***host*** используется для сопоставления IP — Имя хоста .

```
[studentlin@localhost ~]$ host vlsu.ru
vlsu.ru has address 85.142.151.56
vlsu.ru mail is handled by 0 vlsu-ru.mail.protection.outlook.com.
[studentlin@localhost ~]$ host mivlgu.ru
mivlgu.ru has address 185.61.25.134
mivlgu.ru mail is handled by 10 mail.mivlgu.ru.
[studentlin@localhost ~]$ host bav-service.ru
bav-service.ru has address 31.31.196.97
bav-service.ru mail is handled by 20 mx2.hosting.reg.ru.
bav-service.ru mail is handled by 10 mx1.hosting.reg.ru.
[studentlin@localhost ~]$ host ftp.bav-service.ru
ftp.bav-service.ru has address 185.61.24.210
```

Команда ifstat

Отображает статистику сетевого интерфейса.

```
[studentlin@localhost ~]$ ifstat
#kernel
Interface        RX Pkts/Rate    TX Pkts/Rate    RX Data/Rate    TX Data/Rate
                 RX Errs/Drop    TX Errs/Drop    RX Over/Rate    TX Coll/Rate
lo                     26 0            26 0            2188 0          2188 0
                        0 0             0 0               0 0             0 0
enp0s3               24253 0          4954 0         21103K 0         1548K 0
                        0 0             0 0               0 0             0 0
```

Вы можете использовать параметр «**-z**», чтобы очистить статистику и начать заново.

Команда ip addr

Используется для сетевых интерфейсов и маршрутизации. С помощью этой команды проверяется информация об интерфейсе.

```
[studentlin@localhost ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ab:63:f2 brd ff:ff:ff:ff:ff:ff
    inet 210.210.0.4/24 brd 210.210.0.255 scope global dynamic noprefixroute enp0s3
       valid_lft 41944sec preferred_lft 41944sec
    inet6 fe80::a00:27ff:feab:63f2/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Команда iptables

Команда **iptables** — это инструмент, который используется для фильтрации IP-пакетов. С помощью команды iptables мы можем настроить правила таблицы IP, и с помощью этих правил мы предотвращаем несанкционированный доступ к нашему устройству, сети. Команда iptables может использоваться с различными параметрами.

```
[root@localhost studentlin]# iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

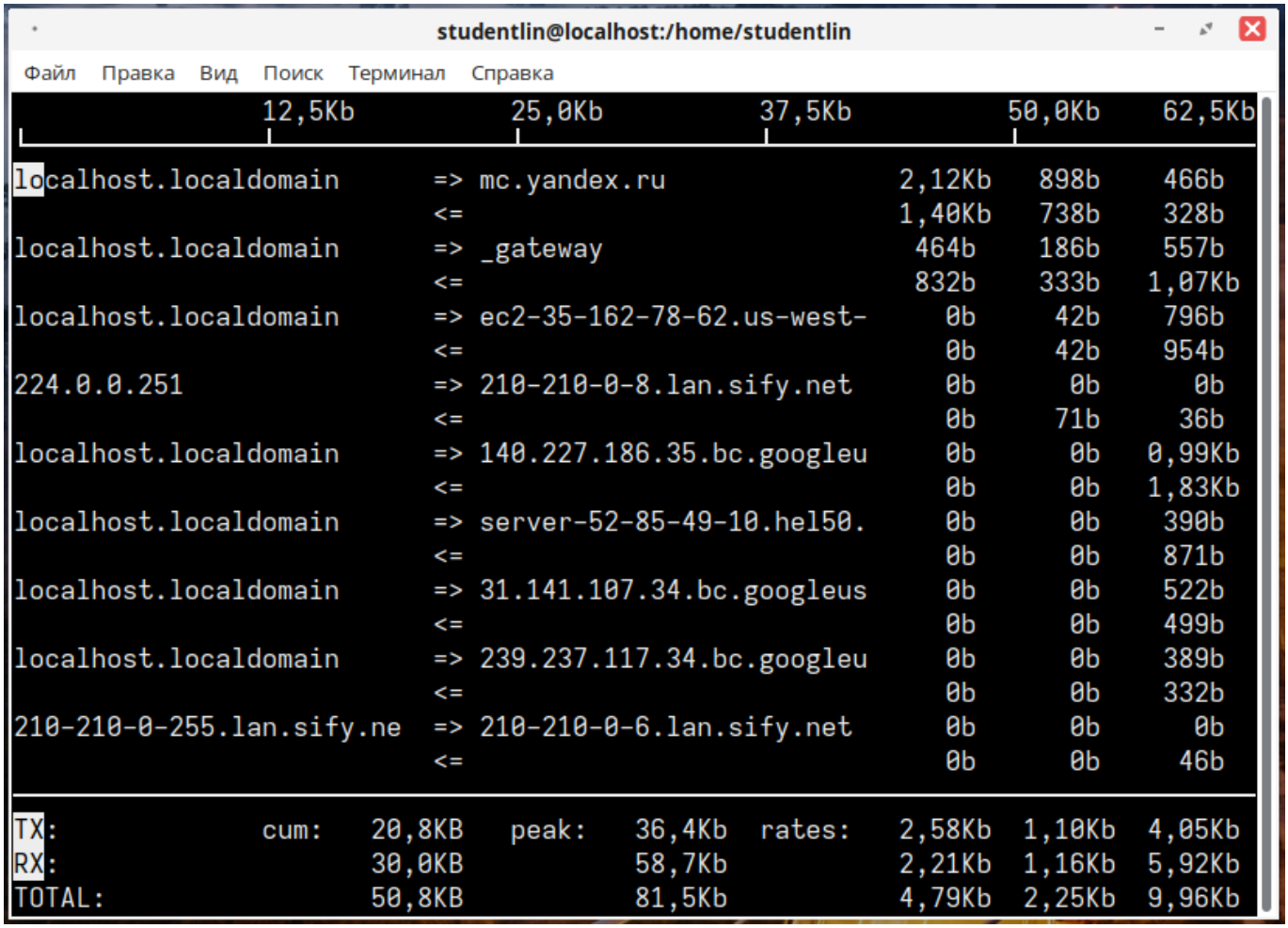
Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Команда iftop

Используется для отображения текущего использования полосы пропускания на сетевых интерфейсах. Вы также можете использовать эту команду для определенного интерфейса с параметром “**-i**” .

```
[root@localhost studentlin]# iftop
interface: enp0s3
IP address is: 210.210.0.4
MAC address is: 08:00:27:ab:63:f2
```



Команда ifup

На ваших серверах есть сетевые интерфейсы. Вы можете административно включить или отключить эти интерфейсы. Ключевые слова для этих заданий — «**ifup**» и «**ifdown**».

```
[root@localhost studentlin]# ifdown enp0s3
Соединение «enp0s3» успешно отключено (активный адрес D-Bus:
/org/freedesktop/NetworkManager/ActiveConnection/2)
[root@localhost studentlin]# ifup enp0s3
```

```
Соединение успешно активировано (адрес действующего D-Bus: /org/freedesktop/NetworkManager/ActiveConnection/3)
```

Команда netstat

Используется для проверки сетевых подключений

Без каких-либо опций netstat предоставляет информацию об открытом сокете. Но есть много вариантов netstat. Например, если мы используем «**netstat -r**», он дает нам информацию о таблице маршрутизации.

```
[studentlin@localhost ~]$ netstat > /home/studentlin/netstat.txt
```

Результат выполнения команды представлен в приложении 1.

```
[studentlin@localhost ~]$ netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
default         _gateway        0.0.0.0         UG        0 0          0 enp0s3
210.210.0.0     0.0.0.0         255.255.255.0   U         0 0          0 enp0s3
```

Команда nslookup

Предоставляет информацию о DNS-сервере.

```
[studentlin@localhost ~]$ nslookup vlsu.ru
Server:           210.210.0.1
Address:    210.210.0.1#53

Non-authoritative answer:
Name: vlsu.ru
Address: 85.142.151.56

[studentlin@localhost ~]$ nslookup mivlgu.ru
Server:           210.210.0.1
Address:    210.210.0.1#53

Non-authoritative answer:
Name: mivlgu.ru
Address: 185.61.25.134

[studentlin@localhost ~]$ nslookup amisod.ru
Server:           210.210.0.1
Address:    210.210.0.1#53
```

```
Non-authoritative answer:
Name: amisod.ru
Address: 31.31.196.97

[studentlin@localhost ~]$ nslookup bav-service.ru
Server:          210.210.0.1
Address:    210.210.0.1#53

Non-authoritative answer:
Name: bav-service.ru
Address: 31.31.196.97

[studentlin@localhost ~]$ nslookup ftp.bav-service.ru
Server:          210.210.0.1
Address:    210.210.0.1#53

Non-authoritative answer:
Name: ftp.bav-service.ru
Address: 185.61.24.210
```

Команда nmap

Показывает, какие услуги доступны и каковы их предложения, операционные системы, другие характеристики и т. д.

```
[studentlin@localhost ~]$ nmap vlsu.ru
Starting Nmap 7.80 ( https://nmap.org ) at 2022-12-04 15:19 MSK
Nmap scan report for vlsu.ru (85.142.151.56)
Host is up (0.020s latency).
Not shown: 997 filtered ports
PORT    STATE SERVICE
53/tcp  open  domain
80/tcp  open  http
443/tcp open  https

[studentlin@localhost ~]$ nmap amisod.ru
Starting Nmap 7.80 ( https://nmap.org ) at 2022-12-04 15:23 MSK
Nmap scan report for amisod.ru (31.31.196.97)
Host is up (0.039s latency).
rDNS record for 31.31.196.97: server206.hosting.reg.ru
Not shown: 979 closed ports
PORT     STATE    SERVICE
```

```
21/tcp   open     ftp
22/tcp   open     ssh
25/tcp   open     smtp
53/tcp   open     domain
80/tcp   open     http
110/tcp  open     pop3
111/tcp  open     rpcbind
135/tcp  filtered msrpc
139/tcp  filtered netbios-ssn
143/tcp  open     imap
443/tcp  open     https
445/tcp  filtered microsoft-ds
465/tcp  open     smtps
587/tcp  open     submission
993/tcp  open     imaps
995/tcp  open     pop3s
1500/tcp open     vlsi-lm
3306/tcp open     mysql
8080/tcp filtered http-proxy
8081/tcp filtered blackice-icecap
9100/tcp filtered jetdirect

Nmap done: 1 IP address (1 host up) scanned in 1.57 seconds
```

Команда ping

Команда «**ping**» — наиболее часто используемая команда в мире сетевых технологий.

```
[studentlin@localhost ~]$ ping mivlgu.ru
PING mivlgu.ru (185.61.25.134) 56(84) bytes of data.
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=1 ttl=61 time=3.02 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=2 ttl=61 time=2.83 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=3 ttl=61 time=2.88 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=4 ttl=61 time=2.91 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=5 ttl=61 time=2.57 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=6 ttl=61 time=3.27 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=7 ttl=61 time=2.94 ms
64 bytes from mivlgu.ru (185.61.25.134): icmp_seq=8 ttl=61 time=2.85 ms
^C
--- mivlgu.ru ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7009ms
```

```
rtt min/avg/max/mdev = 2.565/2.907/3.271/0.184 ms

[studentlin@localhost ~]$ ping amisod.ru
PING amisod.ru (31.31.196.97) 56(84) bytes of data.
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=1 ttl=55
time=16.5 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=2 ttl=55
time=16.5 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=3 ttl=55
time=46.7 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=4 ttl=55
time=16.9 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=5 ttl=55
time=17.2 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=6 ttl=55
time=16.6 ms
64 bytes from server206.hosting.reg.ru (31.31.196.97): icmp_seq=7 ttl=55
time=17.3 ms
^C
--- amisod.ru ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6010ms
rtt min/avg/max/mdev = 16.488/21.103/46.651/10.433 ms
```

Команда route

Чтобы проверить таблицу IP-маршрутизации в Linux, мы используем команду «**route**». В этих таблицах вы можете увидеть все определенные и изученные маршруты к любому пункту назначения.

```
[studentlin@localhost ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         _gateway        0.0.0.0         UG    100    0        0 enp0s3
210.210.0.0     0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
```

Команда ss

Команда **ss** дает подробную информацию о сокетах.

Опцию «**-l**» для вывода списка прослушивающих сокетов.

Опцию «**-t**» только для **tcp-соединений**.

```
[studentlin@localhost ~]$ ss -t
State   Recv-Q    Send-Q       Local Address:Port          Peer Address:Port
ESTAB           0                0                             210.210.0.4:40400
87.250.250.119:https
ESTAB           0                0                             210.210.0.4:52390
44.236.232.139:https
ESTAB           0                0                             210.210.0.4:40942
149.154.167.50:https
```

Команда tcpdump

Команда **Tcpdump** — это наиболее часто используемая команда анализа и захвата в Linux. С помощью этого вывода команды вы можете видеть передачи **TCP** в своей сети.

```
[root@localhost studentlin]# tcpdump
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s3, link-type EN10MB (Ethernet), capture size 262144 bytes
15:44:43.467742 IP 210-210-0-6.lan.sify.net.49725 > 81.19.104.241.https: Flags
[P.], seq 406859092:406859122, ack 2241962767, win 511, length 30
15:44:43.469556 IP localhost.localdomain.48940 > _gateway.domain: 40700+ PTR?
241.104.19.81.in-addr.arpa. (44)
15:44:43.556010 IP 81.19.104.241.https > 210-210-0-6.lan.sify.net.49725: Flags
[.], ack 30, win 1044, length 0
15:44:43.792178 IP _gateway.domain > localhost.localdomain.48940: 40700
NXDomain 0/0/0 (44)
15:44:43.792846 IP localhost.localdomain.56246 > _gateway.domain: 46867+ PTR?
6.0.210.210.in-addr.arpa. (42)
15:44:43.818515 IP _gateway.domain > localhost.localdomain.56246: 46867 1/0/0
PTR 210-210-0-6.lan.sify.net. (80)
15:44:45.634335 ARP, Request who-has localhost.localdomain tell _gateway,
length 46
15:44:45.634359 ARP, Reply localhost.localdomain is-at 08:00:27:ab:63:f2 (oui
Unknown), length 28
15:44:45.638572 ARP, Request who-has localhost.localdomain tell _gateway,
length 46
15:44:45.638583 ARP, Reply localhost.localdomain is-at 08:00:27:ab:63:f2 (oui
Unknown), length 28
15:44:45.818945 IP 210-210-0-6.lan.sify.net.51095 >
s003.srv.iptv2022.com.https: Flags [P.], seq 3930087145:3930087989, ack 4011832930,
win 511, length 844
```

```
15:44:45.819908 IP localhost.localdomain.60100 > _gateway.domain: 41674+ PTR? 3.48.35.194.in-addr.arpa. (42)
15:44:45.855912 IP s003.srv.iptv2022.com.https > 210-210-0-6.lan.sify.net.51095: Flags [P.], seq 1:393, ack 844, win 128, length 392
15:44:45.858620 IP 210-210-0-6.lan.sify.net.51150 > 87.239.27.170.inetcom.ru.https: Flags [S], seq 3620940728, win 64240, options [mss 1460,nop,wscale 8,nop,nop,sackOK], length 0
```

Команда tshark

Используется для анализа пакетов, и более сильная команда, чем наиболее часто используемая команда **tcpdump**.

Чтобы захватить трафик на интерфейсе, мы можем использовать его с параметром «**-i**».

```
[root@localhost studentlin]# tshark > /home/studentlin/tshark-log.txt
Running as user "root" and group "root". This could be dangerous.
Capturing on 'enp0s3'
 ** (tshark:4630) 15:49:37.401321 [Main MESSAGE] -- Capture started.
 ** (tshark:4630) 15:49:37.401441 [Main MESSAGE] -- File: "/var/tmp/wireshark_enp0s3kHEcgO.pcapng"
```

Мы можем перечислить все интерфейсы, которые мы захватываем, с опцией «**-D**».

```
[root@localhost studentlin]# tshark -D
Running as user "root" and group "root". This could be dangerous.
1. enp0s3
2. lo (Loopback)
3. any
4. bluetooth-monitor
5. nflog
6. nfqueue
7. usbmon0
8. usbmon1
9. ciscodump (Cisco remote capture)
10. dpauxmon (DisplayPort AUX channel monitor capture)
11. sdjournal (systemd Journal Export)
12. sshdump (SSH remote capture)
13. udpdump (UDP Listener remote capture)
```

Команда tracepath

Определение хостов на пути к месту назначения.

```
[studentlin@localhost ~]$ tracepath mivlgu.ru
 1?: [LOCALHOST]                      pmtu 1500
 1:  _gateway                                              2.012ms
 1:  _gateway                                              2.499ms
 2:  _gateway                                              2.031ms pmtu 1400
 2:  10.128.0.1                                            3.949ms
 3:  185.61.25.1                                           3.592ms
 4:  mivlgu.ru                                             3.238ms reached
     Resume: pmtu 1400 hops 4 back 4
[studentlin@localhost ~]$ tracepath vlsu.ru
 1?: [LOCALHOST]                      pmtu 1500
 1:  _gateway                                              1.962ms
 1:  _gateway                                              1.989ms
 2:  _gateway                                              2.222ms pmtu 1400
 2:  10.128.0.1                                            4.029ms
 3:  185.61.25.1                                           2.848ms
 4:  nnd02.transtelecom.net                                5.906ms
 5:  msk06.transtelecom.net                               21.282ms asymm  6
 6:  megafon-gw.transtelecom.net                          21.941ms asymm  7
 7:  83.169.204.128                                       15.390ms asymm 10
 8:  188.170.163.102                                      25.923ms asymm  9
 9:  no reply
10:  188.170.163.102                                      25.139ms asymm  9
11:  no reply
12:  no reply
13:  no reply
14:  no reply
15:  no reply
16:  no reply
17:  no reply
18:  no reply
19:  no reply
20:  no reply
21:  no reply
22:  no reply
23:  no reply
24:  no reply
25:  no reply
26:  no reply
```

```
27:  no reply
28:  no reply
29:  no reply
30:  no reply
     Too many hops: pmtu 1400
     Resume: pmtu 1400
[studentlin@localhost ~]$ tracepath amisod.ru
 1?: [LOCALHOST]                      pmtu 1500
 1:  _gateway                                              2.276ms
 1:  _gateway                                              2.208ms
 2:  _gateway                                              1.623ms pmtu 1400
 2:  10.128.0.1                                            2.468ms
 3:  185.61.25.1                                           2.860ms
 4:  nnd02.transtelecom.net                                8.243ms
 5:  no reply
 6:  BlackList-gw.transtelecom.net                        20.630ms
 7:  BL-Servers-NNovgorod-gw.transtelecom.net             29.478ms asymm  6
 8:  no reply
 9:  EdinayaSet-gw.transtelecom.net                       31.708ms asymm  7
10:  no reply
11:  no reply
12:  server206.hosting.reg.ru                             27.910ms reached
     Resume: pmtu 1400 hops 12 back 10
```

Команда traceroute

Используется для проверки узлов на пути к месту назначения пакета данных.

```
[studentlin@localhost ~]$ traceroute mivlgu.ru
traceroute to mivlgu.ru (185.61.25.134), 30 hops max, 60 byte packets
 1  _gateway (210.210.0.1)  5.161 ms  5.135 ms  5.126 ms
 2  10.128.0.1 (10.128.0.1)  5.117 ms  5.110 ms  5.101 ms
 3  185.61.25.1 (185.61.25.1)  5.092 ms  5.084 ms  5.091 ms
 4  mivlgu.ru (185.61.25.134)  5.170 ms  5.162 ms  5.153 ms
[studentlin@localhost ~]$ traceroute vlsu.ru
traceroute to vlsu.ru (85.142.151.56), 30 hops max, 60 byte packets
 1  _gateway (210.210.0.1)  3.038 ms  3.013 ms  3.004 ms
 2  10.128.0.1 (10.128.0.1)  4.982 ms  4.974 ms  4.966 ms
 3  185.61.25.1 (185.61.25.1)  4.958 ms  4.949 ms  4.942 ms
 4  nnd02.transtelecom.net (188.43.7.222)  13.440 ms  13.433 ms  13.425 ms
 5  msk06.transtelecom.net (217.150.58.62)  19.082 ms  18.877 ms  22.379 ms
 6  megafon-gw.transtelecom.net (217.150.58.61)  19.063 ms  17.177 ms  17.142
ms
 7  83.169.204.180 (83.169.204.180)  17.096 ms  17.044 ms  20.641 ms
```

```
 8  188.170.163.102 (188.170.163.102)  20.632 ms *  30.395 ms
 9  * * *
10  * 188.170.163.102 (188.170.163.102)  25.598 ms  20.939 ms
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
[studentlin@localhost ~]$ traceroute amisod.ru
traceroute to amisod.ru (31.31.196.97), 30 hops max, 60 byte packets
 1  _gateway (210.210.0.1)  2.332 ms  5.192 ms  5.178 ms
 2  10.128.0.1 (10.128.0.1)  5.168 ms  5.156 ms  5.143 ms
 3  185.61.25.1 (185.61.25.1)  5.128 ms  5.117 ms  5.037 ms
 4  nnd02.transtelecom.net (188.43.10.190)  8.895 ms  8.878 ms  8.862 ms
 5  * * *
 6      BL-gw.transtelecom.net   (188.43.31.178)      15.987   ms   BlackList-
gw.transtelecom.net    (188.43.30.210)        17.953    ms    BL-Servers-NNovgorod-
gw.transtelecom.net (188.43.245.42)  27.607 ms
 7  BlackList-gw.transtelecom.net (188.43.30.129)  18.974 ms  25.899 ms  31.530
ms
 8  * * *
 9   EdinayaSet-gw.transtelecom.net  (188.43.19.221)   31.495  ms   21.725  ms
20.616 ms
10  * * *
11  * * *
12  server206.hosting.reg.ru (31.31.196.97)  16.917 ms  16.901 ms  11.703 ms
```

Команда whois

Команды **whois** проверяют базу данных whois и возвращают информацию об IP-адресе и домене.

```
[studentlin@localhost ~]$ whois vlsu.ru
% TCI Whois Service. Terms of use:
% https://tcinet.ru/documents/whois_ru_rf.pdf (in Russian)
% https://tcinet.ru/documents/whois_su.pdf (in Russian)

domain:        VLSU.RU
nserver:       ns1.vlsu.ru. 85.142.151.25
nserver:       ns2.vlsu.ru. 85.142.151.26
state:         REGISTERED, DELEGATED, VERIFIED
org:           Vladimir State University
taxpayer-id:   3327102091
registrar:     RU-CENTER-RU
admin-contact: https://www.nic.ru/whois
created:       2006-01-16T21:00:00Z
paid-till:     2023-01-16T21:00:00Z
free-date:     2023-02-17
source:        TCI

Last updated on 2022-12-04T13:01:30Z

[studentlin@localhost ~]$ whois mivlgu.ru
% TCI Whois Service. Terms of use:
% https://tcinet.ru/documents/whois_ru_rf.pdf (in Russian)
% https://tcinet.ru/documents/whois_su.pdf (in Russian)

domain:        MIVLGU.RU
nserver:       ns1.mivlgu.ru. 185.61.25.144
nserver:       ns2.mivlgu.ru. 185.61.25.134
state:         REGISTERED, DELEGATED, VERIFIED
org:           MUROMSKIY INSTITUTE VLGU
taxpayer-id:   3327102091
registrar:     RU-CENTER-RU
admin-contact: https://www.nic.ru/whois
created:       2002-10-24T20:00:00Z
paid-till:     2023-10-24T21:00:00Z
free-date:     2023-11-25
source:        TCI

Last updated on 2022-12-04T13:01:30Z

[studentlin@localhost ~]$ whois amisod.ru
```

```
% TCI Whois Service. Terms of use:
% https://tcinet.ru/documents/whois_ru_rf.pdf (in Russian)
% https://tcinet.ru/documents/whois_su.pdf (in Russian)

domain:         AMISOD.RU
nserver:        ns1.hosting.reg.ru.
nserver:        ns2.hosting.reg.ru.
state:          REGISTERED, DELEGATED, VERIFIED
person:         Private Person
registrar:      REGRU-RU
admin-contact: http://www.reg.ru/whois/admin_contact
created:        2010-12-23T14:45:45Z
paid-till:      2023-12-23T14:45:45Z
free-date:      2024-01-23
source:         TCI

Last updated on 2022-12-04T13:01:30Z
```

Лабораторная работа №3

Тема: Виртуальные частные сети

Цель: Изучить средства Linux обеспечивающие подключение к серверам виртуальных частных сетей и организация собственной VPN.

Задание

Работы выполняется подгруппами по 3 человека

Установление соединения PPTP

1) Вывести сведения о имеющихся в системе сетевых подключениях.

2) В соответствии с инструкцией https://redos.red-soft.ru/base/arm/arm-network/vpn-pptp-connect/?sphrase_id=95845 сконфигурировать подключение к VPN сети по следующим данным:

- Сервер подключения: ftp.bav-service.ru
- Пользователь: vpnuser
- Пароль: VpNUseR-20!22
- Использовать шифрование MPPE

3) Установить соединение каждому члену подгруппы.

4) Вывести сведения о имеющихся в системе сетевых подключениях.

5) Выполнить пингование компьютеров 2 других членов подгруппы.

Установление соединения L2TP / IPsec

1) Вывести сведения о имеющихся в системе сетевых подключениях.

2) В соответствии с инструкцией сконфигурировать подключение к VPN сети по следующим данным:

- Сервер подключения: ftp.bav-service.ru
- Пользователь: vpnuser
- Пароль: VpNUseR-20!22

– Общий ключ: ExtConnect+22#-BAV

– Алгоритмы фазы 1: aes256-sha2_256-modp2048,aes256-sha2_256-modp1536,aes256-sha1-modp2048,aes256-sha1-modp1536,aes256-sha1-ecp_384,aes128-sha1-ecp_256,3des-sha1-modp2048

3) Установить соединение каждому члену подгруппы.

4) Вывести сведения о имеющихся в системе сетевых подключениях.

5) Выполнить пингование компьютеров 2 других членов подгруппы.

Настройка OpenVPN сервера и клиентов

1) Вывести сведения о имеющихся в системе сетевых подключениях.

2) В соответствии с инструкцией https://redos.red-soft.ru/base/server-configuring/other-utilites/openvpn/?sphrase_id=95845 сконфигурировать сервер и 2 клиента OpenVPN.

3) Установить соединение клиентских машин с сервером.

4) Вывести сведения о имеющихся в системе сетевых подключениях.

5) Выполнить пингование сервера с клиентских машин и взаимное пингование клиентов.

Теоретическая часть

Настройка PPTP соединения.

Устанавливаем pptp.

```
# dnf install -y pptp
```

Включаем поддержку.

```
# modprobe ppp_mppe
```

Создаём файл с конфигом подключения, в котором сразу настраиваем автопереподключение при обрыве соединения.

```
# vi /etc/ppp/peers/pptpserver
pty "pptp 10.10.10.10 --nolaunchpppd"
name your_login
```

```
password your_password
remotename PPTP
require-mppe-128
persist holdoff 10
maxfail 0
lcp-echo-interval 15
lcp-echo-failure 4
Запускаем подключение
# pppd call pptpserver
```

Проверяем, что появился интерфейс ppp0, введя команду ifconfig

Переходим в /etc/sysconfig/network-scripts

Создаём файл и заполняем туда следующие параметры:

```
# vi /etc/sysconfig/network-scripts/ifcfg-pptpserver
ONBOOT=yes
USERCTL=yes
TYPE=Modem
PEERDNS=yes
DEVICE=ppp0
DEFROUTE=no
```

Далее создаём файл с маршрутами, которые будут прописаны при подключении

```
# vi /etc/sysconfig/network-scripts/route-pptpserver
192.168.0.0/24 via 192.168.1.1
```

Установка соединения L2TP/IPSEC

Последовательность настройки любая, но алгоритм подключения будет следующий: сначала устанавливается шифрованное соединение IPsec, а в нём уже поднимается L2TP, поэтому ниже будет инструкция по настройке strongswan:

```
dnf install strongswan
```

Настройка IPsec (ikev1)

Использованы дефолтные и рекомендованные параметры в конфиге.

```
SERVER_IP=<IP-адрес сервера с IPsec>
/etc/strongswan/ipsec.conf

conn %default
  lifetime=1h
  margintime=9m
  rekeyfuzz = 100%
  keyingtries=1
  keyexchange=ikev1
  authby=secret
  ike=aes128-sha1-modp2048!
  esp=aes128-sha1-modp1024!

conn conn-ipsec
  keyexchange=ikev1
  #auto=add
  authby=secret
  type=transport
  forceencaps = yes
  leftprotoport=17/1701
  rightprotoport=17/1701
  right="$SERVER_IP"
  auto=route # необходим для автоподнятия ipsec
  dpdaction=restart
```

При настройке стоит обратить внимание на следующие параметры:

- **lifetime**, **margintime** и **rekeyfuzz** – параметры для времени переподключения, т.к. используется IKEv1, где reauth происходит всегда. На клиенте и сервере lifetime могут быть разными, но лучше устанавливать всё одинаково во избежание несогласованности.

- **right** – обязательно указывать IP вместо DNS, иначе соединение не установится, т.к. не сможет отрезолвить домен
- **auto** и **dpdaction** – очень важны для поднятия соединения в случае реконнекта
- **ike** & **esp** – установить шифры, которые указаны на сервере
- forceencaps = yes – без этого параметра на Debian 11 туннель ipsec отваливался

С используемыми шифрами ike и esp стоит поиграться и выбрать для себя наиболее оптимальные по части безопасности и скорости работы, не стоит копировать бездумно.

Далее указывается секретный ключ IPsec:

```
/etc/strongswan/ipsec.secrets
```

: PSK <суперсекретный ключ на Mikrotik>

На этом настройка закончена и можно переходить к xl2tpd.

Настройка L2TP

```
yum install xl2tpd
/etc/xl2tpd/xl2tpd.conf
```

```
[lac conn-l2tp]
lns = "$SERVER_IP"
ppp debug = yes
pppoptfile = /etc/ppp/options.l2tpd.client
length bit = yes
redial = yes
redial timeout = 10
autodial = yes
```

- # redial, redial timeout и autodial – опять же для реконнекта
- # lac – имя соединения для его установления
- # lns – адрес сервера, тут можно указать DNS-адрес

```
/etc/ppp/options.l2tpd.client

ipcp-accept-local
ipcp-accept-remote
refuse-eap
require-mschap-v2
noccp
noauth
mtu 1450
mru 1450
noipdefault
connect-delay 5000
name CLIENT_LOGIN
password CLIENT_PASSWORD
```

- # require-mschap-v2 – установить, если используется mschap-v2 в настройках L2TP сервера на Mikrotik
- name & password – логин\пароль профайла на Mikrotik

Подключение

Создается директория для PID файла и управления xl2tpd:

```
mkdir -p /var/run/xl2tpd && touch /var/run/xl2tpd/l2tp-control
```

Перезагружаются сервисы:

```
systemctl restart strongswan && systemctl restart xl2tpd
```

Запуск IPsec:

```
strongswan up conn-ipsec
```

Запуск L2TP:

```
echo "c conn-l2tp" > /var/run/xl2tpd/l2tp-control
```

Теперь можно проверить, что соединение установилось, появился интерфейс ppp с клиентским IP: `ip a`

```
    ppp0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1450
qdisc pfifo_fast state UNKNOWN group default qlen 3
        link/ppp
        inet 10.0.10.2 peer 10.0.10.25/32 scope global ppp0
           valid_lft forever preferred_lft forever
```

Настройка маршрутов

Соединение есть, но для доступа к нужным подсетям по этому соединению необходимо прописать необходимые маршруты. По-хорошему, для автоматического добавления маршрутов в подсети офиса в скрипте /etc/ppp/ip-up в конец файла прописано несколько маршрутов. Несмотря на то, что в этом файле сказано, что добавлять файлы нужно в /etc/ppp/ip-up.local, такой вариант не сработал, поэтому маршруты добавляются в ip-up прямо в конце файла:

```
    /etc/ppp/ip-up

    #!/bin/bash
    # This file should not be modified -- make local changes
to
    # /etc/ppp/ip-up.local instead

    PATH=/sbin:/usr/sbin:/bin:/usr/bin
    export PATH

    LOGDEVICE=$6
    REALDEVICE=$1

    [ -f /etc/sysconfig/network-scripts/ifcfg-${LOGDEVICE} ]
&&   /etc/sysconfig/network-scripts/ifup-post   --realdevice
${REALDEVICE} ifcfg-${LOGDEVICE}
```

```
/etc/ppp/ip-up.ipv6to4 ${LOGDEVICE}

[ -x /etc/ppp/ip-up.local ] && /etc/ppp/ip-up.local "$@"

if [ $5 = "10.0.10.25" ]
then
ip route add 192.168.34.0/24 via 10.0.10.2
ip route add 10.10.12.0/24 via 10.0.10.2
ip route add 192.168.30.0/24 via 10.0.10.2
ip route add 192.168.178.0/24 via 10.0.10.2
ip route add 192.168.79.0/24 via 10.0.10.2
fi

exit 0
```

Когда ppp связь установлена, этот скрипт вызывается со следующими параметрами:

$1 - имя интерфейса, используемое pppd (напр. ppp3)

$2 - имя устройства tty

$3 - скорость устройства tty

$4 - локальный IP адрес для интерфейса

$5 - удаленный IP адрес

$6 - параметр, указанный опцией 'ipparam' в pppd

Ручное управление

На данном этапе L2TP и IPsec поднимаются автоматически в случае рестарта машины или отвалившегося интернета, но если вдруг возникнет необходимость вручную управлять соединениями, то нужно выполнить следующее для остановки L2TP:

Остановка L2TP:

```
echo "d conn-l2tp" > /var/run/xl2tpd/l2tp-control
```

Остановка IPsec:

```
strongswan down conn-ipsec
```

Или же воспользоваться командами systemd по перезагрузке сервисов.

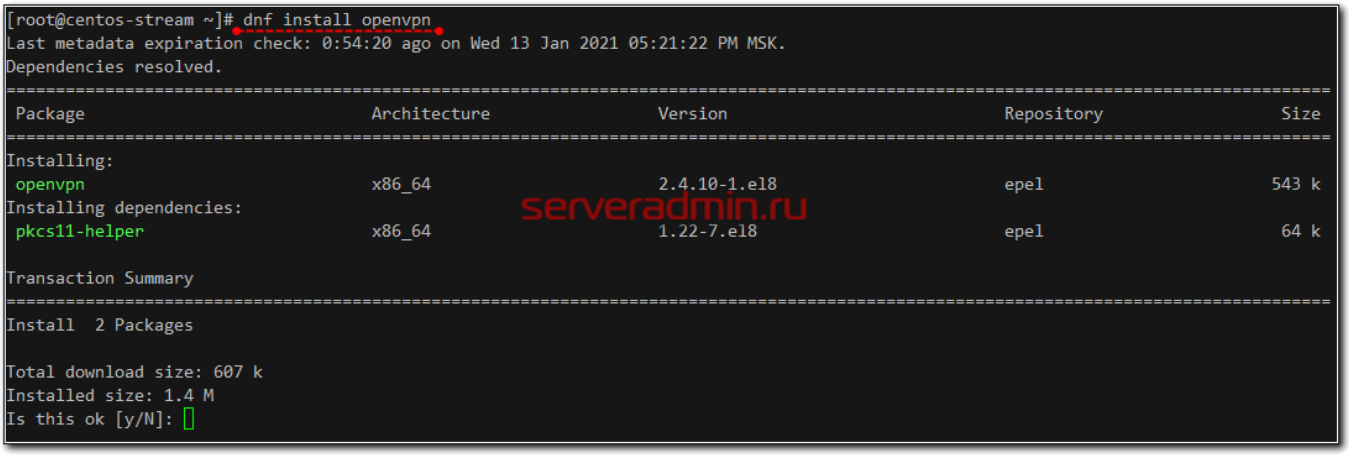
Установка и настройка OpenVPN

Устанавливаем openvpn на оба наших сервера, которые являются шлюзами в своих сегментах сети. Первым делом подключаем репозиторий epel к centos:

```
# yum install epel-release
```

Выполняем непосредственно установку:

```
# yum install openvpn
```



Создание сертификатов

Для работы сервера openvpn необходимо создать соответствующие сертификаты. Для этого воспользуемся программой Easy-RSA, которая генерирует сертификаты с помощью утилиты openssl. Все работы в этом разделе выполняем только на centos-server.

Создаем директорию для ключей:

```
# mkdir /etc/openvpn/keys
```

Скачиваем и устанавливаем утилиту Easy-RSA. Ссылку на последнюю версию можно посмотреть на github.

```
# cd /etc/openvpn/keys
```

```
# wget https://github.com/OpenVPN/easy-rsa/releases/download/v3.0.8/EasyRSA-3.0.8.tgz
```

Если получаете сообщение:

-bash: wget: command not found

То устанавливаете **wget**:

```
# yum install wget

# tar xzvf EasyRSA-3.0.8.tgz && mv EasyRSA-3.0.8 easyrsa && rm -f EasyRSA-3.0.8.tgz
# cd /etc/openvpn/keys/easyrsa
```

Создаем структуру публичных PKI ключей:

```
# mv vars.example vars
# ./easyrsa init-pki
Note: using Easy-RSA configuration from: /etc/openvpn/keys/easyrsa/vars

init-pki complete; you may now create a CA or requests.
Your newly created PKI dir is: /etc/openvpn/keys/easyrsa/pki
```

Создайте удостоверяющий центр CA:

```
# ./easyrsa build-ca
Note: using Easy-RSA configuration from: /etc/openvpn/keys/easyrsa/vars
Using SSL: openssl OpenSSL 1.1.1g FIPS  21 Apr 2020

Enter New CA Key Passphrase:
Re-Enter New CA Key Passphrase:
Generating RSA private key, 2048 bit long modulus (2 primes)
.....................+++++
.........................................+++++
```

e is 65537 (0x010001)

You are about to be asked to enter information that will be incorporated

into your certificate request.

What you are about to enter is what is called a Distinguished Name or a DN.

There are quite a few fields but you can leave some blank

For some fields there will be a default value,

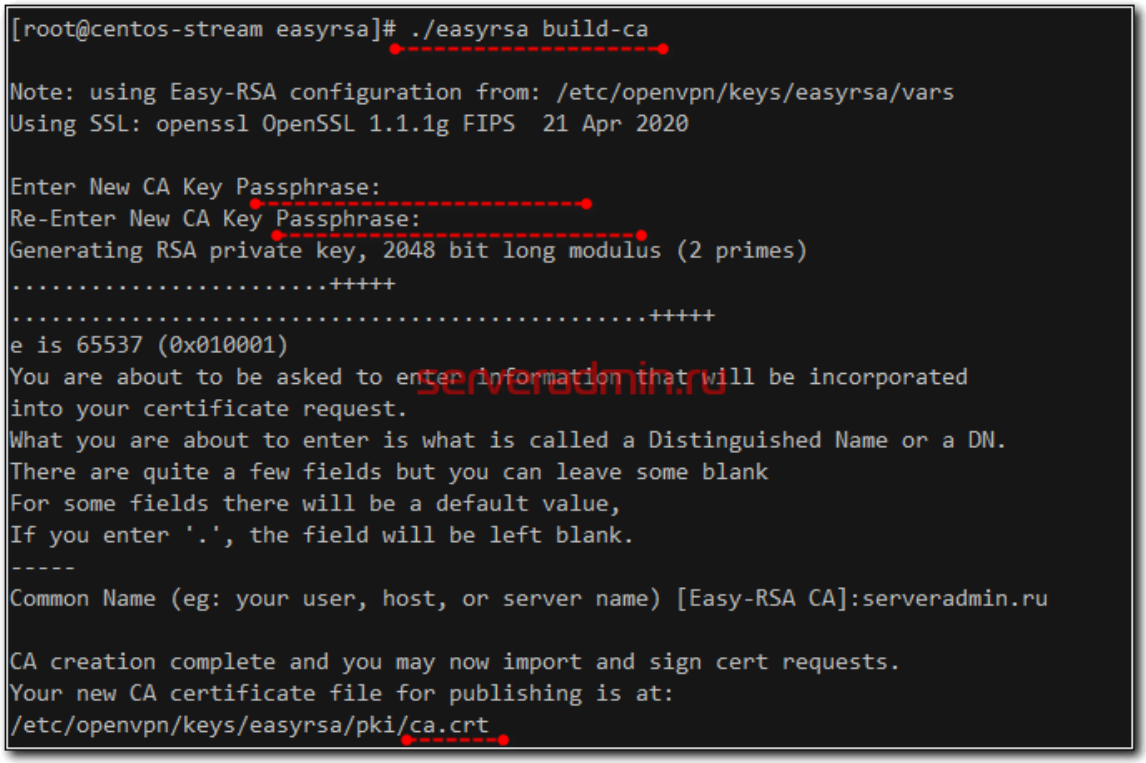
If you enter '.', the field will be left blank.

Common Name (eg: your user, host, or server name) [Easy-RSA CA]:serveradmin.ru

CA creation complete and you may now import and sign cert requests.

Your new CA certificate file for publishing is at:

/etc/openvpn/keys/easyrsa/pki/ca.crt



Не забудьте указанный пароль. Его нужно будет вводить каждый раз при создании нового сертификата openvpn.

Мы получили 2 файла:

- /etc/openvpn/keys/easyrsa/pki/private/**ca.key**
- /etc/openvpn/keys/easyrsa/pki/**ca.crt**

Первый ключ секретный, его нужно оставить на сервере и никому не отдавать. Второй - открытый, его мы будем вместе с пользовательскими сертификатами передавать клиентам.

Создаем запрос сертификата для сервера без пароля с помощью опции **nopass**, иначе придется вводить пароль с консоли при каждом запуске сервера:

```
# ./easyrsa gen-req server nopass

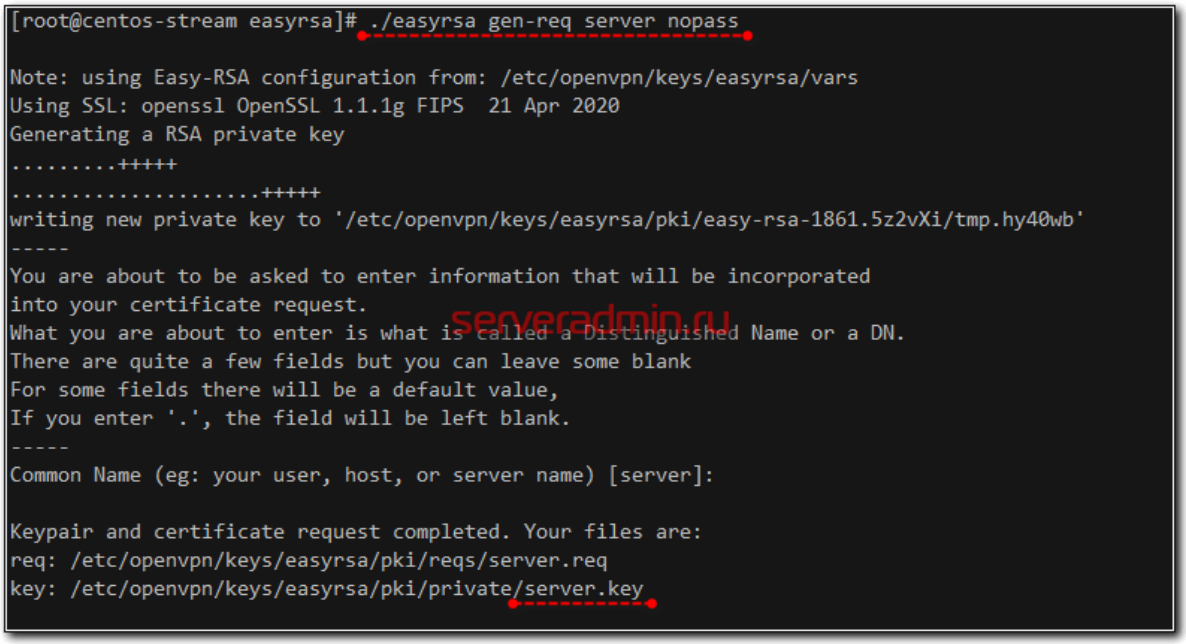
Note: using Easy-RSA configuration from: /etc/openvpn/keys/easyrsa/vars
Using SSL: openssl OpenSSL 1.1.1g FIPS  21 Apr 2020
Generating a RSA private key
.........+++++
.....................+++++
writing new private key to '/etc/openvpn/keys/easyrsa/pki/easy-rsa-1861.5z2vXi/tmp.hy40wb'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
```

Common Name (eg: your user, host, or server name) [server]:

Keypair and certificate request completed. Your files are:

req: /etc/openvpn/keys/easyrsa/pki/reqs/server.req

key: /etc/openvpn/keys/easyrsa/pki/private/server.key



Подписываем запрос на получение сертификата у нашего CA:

./easyrsa sign-req server server

Note: using Easy-RSA configuration from: /etc/openvpn/keys/easyrsa/vars

Using SSL: openssl OpenSSL 1.1.1g FIPS 21 Apr 2020

You are about to sign the following certificate.

Please check over the details shown below for accuracy. Note that this request

has not been cryptographically verified. Please be sure it came from a trusted

source or that you have verified the request checksum with the sender.

Request subject, to be signed as a server certificate for 825 days:

subject=

commonName = server

Type the word 'yes' to continue, or any other input to abort.

Confirm request details: yes

Using configuration from /etc/openvpn/keys/easyrsa/pki/easy-rsa-1888.qaUUvJ/tmp.Y4X2zV

Enter pass phrase for /etc/openvpn/keys/easyrsa/pki/private/ca.key:

Check that the request matches the signature

Signature ok

The Subject's Distinguished Name is as follows

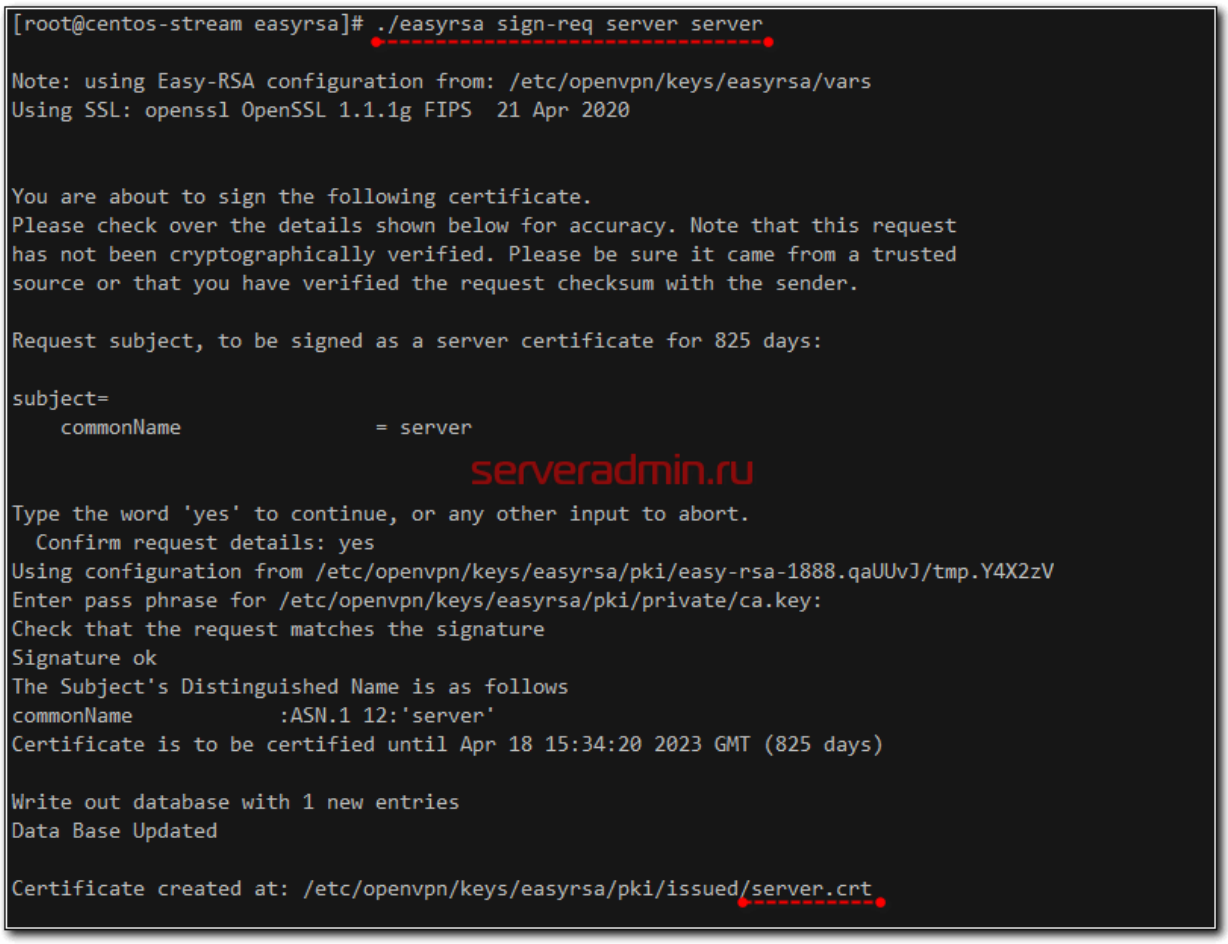
commonName :ASN.1 12:'server'

Certificate is to be certified until Apr 18 15:34:20 2023 GMT (825 days)

Write out database with 1 new entries

Data Base Updated

Certificate created at: /etc/openvpn/keys/easyrsa/pki/issued/server.crt



В процессе работы скрипта вводим пароль от CA, который указывали раньше и отвечаем на вопрос yes. Мы получили подписанный удостоверяющим центром сертификат для сервера - */etc/openvpn/keys/easyrsa/pki/issued/**server.crt***

Нам еще пригодится ключ Диффи-Хелмана, генерируем его:

```
# ./easyrsa gen-dh
```

По завершению работы скрипта получаем файл dh сертификата - */etc/openvpn/keys/easy-rsa/pki/**dh.pem***.

Копируем в папку */etc/openvpn/server/* все необходимые для работы openvpn сервера файлы:

```
# cp pki/ca.crt /etc/openvpn/server/ca.crt
# cp pki/dh.pem /etc/openvpn/server/dh.pem
# cp pki/issued/server.crt /etc/openvpn/server/server.crt
# cp pki/private/server.key /etc/openvpn/server/server.key
```

Так же сгенерируем еще один ключ, который на пригодится для шифрования TLS control channel, что повышает безопасность.

```
# openvpn --genkey --secret /etc/openvpn/server/tc.key
```

Создадим ключ для клиента openvpn:

```
# ./easyrsa gen-req client01 nopass
# ./easyrsa sign-req client client01
```

Процедура аналогична созданию сертификата для сервера. Так же вводим пароль, отвечаем yes. В результате получаем подписанный сертификат клиента:

- /etc/openvpn/keys/easy-rsa/pki/issued/**client01.crt**
- /etc/openvpn/keys/easy-rsa/pki/private/**client01.key**

Команды gen-req и sign-req можно объединить в одну с помощью **build-client-full** примерно так:

```
# ./easyrsa build-client-full client01 nopass
```

Клиенту, которым у нас является шлюз филиала, нужно будет передать следующий набор файлов - **client01.crt, client01.key, ca.crt, tc.key**. Либо использовать единый файл конфигурации клиента, который включает в себя в том числе и сертификаты. Я отдельно покажу как это сделать.

Теперь приступаем к настройке. Создаем файл конфигурации openvpn:

```
# mcedit /etc/openvpn/server/server.conf
port 13555 # я предпочитаю использовать нестандартные порты для работы
proto udp # протокол может быт и tcp, если есть необходимость в этом
dev tun

ca /etc/openvpn/server/ca.crt
cert /etc/openvpn/server/server.crt
key /etc/openvpn/server/server.key
dh /etc/openvpn/server/dh.pem
```

```
    auth SHA256 # выбор алгоритма шифрования для аутентификации
    cipher AES-256-CBC # выбор алгоритма шифрования всех передаваемых через vpn данных
    tls-version-min 1.2 # ограничиваем минимальную версию протокола tls
    tls-crypt tc.key # ключ для шифрования TLS control channel

    server 10.0.0.0 255.255.255.0 # подсеть для туннеля, может быть любой
    route 192.168.150.0 255.255.255.0 # указываем подсеть, к которой будем обращаться через vpn

    ifconfig-pool-persist ipp.txt # файл с записями соответствий clinet - ip
    client-to-client # позволяет клиентам openvpn подключаться друг к другу
    client-config-dir /etc/openvpn/ccd # директория с индивидуальными настройками клиентов

    keepalive 10 120
    comp-lzo # если используется протокол tcp, сжатие надо отключить
    explicit-exit-notify 1 # если используется протокол tcp, поменять на 0
    persist-key
    persist-tun
    status /var/log/openvpn/openvpn-status.log
    log /var/log/openvpn/openvpn.log
    user nobody
```

```
group nobody
verb 3
```

Подробное описание всех параметров можно посмотреть в официальной wiki.

Создаем необходимые директории:

```
# mkdir /etc/openvpn/ccd && mkdir /var/log/openvpn
```

Создаем файл конфигурации клиента в папке, указанной в параметре client-config-dir :

```
mcedit /etc/openvpn/ccd/client01
iroute 192.168.150.0 255.255.255.0
```

Здесь **client01** - имя сертификата пользователя. Параметр **iroute** означает, что за подсеть 192.168.150.0/24 отвечает именно этот клиент. Если в openvpn не передать эту настройку, то сеть, находящуюся за клиентом, будет не видно, при этом сам клиент будет видеть всю сеть, которую обслуживает сервер. Такой вариант подходит для подключения удаленных сотрудников.

Выбор устройства openvpn - TAP или TUN

Чуть подробнее остановлюсь на этом моменте. В моей конфигурации я использую **tun** интерфейс. В чем отличие tun от tap можно прочитать на википедии. Прокомментирую своими словами. Если вам нужно объединить две разные локальные сети в одну условно общую, но с разной адресацией, то вам нужен tun. То есть в нашем случае мы объединяем две сети 192.168.220.0/24 и 192.168.150.0/24 для взаимного совместного доступа.

Если же у вас стоит задача объединить 2 удаленные сети в единое адресное пространство, например сделать и в офисе, и в филиале единую сеть 192.168.220.0/24, то тогда бы мы использовали **tap** интерфейс и указывали бы на компьютерах в обоих сетях не пересекающиеся адреса из одной подсети. То есть оба офиса окажутся в одном широковещательном домене и смогут передавать данные с помощью широковещания на канальном уровне сетевой модели OSI. В таком состоянии openvpn работает в режиме моста. По мне так удобнее первый вариант. Я еще не сталкивался с задачей, где бы мне нужен был tap. Вернемся к настройке.

Запускаем сервер:

```
# systemctl start openvpn-server@server.service
```

Если сервер не запустился, а в логе ошибка:

TCP/UDP: Socket bind failed on local address [undef]: Permission denied

Значит вам нужно либо правильно настроить, либо отключить SELinux. В данном материале я не хочу касаться настройки SELinux, поэтому просто отключаем его:

```
# mcedit /etc/sysconfig/selinux
```

меняем значение

```
SELINUX=disabled
```

Чтобы изменения вступили в силу, выполняет:

```
# setenforce 0
```

Пробуем снова запустить openvpn сервер:

```
# systemctl start openvpn-server@server.service
```

Если опять видите какие-то ошибки, то проверяйте лог файлы */var/log/openvpn/openvpn.log* и */var/log/messages*. Там почти наверняка будет указано, в чем проблема. Проверяем, запустился сервер или нет:

```
# netstat -tulnp | grep 13555

udp    0    0 0.0.0.0:13555    0.0.0.0:*  1900/openvpn
```

Отлично, запустился на указанном порту.

Добавляем сервер openvpn в автозагрузку:

```
# systemctl enable openvpn-server@server.service
```

Created symlink /etc/systemd/system/multi-user.target.wants/openvpn-server@server.service → /usr/lib/systemd/system/openvpn-server@.service.

Теперь внимательно проверим корректность всех настроек на сервере. Сначала посмотрим информацию о сетевых интерфейсах:

```
[root@centos-vpn-server server]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fe:c1:ff:11:58:9d brd ff:ff:ff:ff:ff:ff
    inet 10.20.1.38/24 brd 10.20.1.255 scope global dynamic noprefixroute ens18
       valid_lft 31535047sec preferred_lft 31535047sec
    inet6 fe80::80b1:317b:9b9d:7a37/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 6e:11:51:1b:fc:5e brd ff:ff:ff:ff:ff:ff
    inet 192.168.220.1/24 brd 192.168.220.255 scope global noprefixroute ens19
       valid_lft forever preferred_lft forever
5: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 100
    link/none
    inet 10.0.0.1 peer 10.0.0.2/32 scope global tun0
       valid_lft forever preferred_lft forever
    inet6 fe80::a736:a1ce:80bc:7489/64 scope link stable-privacy
       valid_lft forever preferred_lft forever
[root@centos-vpn-server server]#
```

serveradmin.ru

Обращаем внимание на адреса туннеля vpn. Теперь проверяем статические маршруты:

```
[root@centos-vpn-server server]# netstat -nr
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0         10.20.1.1       0.0.0.0         UG        0 0          0 ens18
10.0.0.0        10.0.0.2        255.255.255.0   UG        0 0          0 tun0
10.0.0.2        0.0.0.0         255.255.255.255 UH        0 0          0 tun0
10.20.1.0       0.0.0.0         255.255.255.0   U         0 0          0 ens18
192.168.150.0   10.0.0.2        255.255.255.0   UG        0 0          0 tun0
192.168.220.0   0.0.0.0         255.255.255.0   U         0 0          0 ens19
[root@centos-vpn-server server]#
```

serveradmin.ru

Тут тоже все в порядке. Траффик для подсети филиала 192.168.150.0/24 будет маршрутизироваться в тоннель. С настройкой сервера закончили, идем теперь на клиент.

Настройка firewall (iptables) для openvpn

Рассмотрю отдельно вопрос настройки iptables для работы openvpn. В целом, там нет никаких нюансов в самих правилах. Нам необходимо открыть входящий порт для подключения к серверу, разрешить транзитный трафик и включить nat для openvpn интерфейсов. Проблемы обычно бывают в реализации настроек самого firewall.

Если вы используете нативные iptables, то нужны будут следующие правила, разрешающие весь трафик в и из vpn туннелей.

```
iptables -A INPUT -i ens18 -p udp --dport 13555 -j ACCEPT
iptables -A INPUT -i tun+ -j ACCEPT
iptables -A OUTPUT -o tun+ -j ACCEPT
iptables -A FORWARD -i tun+ -j ACCEPT
$IPT -t nat -A POSTROUTING -s 10.0.0.0/24 -j MASQUERADE
```

Не забудьте разместить их выше правил блокировки, если таковые у вас имеются. Если вы используете firewalld для настройки фаервола, то набор правил для openvpn будет следующий:

```
firewall-cmd --add-port="13555"/"udp"
firewall-cmd --zone=trusted --add-source=10.0.0.0/24
firewall-cmd --permanent --add-port="13555"/"udp"
firewall-cmd --permanent --zone=trusted --add-source=10.0.0.0/24
firewall-cmd --direct --add-rule ipv4 nat POSTROUTING 0 -s 10.0.0.0/24 -j MASQUERADE
firewall-cmd --permanent --direct --add-rule ipv4 nat POSTROUTING 0 -s 10.0.0.0/24 -j MASQUERADE
```

Если нужно как-то сегментировать доступ в vpn сети, то настраивайте более точечно с использованием конкретных адресов и подсетей. Тонкая настройка правил iptables не тема данной статьи, поэтому я не буду на этом подробно останавливаться.

Сам я обычно использую нативные iptables и управляю ими с помощью скрипта. Более подробно я описываю настройку iptables в отдельной статье. В соответствии с этой статьей я использую следующий итоговый набор правил всего сервера для iptables.

```
#!/bin/bash

export IPT="iptables"

# Внешний интерфейс
export WAN=ens18
```

```
export WAN_IP=10.20.1.38

# Локальная сеть
export LAN1=ens19
export LAN1_IP_RANGE=192.168.220.1/24

# Очищаем правила
$IPT -F
$IPT -F -t nat
$IPT -F -t mangle
$IPT -X
$IPT -t nat -X
$IPT -t mangle -X

# Запрещаем все, что не разрешено
$IPT -P INPUT DROP
$IPT -P OUTPUT DROP
$IPT -P FORWARD DROP

# Разрешаем localhost и локалку
$IPT -A INPUT -i lo -j ACCEPT
$IPT -A INPUT -i $LAN1 -j ACCEPT
$IPT -A OUTPUT -o lo -j ACCEPT
$IPT -A OUTPUT -o $LAN1 -j ACCEPT

# Рзрешаем пинги
$IPT -A INPUT -p icmp --icmp-type echo-reply -j ACCEPT
$IPT -A INPUT -p icmp --icmp-type destination-unreachable -j ACCEPT
$IPT -A INPUT -p icmp --icmp-type time-exceeded -j ACCEPT
```

```
    $IPT -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

    # Разрешаем исходящие подключения сервера
    $IPT -A OUTPUT -o $WAN -j ACCEPT

    # разрешаем установленные подключения
    $IPT -A INPUT -p all -m state --state
ESTABLISHED,RELATED -j ACCEPT
    $IPT -A OUTPUT -p all -m state --state
ESTABLISHED,RELATED -j ACCEPT
    $IPT -A FORWARD -p all -m state --state
ESTABLISHED,RELATED -j ACCEPT

    # Отбрасываем неопознанные пакеты
    $IPT -A INPUT -m state --state INVALID -j DROP
    $IPT -A FORWARD -m state --state INVALID -j DROP

    # Отбрасываем нулевые пакеты
    $IPT -A INPUT -p tcp --tcp-flags ALL NONE -j DROP

    # Закрываемся от syn-flood атак
    $IPT -A INPUT -p tcp ! --syn -m state --state NEW -j
DROP
    $IPT -A OUTPUT -p tcp ! --syn -m state --state NEW -j
DROP

    # Разрешаем доступ из локалки наружу
    $IPT -A FORWARD -i $LAN1 -o $WAN -j ACCEPT
    # Закрываем доступ снаружи в локалку
    $IPT -A FORWARD -i $WAN -o $LAN1 -j REJECT
```

```
# Разрешаем OpenVPN
$IPT -A INPUT -i tun+ -j ACCEPT
$IPT -A OUTPUT -o tun+ -j ACCEPT
$IPT -A FORWARD -i tun+ -j ACCEPT
# Разрешаем доступ из внутренней сети в vpn
$IPT -A FORWARD -i $LAN1 -o tun+ -j ACCEPT

# Включаем NAT
$IPT -t nat -A POSTROUTING -s 10.0.0.0/24 -j MASQUERADE
$IPT -t nat -A POSTROUTING -o $WAN -s $LAN1_IP_RANGE -j
MASQUERADE

# открываем доступ к SSH
$IPT -A INPUT -i $WAN -p tcp --dport 22 -j ACCEPT

# Открываем порт для openvpn
$IPT -A INPUT -i $WAN -p udp --dport 13555 -j ACCEPT

# Сохраняем правила
/sbin/iptables-save  > /etc/sysconfig/iptables
```

Это минимальный набор правил для шлюза локальной сети с openvpn на борту. Про настройку шлюза на базе centos я так же рассказываю отдельно. Для сервера клиента openvpn набор правил будет точно такой же, только ip адреса, подсети и интерфейсы будут другие.

Настройка клиента openvpn

Устанавливаем openvpn:

```
# yum install openvpn
```

Создаем в директории */etc/openvpn/client/* файл конфигурации client.conf:

```
# mcedit /etc/openvpn/client/client.conf
dev tun
```

```
proto udp
remote 10.20.1.38 13555
client
resolv-retry infinite
ca /etc/openvpn/client/ca.crt
cert /etc/openvpn/client/client01.crt
key /etc/openvpn/client/client01.key
tls-crypt /etc/openvpn/client/tc.key
route 192.168.220.0 255.255.255.0
remote-cert-tls server
auth SHA256
cipher AES-256-CBC
persist-key
persist-tun
resolv-retry infinite
nobind
comp-lzo
verb 3
status /var/log/openvpn/openvpn-status.log 1
status-version 3
log-append /var/log/openvpn/openvpn-client.log
```

Не забываем скопировать в */etc/openvpn/client/* сохраненные ранее ключи ca.crt, client01.crt, client01.key и tc.key.

Обращаю внимание на параметр **route** в данном конфиге. Его можно здесь не указывать, сделав конфиг более унифицированным для множества клиентов. Вместо этого данную настройку можно передавать с сервера openvpn, указав в файле настроек клиента параметр **push route** следующим образом:

```
push "route 192.168.220.0 255.255.255.0"
```

Создаем каталог для логов:

```
# mkdir /var/log/openvpn
```

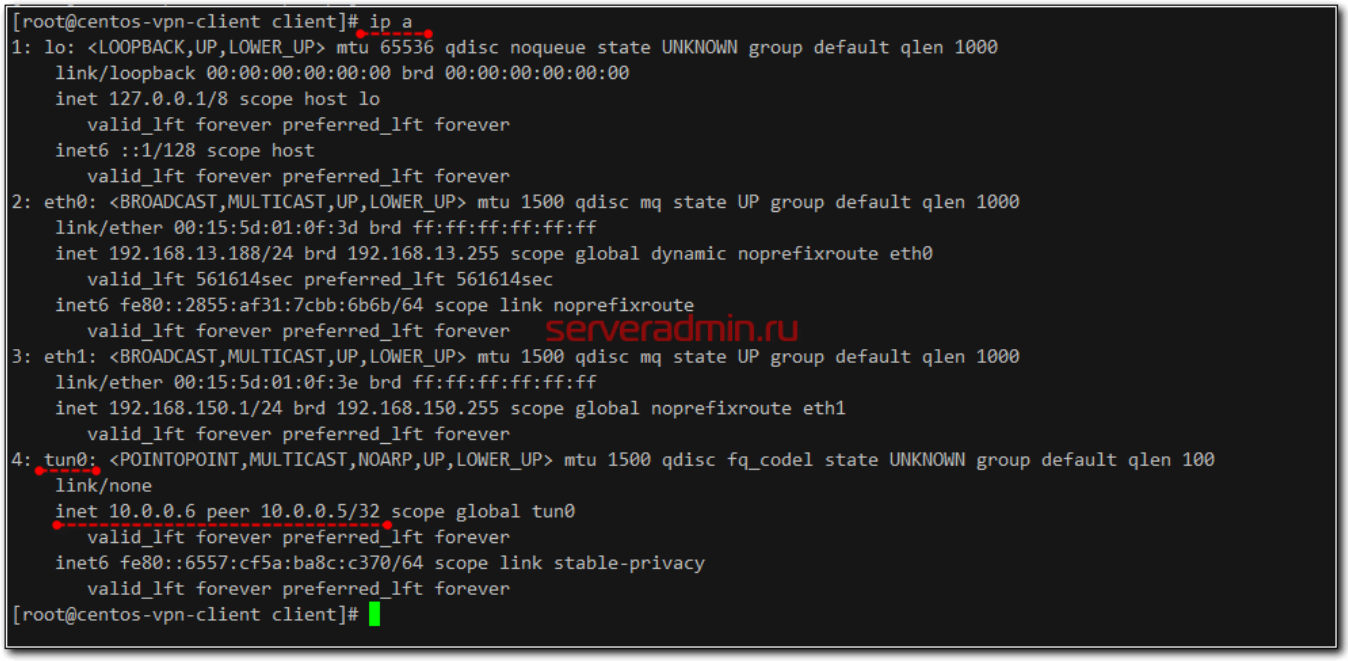
Запускаем openvpn client:

```
# systemctl start openvpn-client@client.service
```

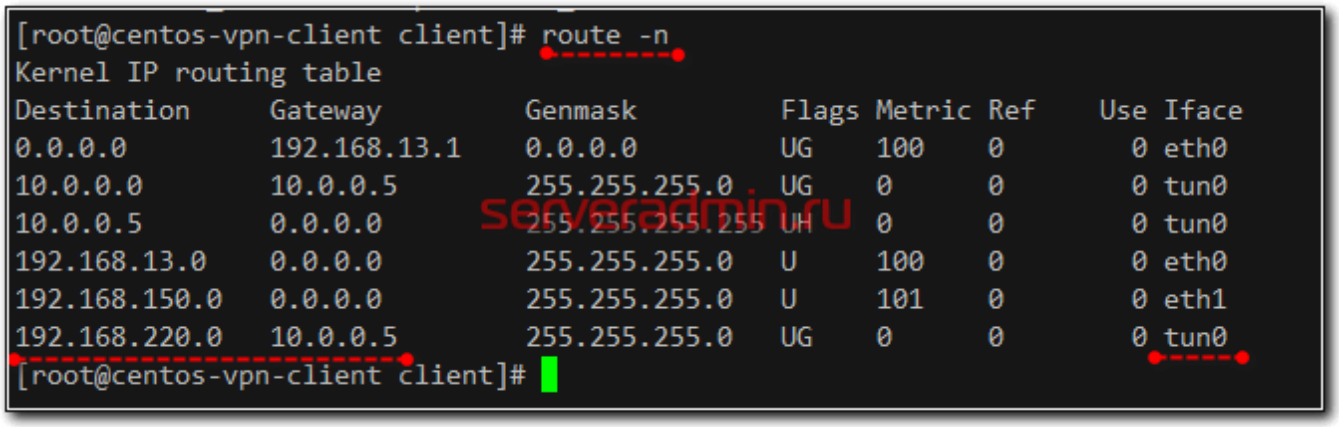
Добавляем в автозагрузку:

```
# systemctl enable openvpn-client@client.service
```

Теперь смотрим картину сетевых настроек на клиенте:



Ну и маршруты глянем заодно:



Все в порядке, подключение к vpn серверу есть, маршруты прописаны верно. Как вы видите, трафик к удаленной сети 192.168.220.0.24 пойдет через vpn туннель. Маршрут для него openvpn поднял автоматически в соответсвии со своими настройками на клиенте или сервере.

В принципе, этого уже достаточно, чтобы трафик забегал в обе стороны из одной подсети в другую. Но для этого у вас должны быть соответствующим образом настроены сами шлюзы, фаерволы на них, маскарадинг, в том числе на интерфейсе **tun**. Как это можно сделать, я приводил выше в примерах своих настроек iptables.

Перед этим напомню важный момент, если вдруг вы с нуля настраиваете шлюз и не проверили его в реальной работе. Для роутинга трафика между сетевыми интерфейсами, необходимо добавить строку:

```
net.ipv4.ip_forward = 1
```

в файл */etc/sysctl.conf* и применить настройку:

```
# sysctl -p
```

Если у вас этого не сделано, то трафик между интерфейсами ходить не будет.

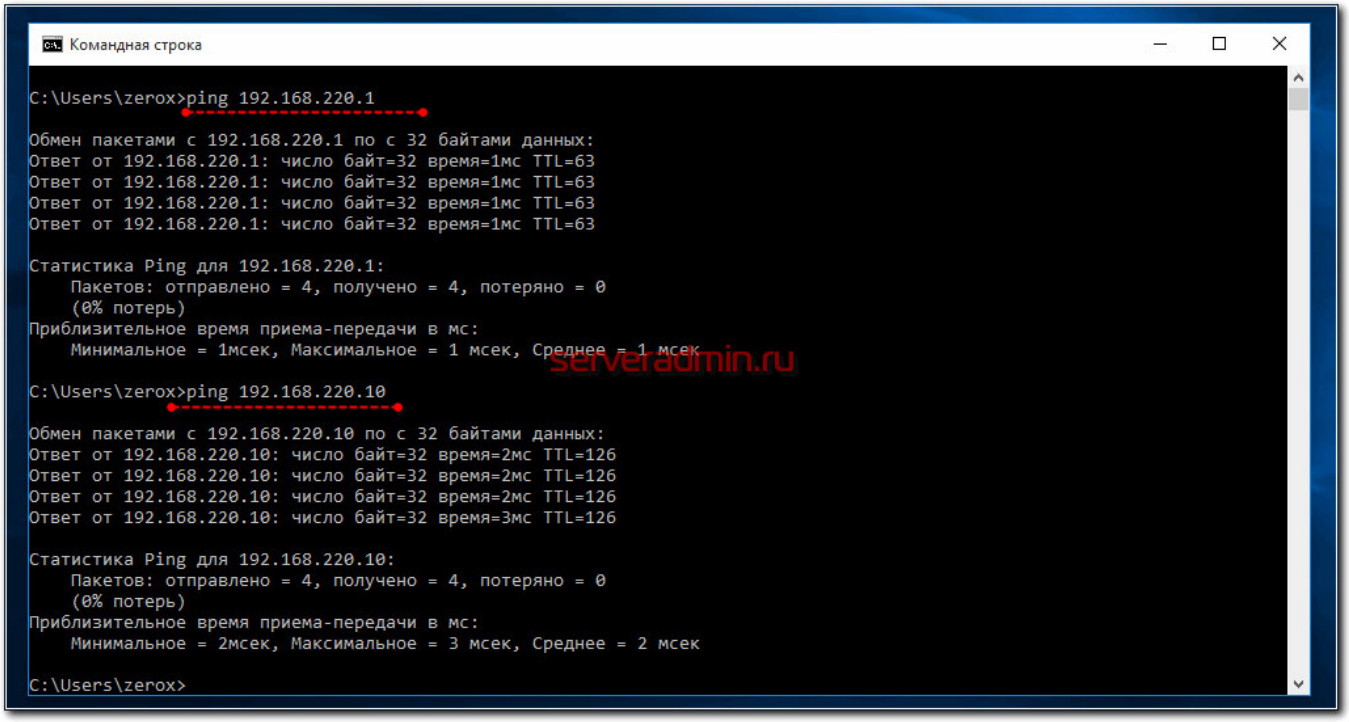
Если ваш openvpn клиент не подключается, то в первую очередь проверяйте настройки firewall - входящие подключения к серверу и исходящие клиента. Обратите внимание на номер порта и тип (TCP или UDP) если вы их меняли. У меня были затупы, когда никак не мог разобраться, почему нет соединения. Оказывалось, что я менял порт с UDP на TCP, но по привычке на фаерволе оставлял UDP.

Теперь давайте проверим, как бегают пакеты в нашей vpn сети. Заходим на centos-client (192.168.150.1) и пингуем centos-server (192.168.220.1) и pc1 (192.168.220.10):

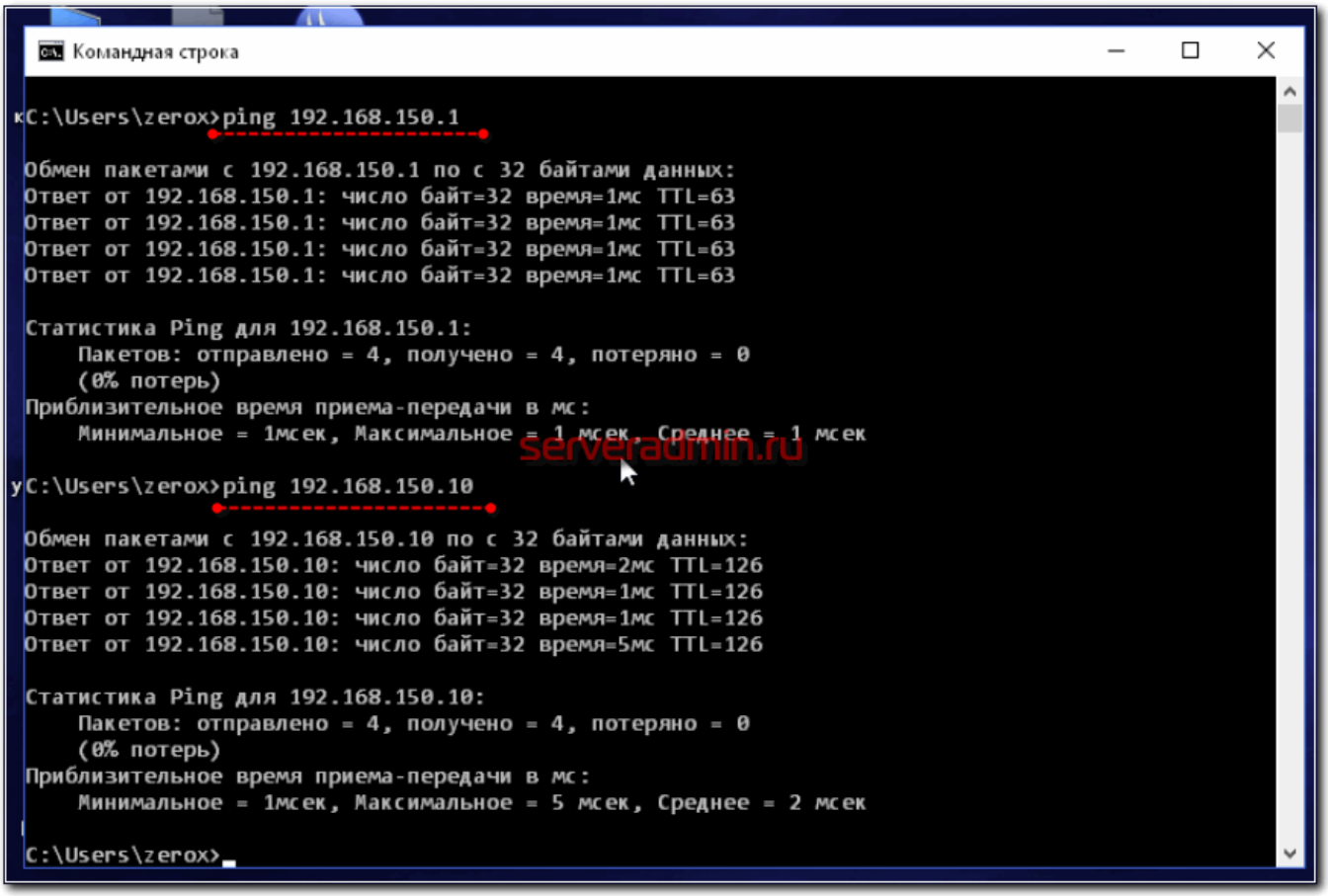
```
[root@centos-vpn-client ~]# ping 192.168.220.1
PING 192.168.220.1 (192.168.220.1) 56(84) bytes of data.
64 bytes from 192.168.220.1: icmp_seq=1 ttl=64 time=0.883 ms
64 bytes from 192.168.220.1: icmp_seq=2 ttl=64 time=1.44 ms
64 bytes from 192.168.220.1: icmp_seq=3 ttl=64 time=1.22 ms
64 bytes from 192.168.220.1: icmp_seq=4 ttl=64 time=1.12 ms
^C
--- 192.168.220.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 0.883/1.165/1.435/0.198 ms
[root@centos-vpn-client ~]# ping 192.168.220.10
PING 192.168.220.10 (192.168.220.10) 56(84) bytes of data.
64 bytes from 192.168.220.10: icmp_seq=1 ttl=127 time=1.59 ms
64 bytes from 192.168.220.10: icmp_seq=2 ttl=127 time=1.41 ms
64 bytes from 192.168.220.10: icmp_seq=3 ttl=127 time=1.46 ms
64 bytes from 192.168.220.10: icmp_seq=4 ttl=127 time=1.83 ms
^C
--- 192.168.220.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.410/1.571/1.830/0.167 ms
[root@centos-vpn-client ~]#
```

serveradmin.ru

Заходим на pc2 (192.168.150.10) и пингуем centos-server и pc1:



Теперь в обратную сторону. Заходим на pc1 (192.168.220.10) и пингуем centos-client (192.168.150.1) и pc2 (192.168.150.10):



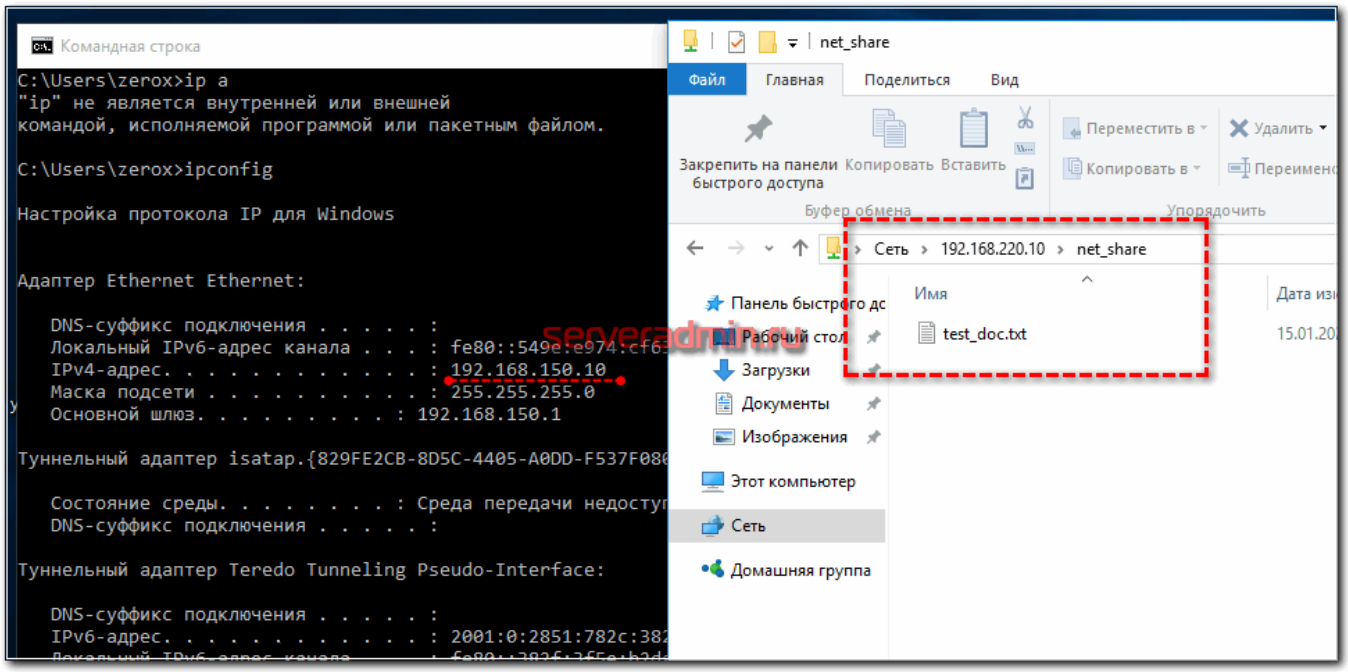
И напоследок пропингуем с vpn сервера подсеть клиента:

```
[root@centos-vpn-server openvpn]# ping 192.168.150.10
PING 192.168.150.10 (192.168.150.10) 56(84) bytes of data.
64 bytes from 192.168.150.10: icmp_seq=1 ttl=127 time=1.89 ms
64 bytes from 192.168.150.10: icmp_seq=2 ttl=127 time=1.72 ms
64 bytes from 192.168.150.10: icmp_seq=3 ttl=127 time=1.60 ms
^C
--- 192.168.150.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.602/1.736/1.890/0.123 ms
[root@centos-vpn-server openvpn]#
```

serveradmin.ru

Как видите, связь настроена в обе стороны. Если у вас что-то где-то не пингуется, то смотрите в первую очередь на наличие маршрутов на самих серверах с openvpn, а так же на настройки firewall. Проблемы могут быть только здесь. Ну и не забудьте настроить windows firewall на самих виндовых машинах. И помните про антивирусы на них. А то бывает кучу времени потратишь на поиск проблемы, а ее на самом деле не было. Пинги не работали, потому что тупо запрещены на виндовых машинах.

В завершение темы объединения удаленных офисов, проверим доступ к общим сетевым ресурсам. Расшарим папку на PC1 и зайдем на нее с PC2:



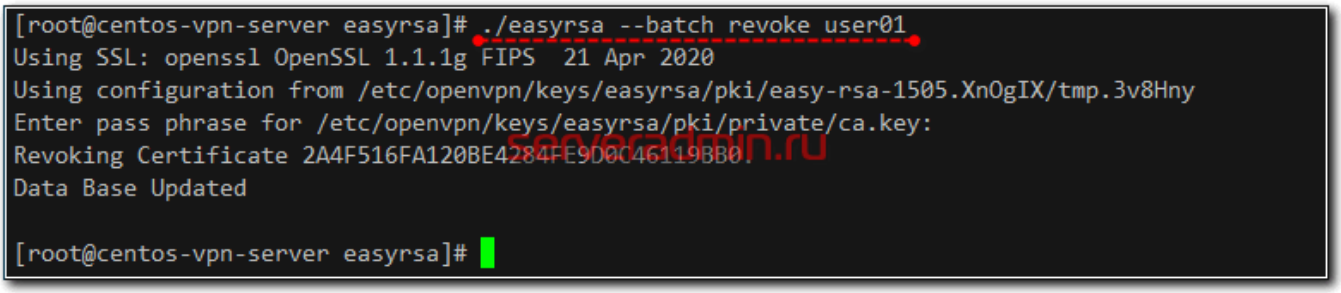
Все получилось. Мы реально объединили удаленные офисы в единую связную сеть с помощью openvpn сервера. Дальше поверх vpn можно настраивать работу остальных сервисов.

Отзыв сертификата клиента

В завершении расскажу об еще одной важной функциональности openvpn сервера - отзыв сертификатов пользователей. Делается это для того, чтобы запретить подключение конкретного пользователя к вашей vpn сети. Выполнить отзыв сертификата можно с помощью easyrsa следующим образом:

```
# ./easyrsa --batch revoke user01

Using SSL: openssl OpenSSL 1.1.1g FIPS  21 Apr 2020
Using configuration from /etc/openvpn/keys/easyrsa/pki/easy-rsa-1505.XnOgIX/tmp.3v8Hny
Enter pass phrase for /etc/openvpn/keys/easyrsa/pki/private/ca.key:
Revoking Certificate 2A4F516FA120BE4284FE9D0C46119BB0.
Data Base Updated
```



После отзыва необходимо сформировать файл со списком отозванных сертификатов. Это нужно будет делать каждый раз после отзыва.

```
# ./easyrsa gen-crl
```

Скопируем теперь этот файл в директорию с конфигом openvpn server.

```
# cp /etc/openvpn/keys/easyrsa/pki/crl.pem /etc/openvpn/server/
```

Если ранее у вас там уже был файл отозванных сертификатов, его надо будет либо удалить, либо перезаписать. Для того, чтобы сервер проверял отозванные сертификаты, в файл настроек server.conf нужно добавить соответствующий параметр.

```
crl-verify /etc/openvpn/server/crl.pem
```

Для применения настроек openvpn, сервер надо перезапустить. Если же у вас там уже был этот параметр и вы только обновили файл отозванных сертификатов, перезапускать сам сервер не обязательно. Но имейте ввиду, что проверка сертификата происходит при подключении клиента. Таким образом, если клиент уже подключен, а вы отозвали сертификат, подключение не будет сброшено и клиент не будет отключен до тех пор, пока он сам не завершит текущее соединение, либо пока сервер openvpn не будет перезапущен.

Так же после отзыва не лишним будет удалить файл настроек и конфигурации клиента, а так же сохраненный ip адрес. Чтобы все это не делать вручную, я написал простенький скрипт *revoke-user.sh*, который включает все необходимые действия.

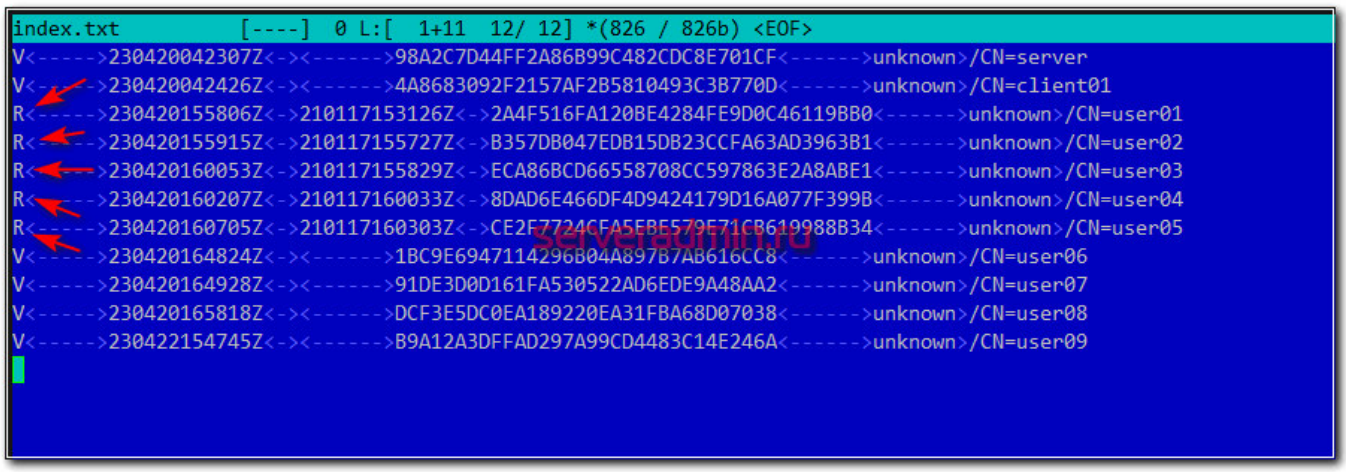
```
#!/bin/bash

echo -n "Enter user name for revoke: "
read user

clientexist=$(tail -n +2 /etc/openvpn/keys/easyrsa/pki/index.txt | grep -c -E "/CN=$user\$")
        if [[ $clientexist == '0' ]]; then
        echo ""
            echo "The specified client name was not found in easy-rsa"
            exit
        else
        cd /etc/openvpn/keys/easyrsa/
        ./easyrsa revoke "$user"
        ./easyrsa gen-crl
        rm -f /etc/openvpn/server/crl.pem
        cp /etc/openvpn/keys/easyrsa/pki/crl.pem /etc/openvpn/server/crl.pem
```

```
rm -f "/etc/openvpn/users-conf/$user.ovpn"
rm -f "/etc/openvpn/ccd/$user"
sed -i "/^$user,.*/d" /etc/openvpn/server/ipp.txt
echo ""
echo "Certificate $user revoked."
fi
```

Отозванные сертификаты в списке **index.txt** помечаются буквой R.



Лабораторная работа №4

Тема: Программирование сетевого протокола TCP. Разработка клиент серверного приложения.

Цель: Изучение средств реализации сетевого взаимодействия приложений на базе классов QTcpSocket и QTcpServer.

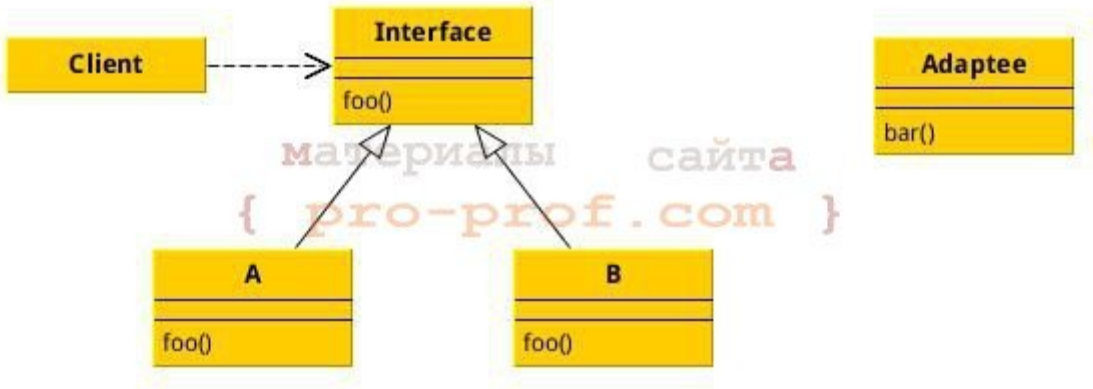
Задание:

Разработать приложение, организованное по клиент-серверной архитектуре и обеспечивающее обмен сообщениями между клиентом и сервером.

Теоретическая часть

1. Шаблон проектирования Adapter

Адаптер относится к структурным шаблонам проектирования и применяется в случаях, когда появляется класс, интерфейс которого нам не подходит по каким-либо причинам. Очевидное решение проблемы — создать новый класс с «правильным» интерфейсом, который будет каким-либо образом делегировать запросы к классу с «плохим» интерфейсом — при этом не угодный нам интерфейс как бы «скрывается». Поэтому шаблон проектирования адаптер также называют оберткой (*wrapper*).



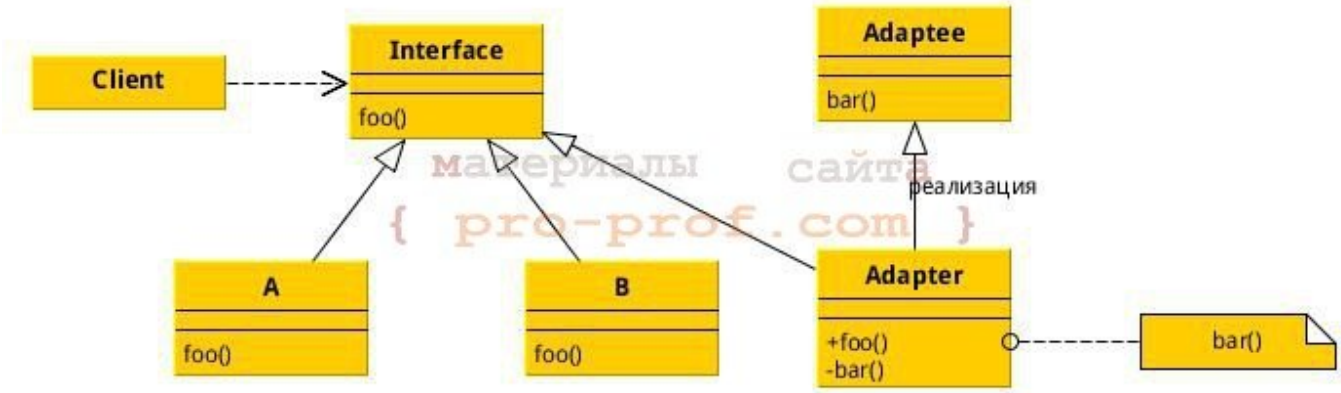
материалы сайта
{ pro-prof.com }

На рисунке показана типичная ситуация, в которой может быть уместным применение адаптера. Есть некоторый код (*Client*), который использует экземпляры классов *A* и *B*, реализующих общий интерфейс (*Interface*). В один прекрасный

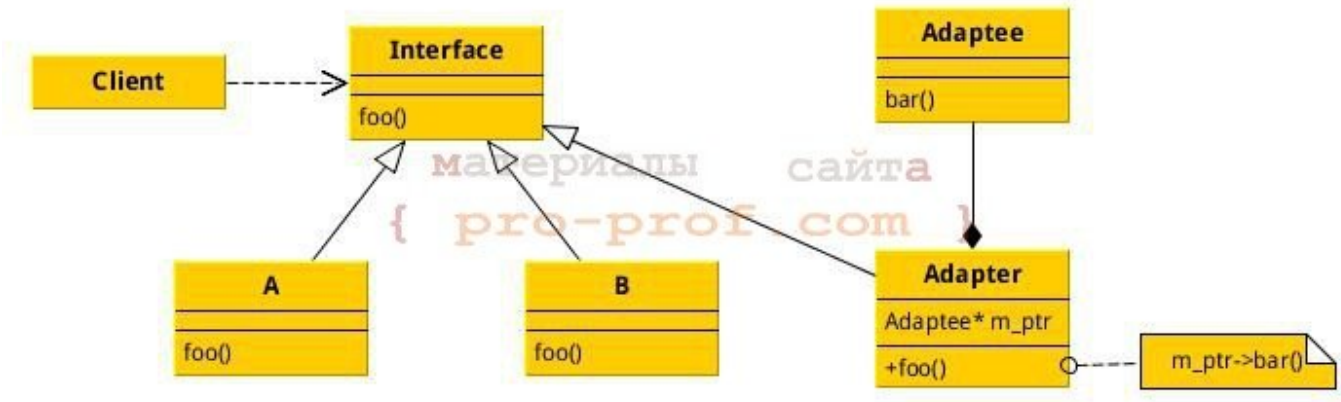
момент, нам потребовалось наравне с *A* и *B* использовать класс *Adaptee*, но он не реализует нужный нам интерфейс.

Вероятно, класс *Adaptee* достаточно сложен, чтобы нам не пришло в голову переписывать его с новым интерфейсом или лезть в него чтобы «подпилить» интерфейс.

Первый вариант решения такой проблемы заключается в наследовании классов *Interface* и *Adaptee* классом *Adapter*. При этом в отношении класса *Adaptee* используется *закрытое наследование*, т.к. адаптер не должен предоставлять лишний интерфейс. Такой вариант решения проблемы называется адаптером классов, использует *множественное наследование* и все вытекающие из него проблемы.



Другим вариантом, является адаптер объектов, заключающийся в *наследовании интерфейса и агрегации адаптируемого класса*. Такое решение является более гибким и не имеет недостатков от *множественного наследования*, однако, за счет того, что доступ к элементам адаптируемого класса осуществляется через указатель, снижается *быстродействие*.



Пример из диаграмм очень прост и абстрактен, в реальности все бывает совсем иначе — стоит задача не просто заменить функцию *bar* на *foo*, а проделать кучу работы по адаптации интерфейса.

2. Работа с сетью в Qt. Классы QTcpServer и QTcpSocket

В библиотеке *Qt* есть модуль работы с сетью — *Qt Network*, для подключения которого в файле проекта достаточно добавить одну строку:

```
QT += network
```

Модуль предоставляет ряд полезных классов, среди которых рассмотренный ранее *QHttp*, *QTcpSocket* и *QTcpServer.*

Основная задача *QTcpServer* — отслеживать подключение клиентов. Сервер слушает определенный *порт*, задаваемый при вызове *метода listen*. При подключении клиента, вырабатывается *сигнал newConnection* и создается *сокет (QTcpSocket)* для обмена данными с клиентом. Получить указатель на сокет можно вызовом *nextPendingConnection*.

Обмен данными с клиентом осуществляется через сокет. При отключении клиента сокет вырабатывает *сигнал disconnected*, а при поступлении в сокет данных — *сигнал readyRead*. *Сигнал readyRead* вызывается всякий раз, когда новая порция данных поступает в сокет, узнать сколько именно данных доступно для чтения в сокете можно вызовом *метода bytesAvailable*.

Чтобы данные поступили в сокет, их нужно туда записать, для этого используется метод *QTcpSocket::write*. Мы можем олдскульно записывать в сокет последовательности байт:

```
int i = 123;
pSock->write((char*)&i, sizeof(int));
```

В приведенном примере в сокет записывается столько байт, начиная с адреса переменной *i*, сколько достаточно для хранения целого числа. На другом конце провода можно будет считать эти байты методом *read*, но это очень не удобно, поэтому есть другое, более красивое решение с использованием *QByteArray* и *QDataStream.*

```
void SocketAdapter::sendString(const QString& str) {
```

```
QByteArray block;
QDataStream sendStream(&block, QIODevice::ReadWrite);
sendStream << quint16(0) << str;
sendStream.device()->seek(0);
sendStream << (quint16)(block.size() - sizeof(quint16));
m_ptcpSocket->write(block);
}
```

На *листинг 1* приведен код записи строки в *сокет*. Если помимо строки потребуется записать еще что-нибудь — изменится только *5 строка*. Мы пишем в сокет данные блоками, в начале блока идет количество байт. В *5 строке*, мы резервируем в начале блока память для будующего размера, пишем в блок остальные данные, в *7 строке* устанавливаем указатель записи на начало и в *8 строке* помещаем в это начало получившийся размер блока.

При считывании данных с сокета нужно учитывать, что *данные могут поступать частями*. Никто не гарантирует, что в результате двух вызовов функции *write*, будет ровно два раза выслан сигнал *readyRead* — он может быть выслан любое количество раз (даже лишь один раз).

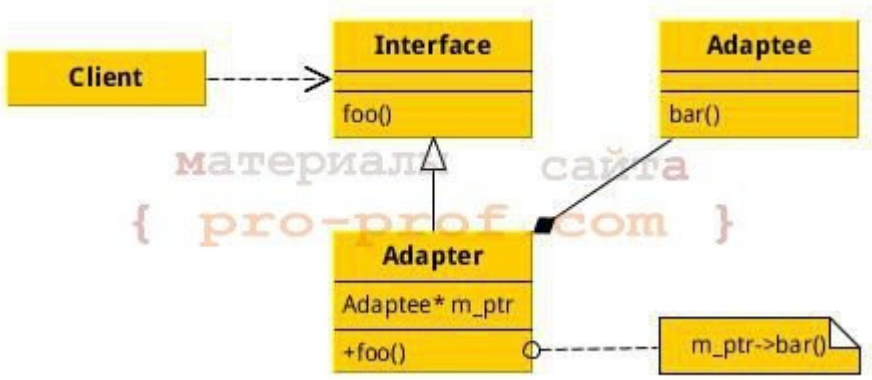
```
void SocketAdapter::on_readyRead() {
QString buff;
QDataStream stream(m_ptcpSocket);
while(true) {
if (m_msgSize < 0) {
if (m_ptcpSocket->bytesAvailable() < sizeof(int))
return;
stream >> m_msgSize;
}
else {
if (m_ptcpSocket->bytesAvailable() < m_msgSize)
return;
stream >> buff;
emit message(buff);
```

```
m_msgSize = -1;
}
}
}
```

В *листинг 2* с сокета считывается ровно одна строка, но если бы считывалось больше данных — изменения коснулись бы только *14 строчки* кода. Вечный цикл в этом коде нужен именно потому, что на несколько операций записи в сокет может прийти один сигнал *readyRead* — обработать же нужно все имеющиеся в сокете данные. При считывании строки, в приведенном примере выработается сигнал *message*.

3. Пример использования паттерна Adapter

Наш пример адаптера будет не совсем типичным, потому что у нас не будет классов *A и B* — мы адаптируем класс не к существующей системе, а к проектируемой.



материал сайта
{ pro-prof.com }

На *рисунке* очень грубо показано что именно мы будем делать. В качестве *Adaptee* будет выступать *сокет библиотеки Qt — класс QTcpSocket.* Работать с этим сокетом не очень приятно — это все же весьма низкоуровневая штука. В качестве клиента будет выступать чат или сервер, при разработке которых нам бы хотелось иметь более удобный интерфейс — по этой причине мы добавим прослойку в виде *адаптера*. Внешне это не очень похоже на стандартный *адаптер*, но он решает те же задачи — интерфейс класса сокета приводится (адаптируется) к желаемому виду.

Хотелось бы, чтобы *сокет* сообщал о разрыве соединения и новых данных, когда они полностью получены, а также, позволял бы отправлять данные. Для этого интерфейс адаптера должен содержать *2 сигнала и метод (или слот) отправки*. Такой интерфейс показан на *листинг 3*.

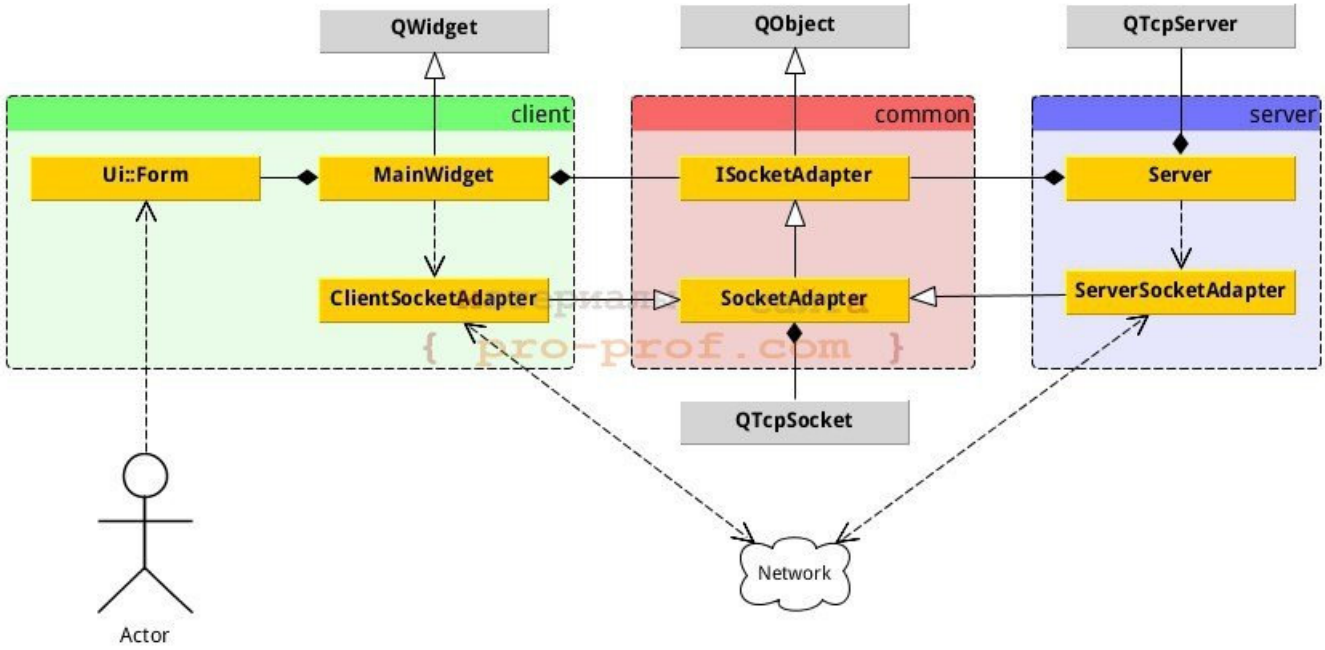
```
class ISocketAdapter : public QObject {
Q_OBJECT
public:
explicit ISocketAdapter(QObject *parent);
virtual ~ISocketAdapter();
virtual void sendString(const QString& str) = 0;
signals:
void message(QString text);
void disconnected();
};
```

Класс адаптера *агрегирует сокет и наследует интерфейс*. В результате все операции сокета оказываются скрыты за приятным интерфейсом.

Пример был бы более удачным, если чат мог бы получать данные из других источников (помимо *TCP-сокета*), которые реализовывали бы наш интерфейс. В качестве таких источников могли бы быть например обертки над *QUdpSocket* или вариации на тему передачи данных в защищенном виде. Наш же пример адаптера по результатам применения получился похожим на фасад.

4. Исходный код сетевого чата

Диаграмма классов клиента и сервера приведена на *рисунке*. Часть классов мы уже подробно рассмотрели.



MainWidget представляет собой главное окно чата, он агрегирует форму, созданную в *Qt Designer* [4]. Форма содержит *2* поля ввода и кнопку. При щелчке на кнопку вызывается функция *sendString* класса *ClientSocketAdapter*, а поле ввода очищается. При получении сигнала *message* от *ClientSocketAdapter*, второе поле ввода главной формы дополняется принятой от сервера строкой.

```
    MainWidget::MainWidget(QWidget *parent) :
    QWidget(parent), m_pForm(new Ui::Form()),
    m_pSock(new ClientSocketAdapter(this)) {
    m_pForm->setupUi(this);
    connect(m_pSock,                SIGNAL(message(QString)),
SLOT(on_message(QString)));
    connect(m_pForm->send,                  SIGNAL(clicked()),
SLOT(on_send()));
    }
    void MainWidget::on_message(QString text) {
    m_pForm->messages->setHtml(m_pForm->messages->toHtml()  +
text + " ");
    }
    void MainWidget::on_send() {
```

```
    m_pSock->sendString(m_pForm->message->text());
    m_pForm->message->clear();
    }
```

Класс *Server* агрегирует *QTcpServer*, а также, *список указателей на адаптеры сокетов*. При подключении клиента наш сервер получает от *QTcpServer* указатель на созданный сокет, на основе которого создается *адаптер серверного сокета (ServerSocketAdapter)* и добавляется в список. Сервер хранит список подключенных клиентов чтобы пересылать между ними сообщения. При получении сообщения от любого клиента, сервер обходит *список адаптеров* и для каждого вызывает метод *sendString*. При отключении клиента, адаптер удаляется из списка. Деструктор адаптера обеспечивает корректное освобождение памяти из под сокета (*QTcpSocket*).

```
    Server::Server(int nPort, QObject *parent) :
QObject(parent),
    m_ptcpServer(new QTcpServer(this)) {
    connect(m_ptcpServer, SIGNAL(newConnection()),
SLOT(on_newConnection()));
    if (false == m_ptcpServer->listen(QHostAddress::Any,
nPort)) {
    m_ptcpServer->close();
    throw m_ptcpServer->errorString();
    }
    }
    void Server::on_newConnection() {
    QTextStream(stdout) << "new connection" << endl;
    QTcpSocket* pclientSock = m_ptcpServer-
>nextPendingConnection();
    ISocketAdapter *pSockHandle = new
ServerSocketAdapter(pclientSock);
    m_clients.push_back(pSockHandle);
    pSockHandle->sendString("connect");
```

```
    connect(pSockHandle,              SIGNAL(disconnected()),
SLOT(on_disconnected()));
    connect(pSockHandle,            SIGNAL(message(QString)),
SLOT(on_message(QString)));
    }
    void Server::on_disconnected() {
    QTextStream(stdout) << "client disconnected" << endl;
    ISocketAdapter*                client                =
static_cast<ServerSocketAdapter*>(sender());
    m_clients.removeOne(client);
    delete client;
    }
    void Server::on_message(QString msg) {
    foreach (ISocketAdapter *sock, m_clients)
    sock->sendString(msg);
    }
```

В связи с тем, что клиент и сервер обменивается сообщения в одинаковом формате, *адаптер сокета клиента* почти не отличается от *адаптера сокета сервера*, поэтому основную часть кода можно перенести в класс *SocketAdapter*.

Первое различие между ними заключается в том, что адаптер серверного сокета создается на основе уже имеющегося объекта *QTcpSocket*, а адаптер сокета клиента должен создать такой объект. Эта разница учтена в конструкторе класса *SocketAdapter*.

```
    SocketAdapter::SocketAdapter(QObject *parent, QTcpSocket
*pSock)
    : ISocketAdapter(parent), m_msgSize(-1) {
    if (0 == pSock)
    m_ptcpSocket = new QTcpSocket(this);
    else
    m_ptcpSocket = pSock;
```

```
    connect(m_ptcpSocket,      SIGNAL(readyRead()),      this,
SLOT(on_readyRead()));
    connect(m_ptcpSocket,    SIGNAL(disconnected()),    this,
SLOT(on_disconnected()));
    }
```

Кроме того, клиентский сокет должен выполнить *метод connectToHost* чтобы начать диалог с сервером. Сервер таких действий предпринимать не должен.

```
    ClientSocketAdapter::ClientSocketAdapter(QObject
*parent)
    : SocketAdapter(parent) {
    m_ptcpSocket->connectToHost("localhost", 1024);
    }
    ServerSocketAdapter::ServerSocketAdapter(QTcpSocket*
pSock, QObject *parent) :
    SocketAdapter(parent, pSock) {
    }
```

Лабораторная работа №5

Тема: Программирование сетевого протокола TCP. Распределенное решение математической задачи

Цель: Изучение средств реализации сетевого взаимодействия приложений на базе классов QTcpSocket и QTcpServer.

Задание:

1. Основываясь на программе, разработанной в лабораторной работе №4 реализовать программы нахождения искомой величины согласно индивидуальному варианту задания.
2. Программа реализуется по клиент серверной архитектуре.
3. Клиенты подключаются к серверному компоненту по адресу IP и номеру порта.
4. Сервер распределяет задания на выполнение вычислений между клиентами.
5. Клиентские компоненты передают результаты вычислений на сервер.
6. Сервер выполняет объединение результатов поиска и выводит их пользователю.

Варианты заданий:

№	Функция	Точность	Искомая величина
1.	$Y = \sin(x)$	10^{-6}	Площадь фигур при Y>0
2.	$Y = \cos(x)$	10^{-6}	Площадь фигур при Y<0
3.	$Y = (\sin(x) / 2)^2$	10^{-6}	Найти экстремумы
4.	$Y = \cos^3(x)$	10^{-6}	Найти корни
5.	$Y = \sin(\ln(x))$	10^{-6}	Площадь фигур при Y>0
6.	$Y = \cos(\lg(x^3))$	10^{-6}	Площадь фигур при Y<0
7.	$Y = \sin(x) - \cos(x/5)$	10^{-6}	Найти экстремумы
8.	$Y = \cos(x) + \cos(2x)$	10^{-6}	Найти корни
9.	$Y = \sin(x) / 2$	10^{-6}	Площадь фигур при Y>0
10.	$Y = \cos(x) * 2 - 1{,}5$	10^{-6}	Площадь фигур при Y<0

11.	$Y = \sin(x) / 3 + 0{,}5$	10^{-6}	Найти экстремумы
12.	$Y = \cos(x) / 0{,}5 - 0{,}5$	10^{-6}	Найти корни
13.	$Y = \sin(3x) - 0{,}5$	10^{-6}	Площадь фигур при Y>0
14.	$Y = \cos(x/2) * 2 + 3$	10^{-6}	Площадь фигур при Y<0
15.	$Y = \sin(x) / 3 + 5$	10^{-6}	Найти экстремумы
16.	$Y = 2 / \sin(x) + 2$	10^{-6}	Найти корни
17.	$Y = 3 / \cos(2x) - 0{,}1$	10^{-6}	Площадь фигур при Y>0
18.	$Y = \sin^2(x^{1/3}) + 10$	10^{-6}	Площадь фигур при Y<0
19.	$Y = \sin(x^3) / 3 - 5$	10^{-6}	Найти экстремумы
20.	$Y = \cos(2x^2) / 5 + 5$	10^{-6}	Найти корни

Лабораторная работа №6

Тема: Программирование сетевого протокола UDP. Разработка клиент серверного приложения

Цель: Изучение средств реализации сетевого взаимодействия приложений на базе класса QUDPSocket.

Задание:

Разработать программу, реализующую обмен сообщениями между клиентским и серверным приложением по протоколу UDP.

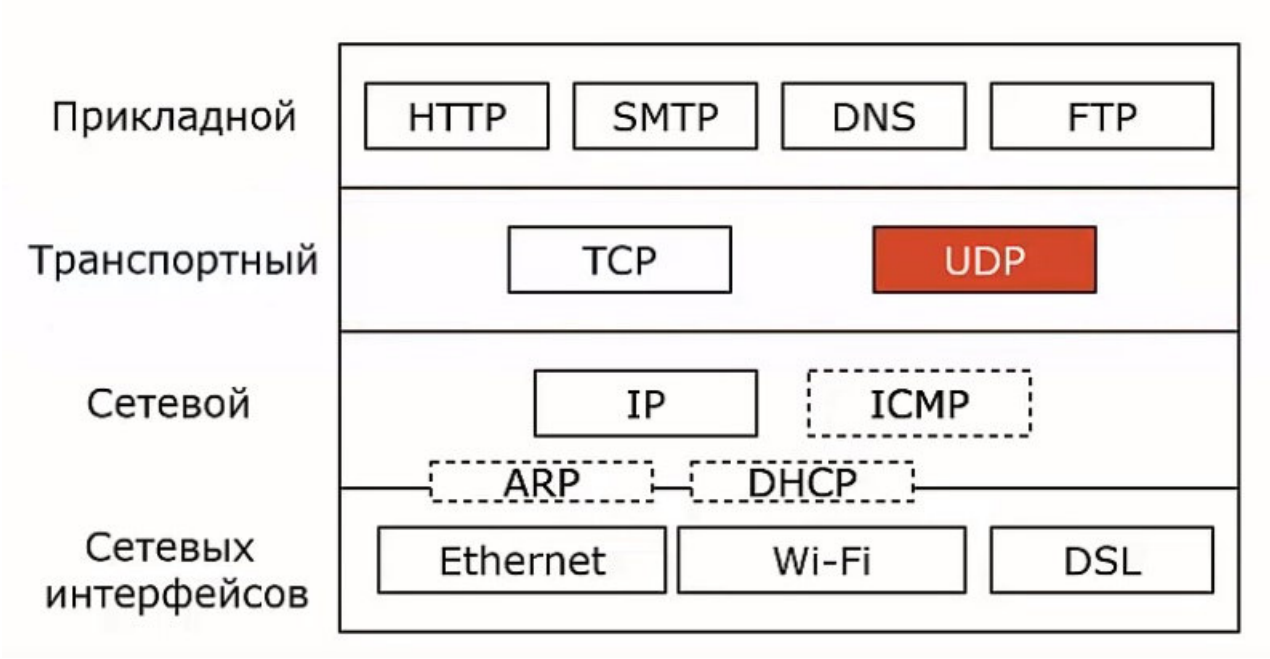
Теоретические сведения

UDP (*User Datagram Protocol* — протокол пользовательских датаграмм) — один из ключевых элементов набора сетевых протоколов для Интернета. С UDP компьютерные приложения могут посылать сообщения (в данном случае называемые датаграммами) другим хостам по IP-сети без необходимости предварительного сообщения для установки специальных каналов передачи или путей данных. Протокол был разработан Дэвидом П. Ридом в 1980 году и официально определён в RFC 768.

UDP использует простую модель передачи, без явных «рукопожатий» для обеспечения надёжности, упорядочивания или целостности данных. Датаграммы могут прийти не по порядку, дублироваться или вовсе исчезнуть без следа, но гарантируется, что если они придут, то в целостном состоянии. UDP подразумевает, что проверка ошибок и исправление либо не нужны, либо должны исполняться в приложении. Чувствительные ко времени приложения часто используют UDP, так как предпочтительнее сбросить пакеты, чем ждать задержавшиеся пакеты, что может оказаться невозможным в системах реального времени.

Природа UDP как протокола без сохранения состояния также полезна для серверов, отвечающих на небольшие запросы от огромного числа клиентов,

например DNS и потоковые мультимедийные приложения вроде IPTV, Voice over IP, протоколы туннелирования IP и многие онлайн-игры.



16 бит Порт отправителя	16 бит Порт получателя
16 бит Длина UDP	16 бит Контрольная сумма UDP

Длина UDP:

- Минимум 8 байт (только заголовок)
- Максимум 65 515 байт (максимальная длина данных IP-пакета)

Номера стандартных портов TCP, UDP

Десятичный номер порта	Обозначение порта		Описание
	в Интернет	в Unix	
0	-	-	Зарезервировано
1	TCPmux	-	TCP Мультиплексор
2	Compressnet	-	Программа управления
3	Compressnet	-	Процесс сжатия

5	RJE	-	Вход в удаленную задачу
7	Echo	echo	**Эхо**
9	Discard	discard	Сброс
11	Users	systat	Активные пользователи
13	Daytime	daytime	Время дня
15	-	Netstat	Кто работает или netstat
19	Chargen	chargen	Генератор символов
20	FTP-data	ftp-data	FTP (данные)
21	FTP	ftp	Протокол пересылки файлов (управление)
23	telnet	telnet	Подключение терминала
24	-	-	Любая частная почтовая система
25	SMTP	SMTP	Протокол передачи почтовых сообщений
31	MSG-auth		Распознавание сообщения (аутентификация)
35	-	-	Любой частный принт-сервер
37	Time	time	Время
39	RLP	-	Протокол поиска ресурсов
41	Graphics		Графика
42	nameserver	name	Сервер имен
43	Nicname	whois	Кто это? (whois-сервис)
45	MPM	-	Блок обработки входных сообщений
46	MPM-snd	-	Блок обработки выходных сообщений
48	Auditd	-	Демон цифрового аудита
49	login	-	Протокол входа в ЭВМ
50	RE-mail-ck	-	Протокол удаленного контроля почтовым обменом
53	Domain	nameserver	Сервер имен доменов (dns)
57	-	-	Любой частный терминальный доступ
59	-	-	Любой частный файл-сервер
64	covia	-	Коммуникационный интегратор (ci)
66	SQL*net	-	Oracle SQL*net
67	Bootps	Bootps	Протокол загрузки сервера
68	Bootpc	bootpc	Протокол загрузки клиента
69	TFTP	tftp	Упрощенная пересылка файлов
70	Gopher	-	Gopher (поисковая система)
71	-	Netrjs-1	Сервис удаленных услуг
77	-	rje	Любой частный RJE-сервис
79	Finger	finger	finger
80	WWW-HTTP		World Wide Web HTTP
81	Hosts2-NS	-	Сервер имен 2
87	-	-	Любая частная терминальная связь
88	Kerberos		Kerberos
92	NPP	-	Протокол сетевой печати
93	DCP	-	Протокол управления приборами
95	Supdup	supdup	Supdup протокол
97	Swift-rvf	-	swift-протокол удаленных виртуальных файлов
101	Hostname	hostnames	Сервер имен ЭВМ для сетевого информационного центра
102	ISO-Tsap	iso-tsap	ISO-Tsap
103	GPPitnp		Сети точка-точка
104	ACR-nema		ACR-nema digital IMAG. & comm. 300
108	Snagas		sna-сервер доступа
109	POP2	-	Почтовый протокол pop2
110	POP3	-	Почтовый протокол POP3

111	SUNRPC	sunrpc	SUN microsystem RPC
113	Auth	auth	Служба распознавания
114	Audionews		Аудио-новости
115	SFTP		Простой протокол передачи файлов
117	UUCP-path	uucp-path	Служба паролей UUCP
118	SQLserv		SQL-сервер
119	NNTP	NNTP	Протокол передачи новостей
123	NTP	NTP	Сетевой протокол синхронизации
129	PWDgen		Протокол генерации паролей
130-132			Cisco
133	Statsrv		Сервер статистики
134	Ingres-net		Ingres-net-сервис
135	LOC-srv		Поисковый сервис
137	Netbios-SSN	-	Служба имен Netbios
138	Netbios-DGM		Служба дейтограмм netbios
139	Netbios-SSN		Служба сессий Netbios
147	ISO-IP		ISO-IP
150	SQL-net		SQL net
152	BFTP		Протокол фоновой пересылки файлов
156	SQLsrv		SQL-сервер
158	PCmail-srv		PC почтовый сервер
161	-	SNMP	Сетевой монитор SNMP
162	-	SNMP-trap	SNMP-ловушки
170	Print-srv		postscript сетевой сервер печати
179	BGP		Динамический протокол внешней маршрутизации
191	Prospero		Служба каталогов Prospero
194	IRC		Протокол Интернет для удаленных переговоров
201-206			Протоколы сетей Apple talk
213	IPX		ipx
348	CSI-SGWP		Протокол управления cabletron
396	Netware-IP		Novell-Netware через IP
398	Kryptolan		Kryptolan
414	Infoseek		Infoseek (информационный поиск)
418	Hyper-g		Hyper-g
444	SNPP		Простой протокол работы со страницами
512	-	biff (exec)	Unix Comsat (удаленное исполнение)
513	-	Who	Unix Rwho daemon
514	-	syslog	Дневник системы
515	Printer		Работа с буфером печати (spooler)
525	-	Timed	Драйвер времени

Принципиальные отличия TCP и UDP

Функции	UDP	TCP
Процедура установки соединения	Не требуется	Обязательная трехэтапная процедура начала сеанса связи
Гарантированный обмен трафиком	Пакеты могут теряться	Подтверждение и повторная передача исключают потерю

		данных
Сохранение порядка доставки сообщений	Неупорядоченные датаграммы	Строгая очередность пересылаемых пакетов
Контроль и управление потоком	Не контролируется передача пакетов данных	Контролирует и управляет потоком данных
Уведомляет о перегрузках	Не защищен от перегрузок	Уведомляет о перегрузках
Сохраняет границы передаваемых сообщений	Всегда сохраняет границы пересылаемых датаграмм	Не сохраняет границы пересылаемых сообщений, но гарантирует их целостность
Сегментация и сборка информационных пакетов	Нет	Поддерживается
Взаимодействия с полуоткрытыми соединениями	Повторная синхронизация не происходит	Соединение осуществляется посредством повторной синхронизации
Проверка достижимости	Нет	Да

Преимущества UDP

Однонаправленное (unicast) сообщение отправляется из одного узла только в один другой узел. Это также называется связью "точка-точка". Протокол TCP поддерживает лишь однонаправленную связь. Если серверу нужно с помощью TCP взаимодействовать с несколькими клиентами, каждый клиент должен установить соединение, поскольку сообщения могут отправляться только одиночным узлам.

Широковещательная передача (broadcast) означает, что сообщение отправляется всем узлам сети. *Групповая рассылка (multicast)* - это промежуточный механизм: сообщения отправляются выбранным группам узлов.

Реализация взаимодействия по протоколу UDP в Qt Creator

В файл проекта добавляем модуль Network

```
QT + = network
```

Реализация сервера

Файл *.h

```
#ifndef UDPSERVER_H
#define UDPSERVER_H

#include <QWidget>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QVBoxLayout>
#include <QUdpSocket>
#include <QTimer>

class UdpServer : public QWidget
{
    Q_OBJECT

public:
    UdpServer(QWidget *parent = 0);
    ~UdpServer();
public slots:
    void StartBtnClicked();
    void timeout();
private:
    QLabel *TimerLabel;
    QLineEdit *TextLineEdit;
    QPushButton *StartBtn;
    QVBoxLayout *mainLayout;
    int port;
    bool isStarted;
    QUdpSocket *udpSocket;
    QTimer *timer;
};

#endif // UDPSERVER_H
```

Файл *.cpp

```
#include "udpserver.h"
#include <QHostAddress>
#include <QUdpSocket>
#include <QDebug>

UdpServer::UdpServer(QWidget *parent)
    : QWidget(parent)
{
    setWindowTitle(tr("UDP Server"));
    / * Инициализируем каждый элемент управления * /
    TimerLabel = new QLabel(tr("Таймер:"),this);
    TextLineEdit = new QLineEdit(this);
    StartBtn = new QPushButton(tr("начать"),this);
    / * Устанавливаем макет * /
```

```
    mainLayout = new QVBoxLayout(this);
    mainLayout->addWidget(TimerLabel);
    mainLayout->addWidget(TextLineEdit);
    mainLayout->addWidget(StartBtn);

connect(StartBtn,SIGNAL(clicked()),this,SLOT(StartBtnClicked()));
    port = 5555;     // Устанавливаем параметр номера порта UDP
    isStarted = false;
    udpSocket=new QUdpSocket;// Создаем QUdpSocket
    timer = new QTimer(this);
    // Регулярно отправлять информацию о трансляции
    connect(timer,SIGNAL(timeout()),this,SLOT(timeout()));
}

UdpServer::~UdpServer()
{

}

void UdpServer::StartBtnClicked()
{
    if(!isStarted)
    {
        StartBtn->setText("Стоп");
        timer->start(1000);
        isStarted=true;
    }
    else
    {
        StartBtn->setText("начать");
        isStarted=false;
        timer->stop();
    }
}

void UdpServer::timeout()
{
    QString msg =TextLineEdit->text();
    qDebug()<<msg.toLatin1();
    int length=0;
    if(msg=="")
    {
        return;
    }
    if((length=udpSocket-
>writeDatagram(msg.toLatin1(),msg.length(),QHostAddress::Broadcast
,port))!=msg.length())// QHostAddress :: Broadcast отправляет на
широковещательный адрес
    {
        return;
    }
}
```

Реализация клиентского приложения

Файл *.h

```
#ifndef UDPCLIENT_H
#define UDPCLIENT_H

#include <QDialog>
#include <QVBoxLayout>
#include <QTextEdit>
#include <QPushButton>
#include <QUdpSocket>

class UdpClient : public QDialog
{
    Q_OBJECT

public:
    UdpClient(QWidget *parent = 0);
    ~UdpClient();
public slots:
    void CloseBtnClicked();
    void dataReceived();
private:
    QTextEdit *ReceiveTextEdit;
    QPushButton *CloseBtn;
    QVBoxLayout *mainLayout;
    int port;
    QUdpSocket *udpSocket;
};

#endif // UDPCLIENT_H
```

Файл *.cpp

```
#include "udpclient.h"
#include <QUdpSocket>
#include <QMessageBox>
#include <QHostAddress>

UdpClient::UdpClient(QWidget *parent)
    : QDialog(parent)
{
    setWindowTitle(tr("UDP Client"));        // Устанавливаем
заголовок формы
    / * Инициализируем каждый элемент управления * /
    ReceiveTextEdit = new QTextEdit(this);
    CloseBtn = new QPushButton(tr("Close"),this);
    / * Устанавливаем макет * /
    mainLayout=new QVBoxLayout(this);
    mainLayout->addWidget(ReceiveTextEdit);
    mainLayout->addWidget(CloseBtn);
```

```
connect(CloseBtn,SIGNAL(clicked()),this,SLOT(CloseBtnClicked()));
    port =5555;                                 // Устанавливаем
параметр номера порта UDP, указываем для мониторинга данных по
этому порту
    udpSocket = new QUdpSocket(this);         // Создаем QUdpSocket

connect(udpSocket,SIGNAL(readyRead()),this,SLOT(dataReceived()));
    bool result=udpSocket->bind(port);// Привязать порт
    if(!result)
    {
        QMessageBox::information(this,"error","udp socket create
error!");
        return;
    }
}

UdpClient::~UdpClient()
{

}

void UdpClient::CloseBtnClicked()
{
    close();
}

void UdpClient::dataReceived()
{
    while(udpSocket->hasPendingDatagrams())// Есть дейтаграмма для
чтения
    {
        QByteArray datagram;
        datagram.resize(udpSocket->pendingDatagramSize());
        udpSocket->readDatagram(datagram.data(),datagram.size());
        QString msg=datagram.data();
        ReceiveTextEdit->insertPlainText(msg);
    }
}
```

Лабораторная работа №7

Тема: Программирование сетевого протокола UDP. Решение задачи подбора ключа.

Цель: Изучение средств реализации сетевого взаимодействия приложений на базе класса QUDPSocket, с добавлением поддержки многопоточности.

Задание:

Разработать программу, реализующую распределенный поиск ключевой-фразы шифра Вижинера, с использованием протокола UDP при известном алфавите и кодированной комбинации.

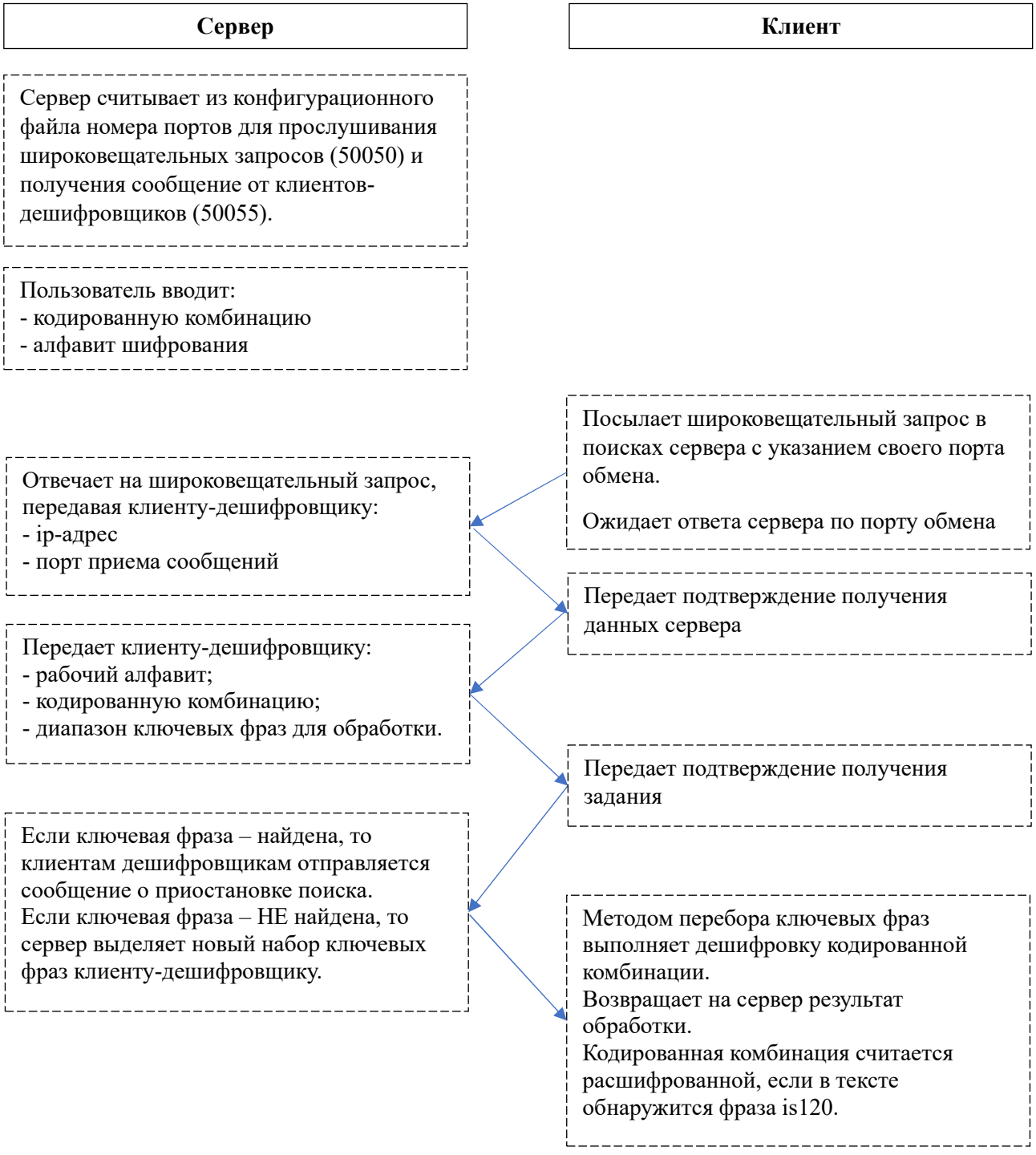
Реализацию алгоритма декодирования по методу Вижинера использовать из приложения KMZI.

Контрольные комбинации.

	Алфавит	
	АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯабвгдеёжзийклмнопрстуфхцчшщъыьэюяABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghigklmnopqrstuvwxyz 0123456789-+=.!?:;,	
	Исходный текст	**Кодированная фраза**
1	Дождались меня белые ночи Над procis120тором густых островов…	VП6XT9Й;нm-Ё.pmzЁ+мЧэО!иЫQ1 Xm.C!гUxWCme=C!Юm0Ф;днAU!ге!П1аФQ
2	В этот лес Is120 завороженный, По пушинкам серебра,	ДЫIvвЭфЫdФа13Xll2ЫZПРpggЦУпdtЩ0БюggЮхоаЭЩв сЭбУтUSaОБ
3	У лукоморья дуб зелёный; Златая цеiS120пь на дубе том:	maыZы.УюWNКауZсюОфRц=вщгПаТпYpKaGKzЭс,ТАЗа эF:2ЪрK:;Хьв
4	- Скажи-ка, дядя, ведь недаром Москва, спалIS120енная пожаром,	ЖА6ВуцьgВуОАЁЦ1PTV=2yQVE2yyТЁ- ОаРИ8cуоK?яуНЯЬ;ГW?+эубK.юъВЗ=ьТ
5	Я к вам пишу - IS120 чего же боле? Что я могу еще сказать?	?ъыМО:7:QVX!:ЧШеЩ,ПЩQ,xDbQ1xMNЛ6хЬк7.яMsQ7 яDgQ KFШО5pIМПДМ
6	Жди meiS120ня, и я вернусь. Только очень жди,	УEMG=c!xIC.ОгZяхМгG1cRСЖ;JЪqt!шdОВяыYИБИ=НЗ ?S
7	У неё всё своё - и бельё, и жильё, Ну а я ангажиIs120рую угол у тёти.	nUъФЧяГюXl;ГыXlKUxgТ4MJX3яЙ=ЦЪ+Эт13hФ=ПlЛU мЭФ 3хчdВХ.aeKUATa+UAgд5Ухх
8	Муха, Муха - IS120 Цокотуха, Позолоченное брюхо!	ГАЦ 2qЩФВРqДUФCs?VяЖечП,длmnSAефП+ЯнсО.ЯЫI=В:ол ыg
9	Эльвина, милый друг, приди, подай мне руку, Я вяну, прекis120рати тяжелый жизни сон;	ORAx3fFJъЛaR,4SVW-ylЭV73Гадц- HVF0ъЛfКц+TcZJN;ЭНГ8ТпС6+ДсАоьVЮWr.3ЭYГ1Ддh 0ъЁaN43SgU4M
10	Утихла брань племен; в пределах отдаленных Не слышен битвы is120шум и голос труб военных;	Юi6ЫI8кЫIzЦхшsэХ8пс2УО-RэХ.пТ2СхАЫ=Ш0кb2У- GlQ61-h9бВпdэЖ5эRЁZк2ЭАаВюсэОьне9Ф!-i!Щу-R=К- шгAr
11	Не пора ль нам, братия, начать О походеIS120 Игоревом слово,	ъеQГ=eaQ,ЖЩн:AQЩбН6:WягG+Nч:ЁЖлOQГ=goБ+Т+ 1ТНэхгЛД2РоЙG?Zo,BQ

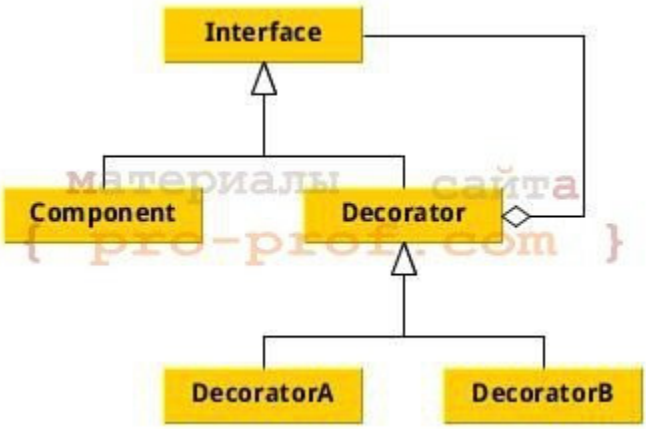
12	Мороз и солнце; день чудесный! Еще тыIs120 дремлешь, друг прелестный	ЪоЙВЛ.иМЕТщнП+р.д;Ба.чМ-ИянФ:mМЕТ+ХАыбюZ:0М-ФтмЕ+ЬК,М-ФВгМГФтл;ЕЦыыГ
13	Не выходи из комнаты, не совершай ошибку. Зачем тебе СолнцеiS120, если ты куришь Шипку?	uxЫ1пОяТ8оВшЫ-гЁюР,пf:d4оКяR4еСрZягСщQ-заПц л;эЫ,Щ.хЫlгЕюm4Wк,ЮА6Мxh+ЭМDгяЯМВYЕрМиY?ЯММ
14	Я вас любилIS120: любовь еще, быть может, В душе моей угасла не совсем;	sGy:Еш,Q;?4ЫkSJщX;ИСuВуЩGyMцgGuОЕЩG5ВшВЁLZУQ-=ЛцQА7+ыQЖw6DИ6шБцQЕ78DBAK
15	Словно тихий ребенок, обласкаis120нный тьмой, С бесконечным томленьем в блуждающем взоре,	ЬМ78АщМ+?ЗуКшД-лF6В:ЙМ77;кS36у2ОыНАшс2GEHN7:YЙсш7-ьL7Б-СОБАFэР5,-шdyAFмMu,ЁсEtСЛпNш8.щRyZ

Программа должна реализовывать следующую логику работы.



Теоретические сведения

Декоратор — структурный шаблон проектирования, который является гибкой альтернативой наследованию. Чаще всего декоратор применяется в случаях, когда объекту требуется добавить функциональность во время выполнения программы, но может применяться в других случаях.



Некоторые компоненты системы реализуют определенный *интерфейс* , но однажды возникает необходимость добавить компонентам *новую функциональность*. *Например*, компонентами могут являться графические объекты, которые могут перемещаться, возвращать свои координаты и размер (набор таких общих операций образует *интерфейс*). В качестве *новой функциональности* может быть изменение цвета, масштабирование или привязка к сетке.

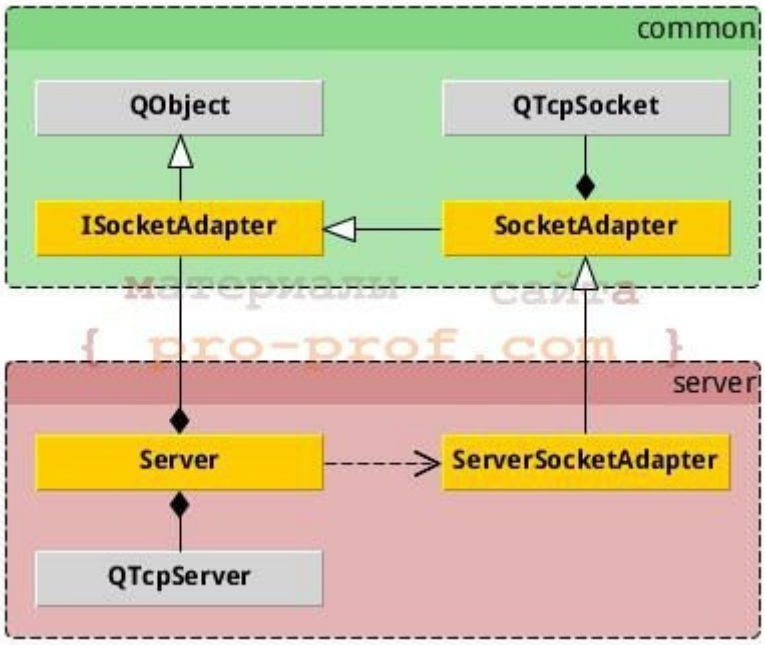
Решить проблему можно наследованием, породив подклассы с новой функциональностью от каждого компонента, однако, это не будет гибким решением. Шаблон проектирования декоратор позволяет:

- *добавлять новую функциональность* уже существующим объектам прямо во время выполнения. Гамма описывает пример с полосами прокрутки, которые могут появляться лишь при определенных условиях;
- *смешивать функциональность*. Например, графический элемент может одновременно изменять цвет, иметь привязку к сетке и предоставлять полосу прокрутки — если бы мы решали эту (повседневную и простую, в общем-то) задачу лишь наследованием — иерархия классов получилась бы громадной («комбинаторный взрыв»).

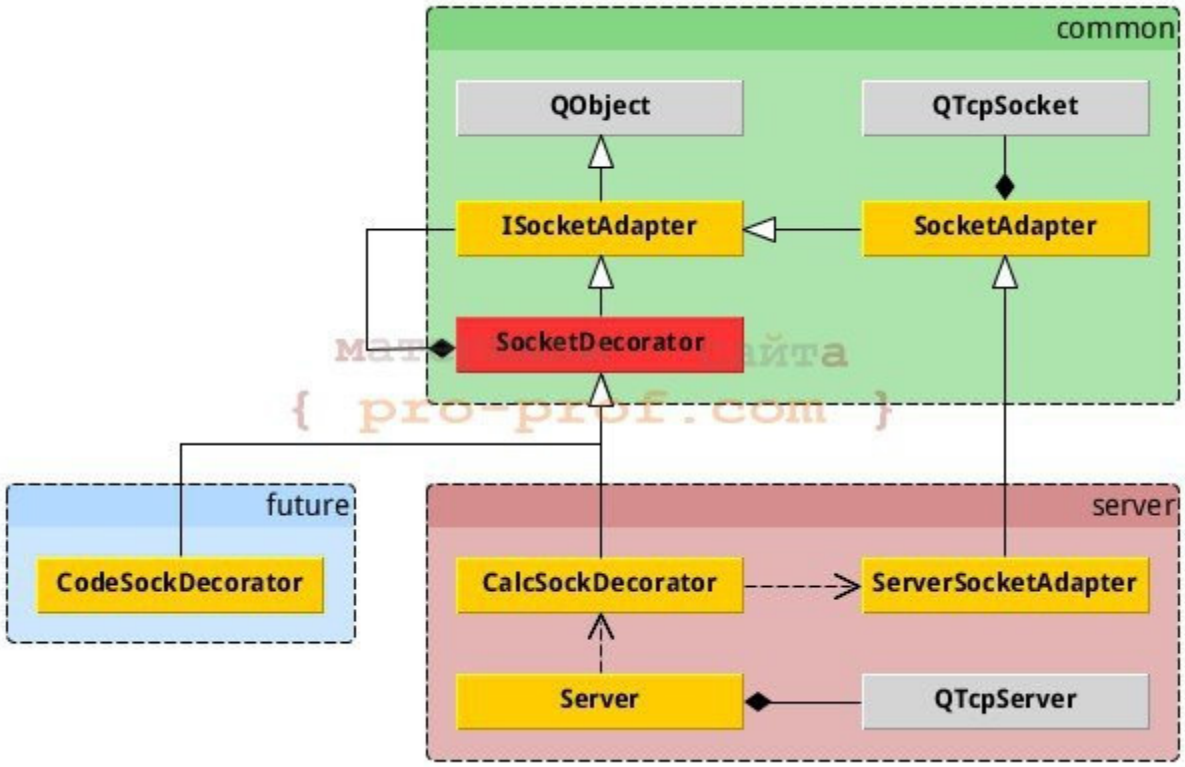
Как показано на *рисунке*, и компонент, и декоратор реализуют один и тот же интерфейс, т.е. декоратор является таким же компонентом, как и все остальные. Тем не менее, Гамма указывает, что в некоторых случаях, декоратор может сильно отличаться от других компонентов и это надо учитывать. Класс *Decorator* хранит ссылку на *декорируемый компонент,* а его наследники каким-либо образом переадресуют этому компоненту запросы, но кроме этого, выполняют еще какие-либо действия (рисуют рамку или выводят ползунки прокрутки, например).

Шаблон проектирования «Декоратор». Пример

В нашей задаче многопоточного сервера, декорироваться будет сокет. За основу взят код однопоточного сервера чата.



На *рисунке* видно, что сервер работает с указателем на *ISocketAdapter*, по которому находится экземпляр *ServerSocketAdapter*, который умеет принимать и отправлять текстовые сообщения. Нам требуется научить сервер новым трюкам, при этом, мы не хотим сильно изменять уже написанный и отлаженный код. Очевидно, надо породить нового наследника *ISocketAdapter*, который будет делать тоже, что и *ServerSocketAdapter* и еще чуть-чуть.



На *рисунке*, помимо декоратора, вычисляющего поступившие в сокет арифметические выражения изображен декоратор, обеспечивающий прием и передачу зашифрованных сообщений. Не трудно представить ситуацию, когда некоторый пользователь решит в *рантайме* перейти на защищенную передачу сообщений. *Декоратор* делает реализацию такого перехода очень простой.

```
    class SocketDecorator : public ISocketAdapter {
    Q_OBJECT
    public:
    explicit SocketDecorator(ISocketAdapter *sock, QObject
*parent = 0);
    virtual ~SocketDecorator();
    virtual void sendString(const QString& str) = 0;
    //!< отправка сообщения через сокет
    protected slots:
    void on_disconnected();
    //!< соединение разорвано (пришел сигнал от m_pSock)
    virtual void on_send_request(QString) = 0;
```

```
    //!< слот для отправки данных через сокет
    virtual void on_message(QString) = 0;
    //!< слот обработки сообщений, поступивших в сокет
    protected:
    ISocketAdapter *m_pSock;
    };
    // ...
    SocketDecorator::SocketDecorator(ISocketAdapter   *sock,
QObject *parent) :
    ISocketAdapter(parent), m_pSock(sock) {
    connect(m_pSock,                SIGNAL(disconnected()),
SLOT(on_disconnected()));
    connect(m_pSock,               SIGNAL(message(QString)),
SLOT(on_message(QString)));
    }
    SocketDecorator::~SocketDecorator() {
    }
    void SocketDecorator::on_disconnected() {
    emit disconnected();
    }
```

Видно, что базовый класс декоратора остается абстрактным, наследует интерфейс сокета и, одновременно, агрегирует сокет и соединяет свои слоты с его сигналами.

```
    class CalcSockDecorator : public SocketDecorator {
    Q_OBJECT
    public:
    explicit CalcSockDecorator(ISocketAdapter *sock, QObject
*parent = 0);
    virtual void sendString(const QString& str);
    protected slots:
    virtual void on_send_request(QString);
```

```
    virtual void on_message(QString);
    };
    // ...
    CalcSockDecorator::CalcSockDecorator(ISocketAdapter
*sock, QObject *parent) :
    SocketDecorator(sock, parent) {
    }
    void CalcSockDecorator::on_message(QString msg) {
    QScriptEngine engine;
    QString result = msg + " = " +
engine.evaluate(msg).toString();
    if (engine.hasUncaughtException())
    m_pSock->sendString("bad request");
    else
    emit message(result);
    }
    void CalcSockDecorator::sendString(const QString &str) {
    m_pSock->sendString(str);
    }
    void CalcSockDecorator::on_send_request(QString msg) {
    m_pSock->sendString(msg);
    }
```

CalcSockDecorator по-особенному обрабатывает поступающие сообщения (*слот on_messag*e), все остальные к запросы к себе он транслирует агрегируемому объекту. При поступлении сообщения, он пытается выполнить его как выражение *ECMAScript,* в случае возникновения *исключительной ситуации* возвращает строку с ошибкой, иначе — результат вычисления. Именно слот *on_message* выполняет самую сложную работу сервере, которую мы будем пытаться выполнять в отдельном потоке в остальной части статьи.

```
    void Server::on_newConnection() {
    QTextStream(stdout) << "new connection" << endl;
```

```
    CalcSockDecorator *decoratedSock = new CalcSockDecorator
    (new                    ServerSocketAdapter(m_ptcpServer-
>nextPendingConnection(), this), this);
    decoratedSock->sendString("connect");
    connect(decoratedSock,           SIGNAL(disconnected()),
SLOT(on_disconnected()));
    connect(decoratedSock,         SIGNAL(message(QString)),
SLOT(on_message(QString)));
    connect(this,  SIGNAL(message(QString)),  decoratedSock,
SLOT(on_send_request(QString)));
    }
```

Коренным образом изменилось только создания сокета — создается экземпляр *CalcSockDecorator* вместо *ServerSocketAdapter.* Кроме того, на *листинг 3* виден подводный камень — с использованием *механизмов библиотеки Qt* нам придется следить за тем, чтобы клиентский код был связан с сигналами и слотами декораторов самого верхнего уровня. Иными словами, если пользователь решит передавать сообщения зашифрованными — сервер должен будет:

1. создать соответствующий декоратор сокета, передав старый сокет в качестве аргумента;
2. отсоединить весь клиентский код от старого сокета;
3. соединить клиентский код с декорированным сокетом.

Все это может быть очень трудно реализовываться и, кажется, возможны случаи, в которых из за сигналов и слотов *использовать паттерн декоратор будет почти не возможно*. Это надо учитывать при проектировании.

Многопоточный сервер. Поток на каждого клиента

Очевидно, если мы хотим работать с каждым клиентом в отдельном потоке, то поток должен создаваться в момент подключения. Изменения коснутся лишь метода сервера *on_newConnection.*

```
    void Server::on_newConnection() {
    QTextStream(stdout) << "new connection" << endl;
```

```
    CalcSockDecorator *decoratedSock = new CalcSockDecorator
    (new                    ServerSocketAdapter(m_ptcpServer-
>nextPendingConnection(), this), this);
    QThread *sockThread = new QThread(this);
    decoratedSock->moveToThread(sockThread);
    sockThread->start();
    decoratedSock->sendString("connect");
    connect(decoratedSock,           SIGNAL(disconnected()),
SLOT(on_disconnected()));
    connect(decoratedSock,         SIGNAL(message(QString)),
SLOT(on_message(QString)));
    connect(this,  SIGNAL(message(QString)),  decoratedSock,
SLOT(on_send_request(QString)));
    }
```

В *строках 7-9* описано создание потока и перемещение в него объекта-обработчика сокета. Ничего нового в этом нет и, очевидно, это не лучшее решение. *Поток Qt* — это поток операционной системы, который, в свою очередь, является достаточно крупным объектом. Если к нашему чату подключится тысяча пользователей, серверу будет очень не легко, даже если пользователи почти не будут общаться. В связи с этим, количество потоков на сервере стараются ограничить.

Можно, например, завести на сервере определенное количество потоков (*пул потоков*) и направлять новых клиентов к наименее загруженным потокам. Примерно такой вариант мы уже использовали при написании парсера проектов с биржи фриланса *[3]*, но в библиотеке *Qt* есть готовый *пул потоков*, который используется чуть-чуть иначе (ниже описано подробнее).

Важно отметить, что в некоторых языках (*Erlang*, например) поддерживаются легковесные потоки. Для этих языков решение с отдельным потоком на каждого клиента было бы не только самым простым, но правильным и естественным.

Пул потоков Qt

Пул потоков Qt (*QThreadPool*) управляет набором потоков (*QThread*). Количество потоков задается *методом maxThreadCount()*, по умолчанию их ровно столько, сколько ядер имеется у вашего процессора. Когда мы писали свой пул, потоки там существовали вечно, но в *QThreadPool* поток удаляется если в течении определенного времени в него не поступают задачи, задать такой таймаут можно методом *setExpiryTimeout()*. При поступлении задачи поток вновь будет создан.

Мы могли бы создать на сервере экземпляр *QThreadPool* и добавлять задачи в него, однако, каждая программа, написанная с использованием *Qt* уже имеет запущенный пул потоков (глобальный пул программы). Обратиться к глобальному пулу можно с помощью статической функции *QThreadPool::globalInstance()*. С этим пулом мы и будем работать.

Задача, добавляемая в пул должна наследовать *класс QRunnable* — это абстрактный класс с чисто виртуальным методом *run()*. Метод *run()* должен содержать код, который мы хотим вынести в отдельный поток. После завершения работы, объект может быть *автоматически удален*, если установлен соответствующий флажок (по умолчанию установлен). Установить или снять такой флажок можно методом *QRunnable::setAutoDelete()*.

Чтобы передать результаты выполнения задачи во внешний мир можно использовать механизм сигналов и слотов, но *QRunnable* не является наследником *QObject*, поэтому наша задача для пула будет использовать *множественное наследование*. Передавать задаче данные, которые должны быть обработаны, удобно во *время конструирования объекта*.

В *QThreadPool* мы будем добавлять задачи, связанные с вычислением выражения, полученного через сокет.

```
class Calc : public QObject, public QRunnable {
Q_OBJECT
public:
explicit Calc(QString expr, QObject *parent = 0);
void run();
signals:
```

```
    void result(QString);
    protected:
    QString m_expr;
    };
    // ...
    Calc::Calc(QString expr, QObject *parent)
    : QObject(parent), m_expr(expr) {
    }
    void Calc::run() {
    QScriptEngine engine;
    QString res = m_expr + " = " +
engine.evaluate(m_expr).toString();
    if (engine.hasUncaughtException())
    emit result("");
    else
    emit result(res);
    }
```

Теперь при получении сообщения, объект *CalcSockDecorator* создает объект *Calc* и добавляет его в пул вызовом *QThreadPool::start()*, после чего, ожидает сигнала с результатами от этого объекта.

```
    void CalcSockDecorator::on_message(QString msg) {
    Calc *calc = new Calc(msg, this);
    connect(calc, SIGNAL(result(QString)),
SLOT(on_result(QString)));
    QThreadPool::globalInstance()->start(calc);
    }
    void CalcSockDecorator::on_result(QString res) {
    if (res == "")
    m_pSock->sendString("bad request");
    else
    emit message(res);
```

```
}
```

Вот таким не хитрым образом, почти без изменения уже написанного кода можно распараллелить программу с использованием пула потоков библиотеки *Qt*.

В одной из следующих статей я наверное опишу работу с *QtConcurrent* — это довольно специфическая штука, но тоже используется для распараллеливания.

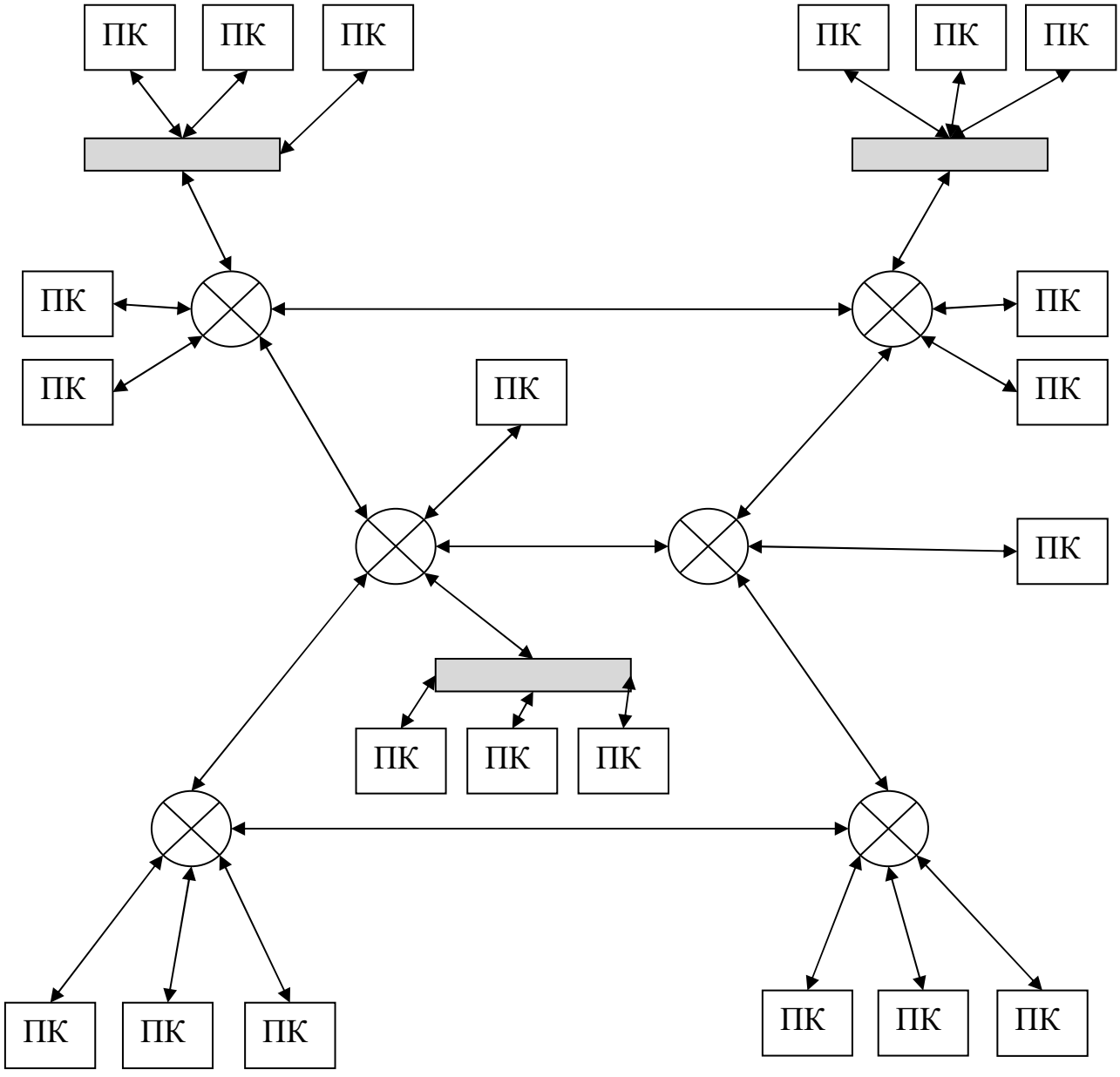
Лабораторная работа №8

Тема: Моделирование локальной сети в среде GNS3.

Цель работы: Научиться устанавливать GNS3 на операционную систему Linux (RedOS), включая необходимые компоненты для корректной работы.

Задание:

1. выполнить установку и конфигурирование среды GNS3 в РедОС;
2. подключить бинарный образ роутера;
3. построить сеть согласно схеме



4. сконфигурировать сеть;

5. подтвердить работоспособность сети приведя результаты попарного выполнения команды `ping` между компьютерами каждой подсети и внутри их.

Теоретическая часть.

Часть 1: Установка GNS3

Войдите в консоль под администратором и установите пакеты GNS3 из репозитория, используя команду:

```
dnf install gns3*
```

Часть 2: Установка и компиляция компонентов GNS3

Установите необходимые для компиляции компонентов зависимости, путём ввода команды:

```
dnf install glibc-devel elfutils-libelf-devel libpcap-devel
```

Проведите копирование исходников dynamips и их компиляцию, путём ввода последовательности комманд:

```
git clone https://github.com/GNS3/dynamips
cd dynamips
mkdir build
cd build
cmake ..
```

После успешной компиляции, установите dynamips, введя команду:

```
make install
```

Проведите клонирование и компиляция компонента vpcs, введя такие команды, как:

```
git clone https://github.com/GNS3/vpcs
cd vpcs/src
./mk.sh
```

ОПЦИОНАЛЬНО: УСТАНОВКА ОБРАЗОВ CISCO

Выберите необходимый вам образ эмулятора роутера на официальном сайте. Ссылку на загрузку вы можете получить, загуглив полное имя образа.

Использованный образ:

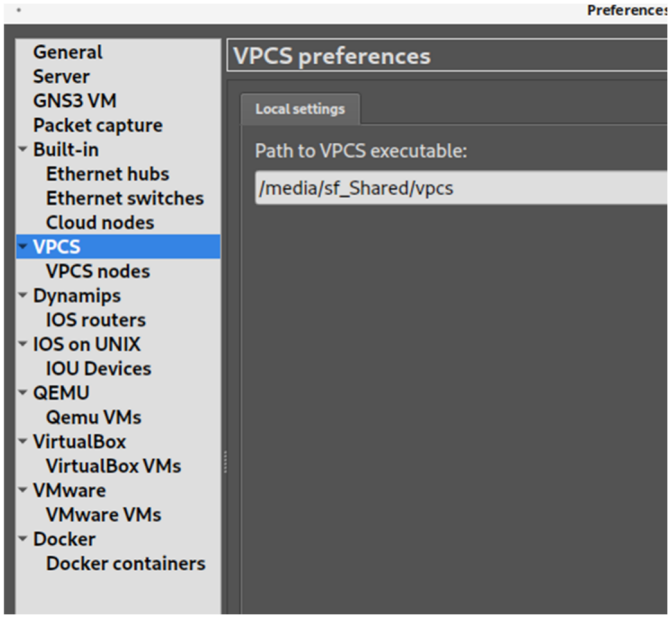
https://lms.onnocenter.or.id/pustaka/docs/GNS3/images/c3745-adventerprisek9-mz.124-25d.bin

Часть 3: Запуск и настройка

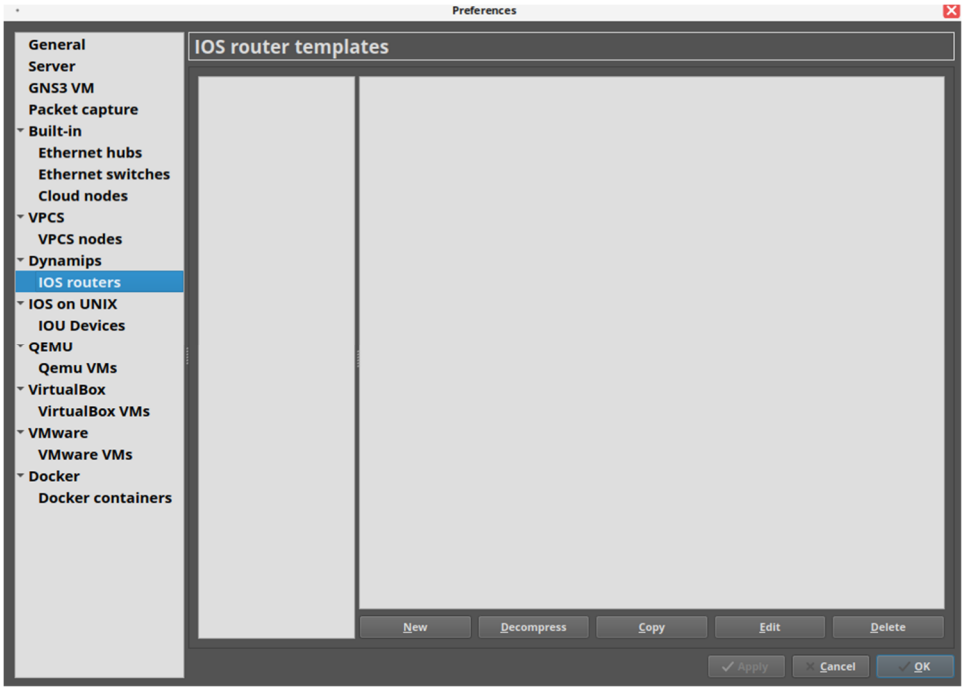
Найдите установленную программы в меню пуск, в разделе «Образовательные».

Создайте ваш первый проект, для дальнейшей настройки и последующей работы с GNS3.

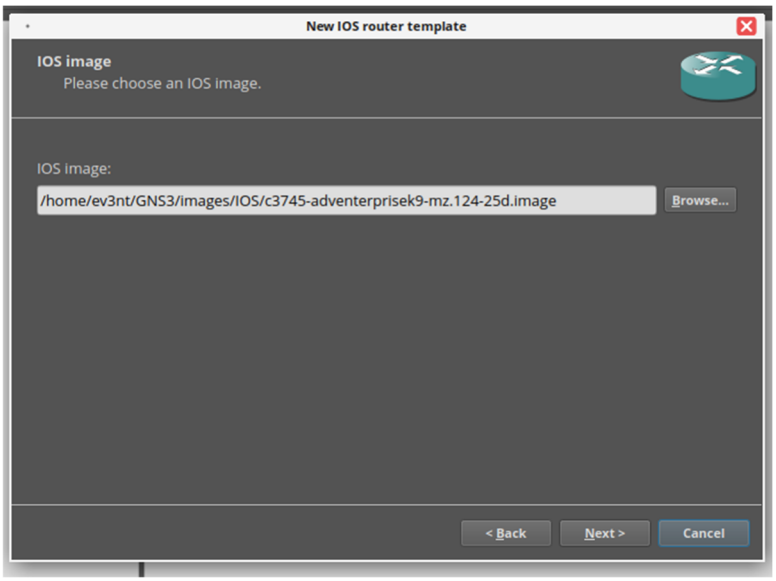
Откройте настройки GNS3 и укажите путь к скомпилированной версии vpcs во вкладке «VPCS» (без этого у вас не будут работать виртуальный компьютеры сети).



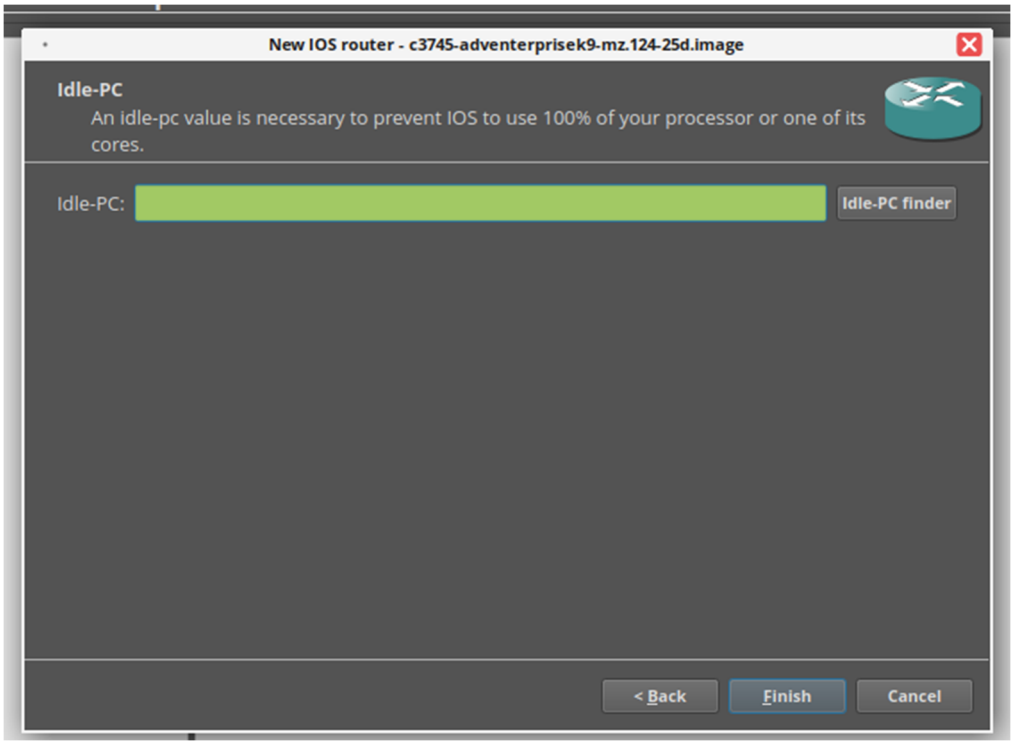
Далее для тех, кто скачал образ роутера.



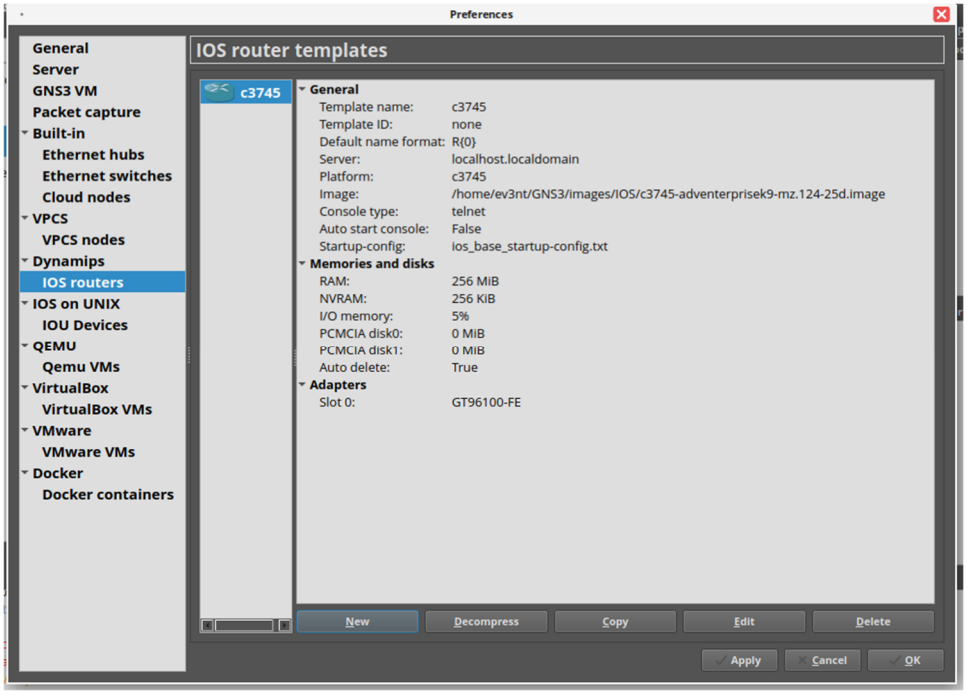
В том же меню настроек откройте вкладку «IOS routers» и нажмите на кнопку «New».



Укажите путь до скачанного образа, чтобы произошла распаковка (у вас появится всплывающее окно, где нужно будет подтвердить распаковку), далее жмёте везде «Next».



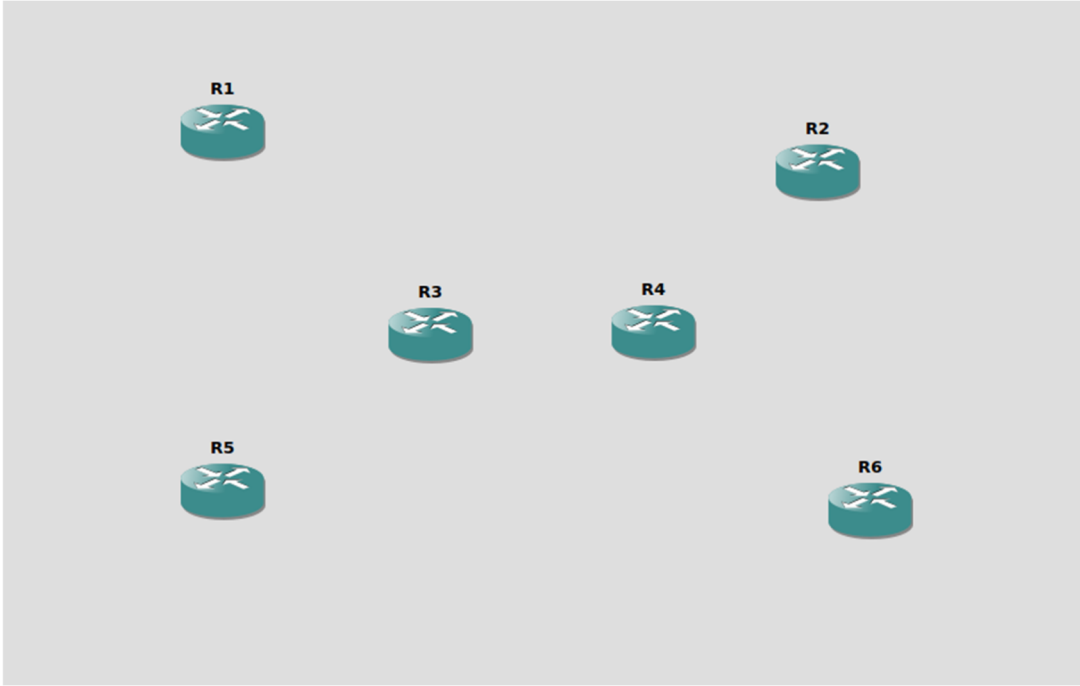
Значение поля Idle-PC очищаем.



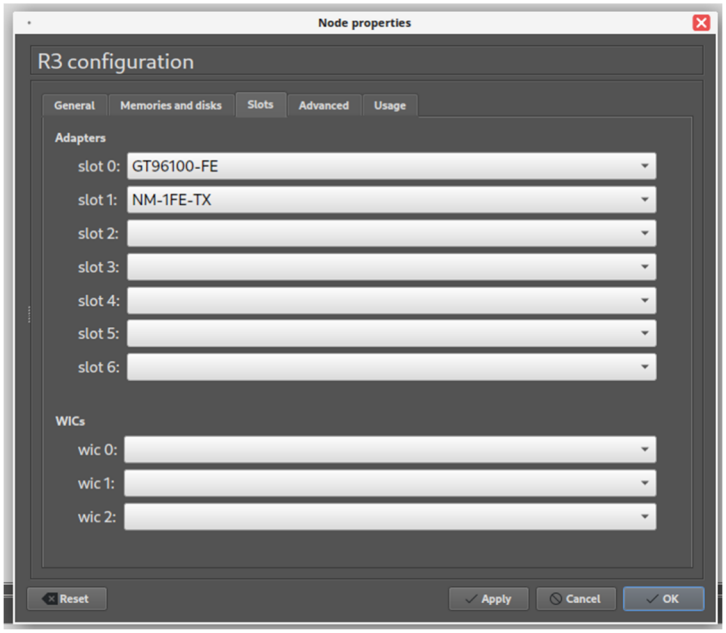
Часть 4: Создание сети

Для расположения элементов, достаточно просто перетянуть их с панели инструментов на рабочую область. Построим таким образом модель сети.

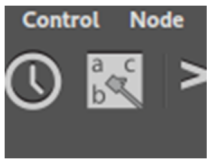
Для начала, выстроим роутеры.



Где нужно, добавим дополнительные порты через конфигурацию (правой кнопкой мыши по роутеру -> “Configure”).



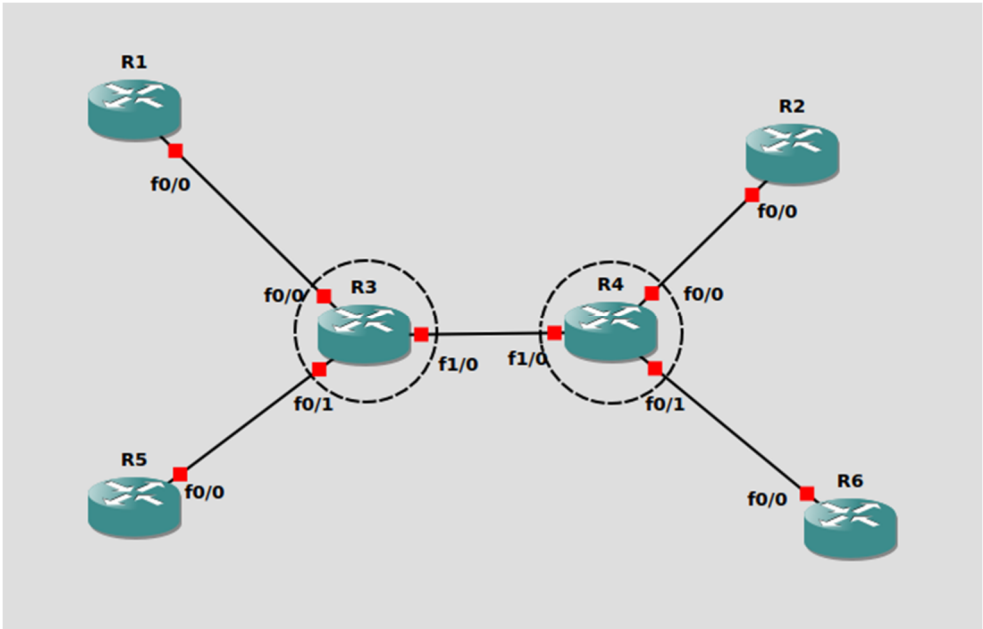
Соединим роутеры между собой. Вы можете отобразить названия сетей, нажав на эту кнопку.



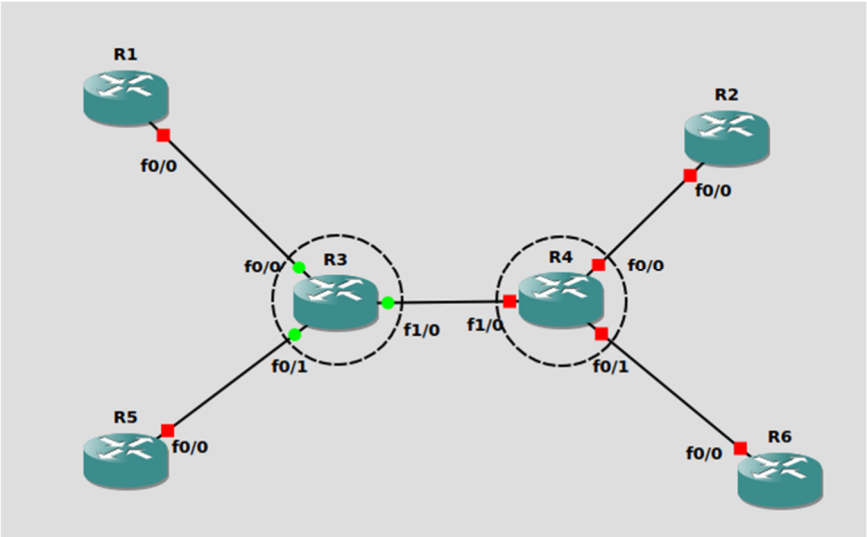
Само же соединение происходит за счёт инструмента:



У нас получилась такая модель (для своего удобства я пометил центральные роутеры обводкой):



Далее займёмся настройкой роутеров, начнём с R3. Запустите его, а после откройте консоль двумя нажатиями по роутеру. Когда роутер будет запущен, красный цвет соединений заменится на зелёный.



Чтобы перейти в режим конфигурации роутера, пропишите в консоль команду: conf t и нажмите Enter.

```
R3#
R3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#
```

Для начала, настроим дочерние роутеры, для этого определим для каждого из них основной шлюз, пусть для R1 это будет 192.168.1.1, а для R5 192.168.5.1, запомним это. Вводим в консоль команду для выбора текущего интерфейса: int fa0/0. Мы перешли в режим конфигурации интерфейса. Укажем ему ip адрес, для этого вводим команду: ip address 192.168.1.1 255.255.255.0, а после пропишем no shutdown, чтобы включить порт. Таким образом мы указали gateway для роутера R1. Прописываем exit для выхода в меню конфигурации роутера и проделываем всё то же самое для роутера R5. Не забудьте, что у него другой ip адрес. В конце настройки, вместо exit пропишите end, а после, для сохранения, пропишите wr mem.

```
R3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#int fa0/0
R3(config-if)#ip address 192.168.1.1 255.255.255.0
R3(config-if)#no shutdown
R3(config-if)#
*Mar  1 00:05:07.271: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state t
o up
*Mar  1 00:05:08.271: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et0/0, changed state to up
R3(config-if)#exit
R3(config)#in fa0/1
R3(config-if)#ip address 192.168.5.1 255.255.255.0
R3(config-if)#no shutdown
R3(config-if)#end
R3#
*Mar  1 00:05:52.475: %SYS-5-CONFIG_I: Configured from console by console
R3#
*Mar  1 00:05:52.843: %LINK-3-UPDOWN: Interface FastEthernet0/1, changed state t
o up
*Mar  1 00:05:53.843: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et0/1, changed state to up
R3#wr mem
Building configuration...
[OK]
R3#
```

Теперь займёмся дочерними роутерами R1 и R5. По очереди настраиваем каждый из них на свой интерфейс, то есть у R1 мы заходим в режим настройки интерфейса и прописываем: ip address 192.168.1.2 255.255.255.0. Не забудьте включить роутеры!

```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#int fa0/0
R1(config-if)#ip address 192.168.1.2 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#end
R1#wr mem
*Mar  1 00:01:03.679: %SYS-5-CONFIG_I: Configured from console by console
R1#wr mem
Building configuration...
[OK]
R1#
```

```
R5#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R5(config)#int fa0/0
R5(config-if)#ip address 192.168.5.2 255.255.255.0
R5(config-if)#no shutdown
R5(config-if)#end
R5#w
*Mar  1 00:00:33.871: %LINK-3-UPDOWN: Interface FastEthernet0/0, changed state t
o up
R5#wr mem
*Mar  1 00:00:34.003: %SYS-5-CONFIG_I: Configured from console by console
*Mar  1 00:00:34.871: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et0/0, changed state to up
R5#wr mem
Building configuration...
[OK]
```

Проверим работу сети, попробовав пропинговать шлюз.

```
[OK]
R5#ping 192.168.5.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.5.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 12/18/24 ms
R5#
```

Однако, если вы попробуете пропинговать другой дочерний роутер, вы заметите, что ничего не получится. Для того, чтобы разрешить это, нужно проложить маршрут в каждом из дочерних роутеров. Делается это в режиме конфигурации роутера командой: route 0.0.0.0 0.0.0.0 <адрес шлюза>, т.е. для R1 будет:

```
ip route 0.0.0.0 0.0.0.0 192.168.1.1
```

для R5 будет

```
ip route 0.0.0.0 0.0.0.0 192.168.5.1.
```

Не забудьте сохранить конфигурацию командой:

```
wr mem
```

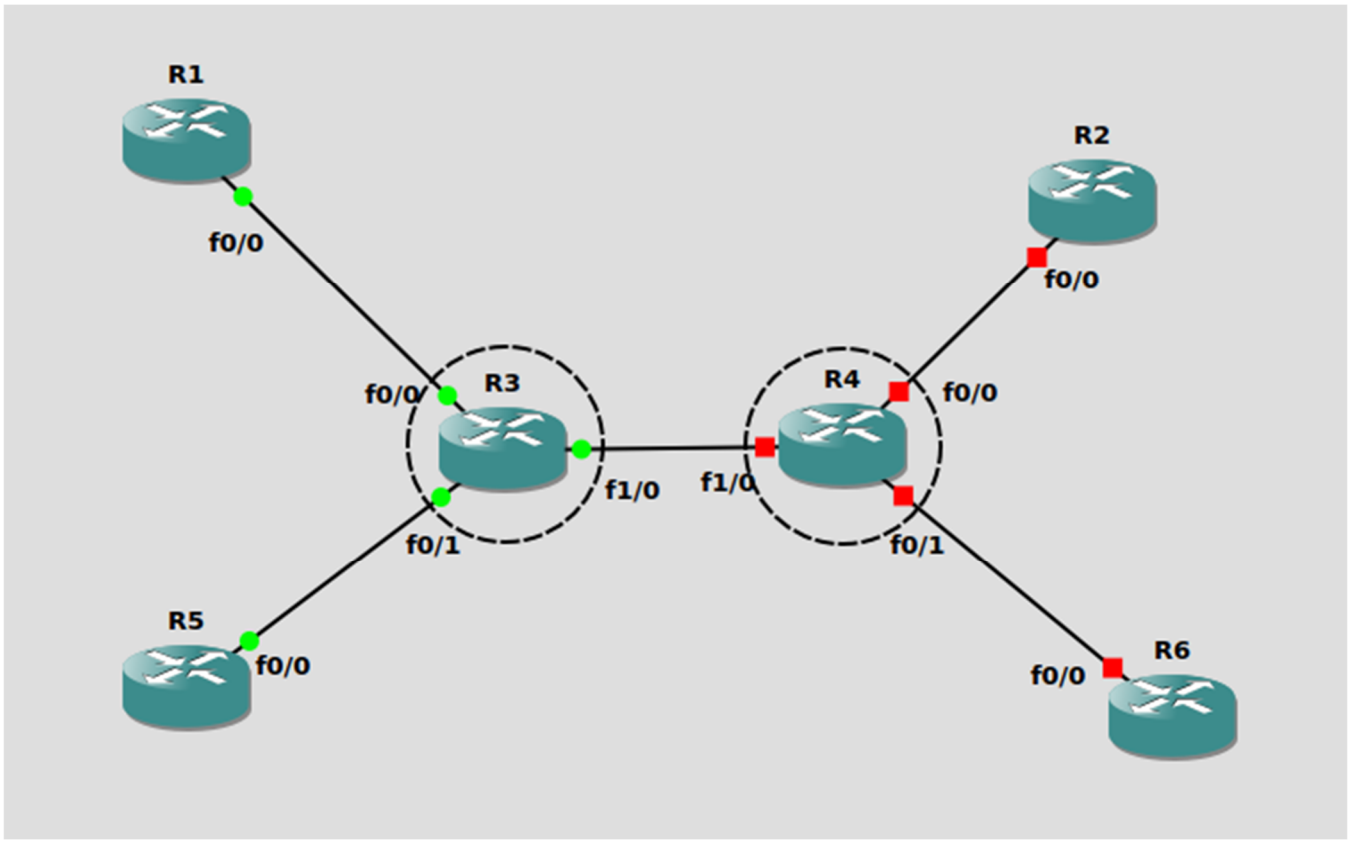
```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#ip route 0.0.0.0 0.0.0.0 192.168.1.1
R1(config)#end
R1#wr me
*Mar  1 00:06:47.451: %SYS-5-CONFIG_I: Configured from console by console
R1#wr mem
Building configuration...
[OK]
R1#
```

```
R5#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R5(config)#ip route 0.0.0.0 0.0.0.0 192.168.5.1
R5(config)#end
R5#wr mem
*Mar  1 00:06:07.727: %SYS-5-CONFIG_I: Configured from console by console
R5#wr mem
Building configuration...
[OK]
R5#
```

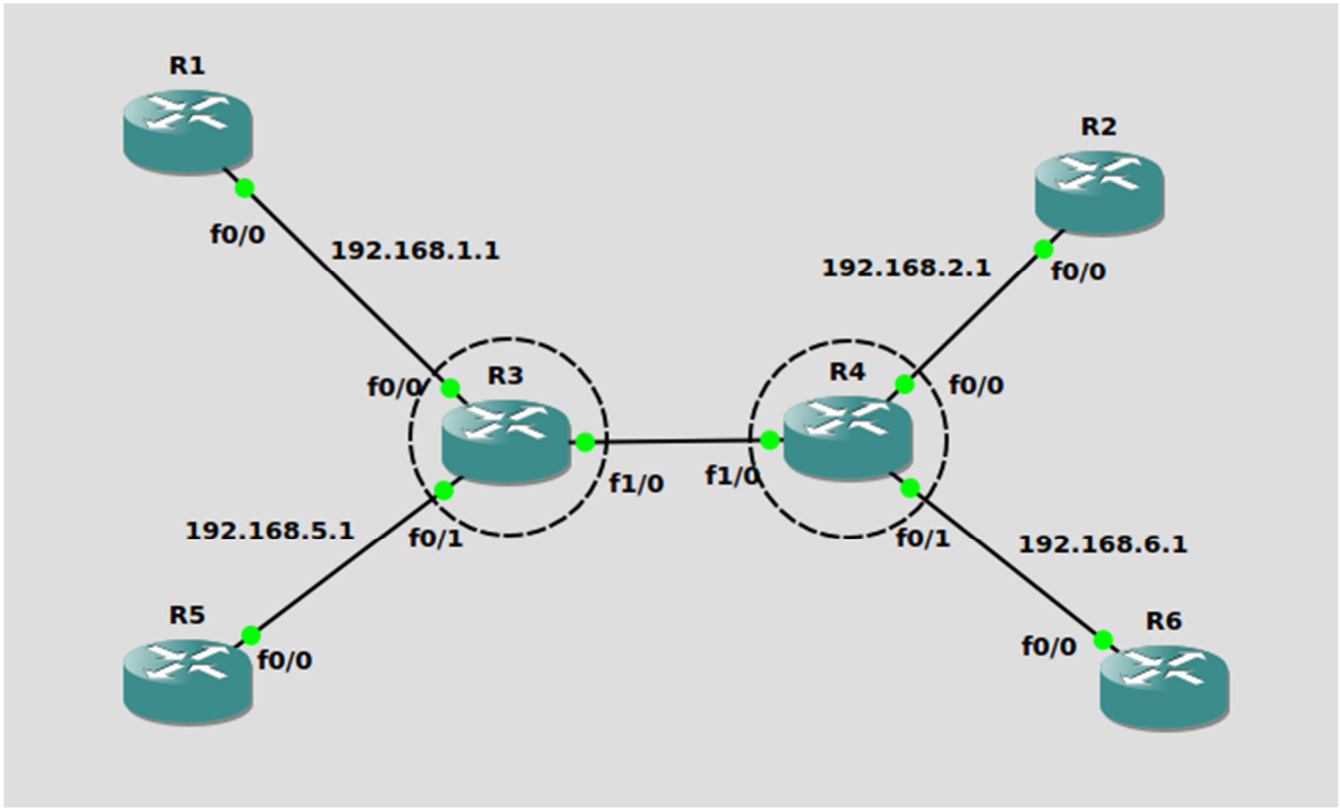
Теперь вы можете проверить работу сети, пропинговав через один роутер, другой, к примеру, через R5 пропинговать R1.

```
R5#ping 192.168.1.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 20/36/64 ms
R5#
```



В данный момент у нас работает лишь половина сети, проделайте все те же действия для другой её половины.



Как вы можете увидеть, мы почти настроили всю сеть, осталось добавить взаимодействие между двумя центральными роутерами, для этого выдадим им ip адреса. Пусть роутер R3 будет главным, тогда у него будет ip 192.168.4.1, а у R4 192.168.4.2.

```
R3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#int fa1/0
R3(config-if)#ip address 192.168.4.1 255.255.255.0
R3(config-if)#no shutdown
R3(config-if)#exit
R3(config)#ip route 0.0.0.0 0.0.0.0 192.168.4.2
R3(config)#end
R3#
R3#wr mem
*Mar  1 00:01:30.275: %SYS-5-CONFIG_I: Configured from console by console
R3#wr mem
Building configuration...
[OK]
R3#
```

```
R4#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R4(config)#int fa1/0
R4(config-if)#ip address 192.168.4.2
% Incomplete command.

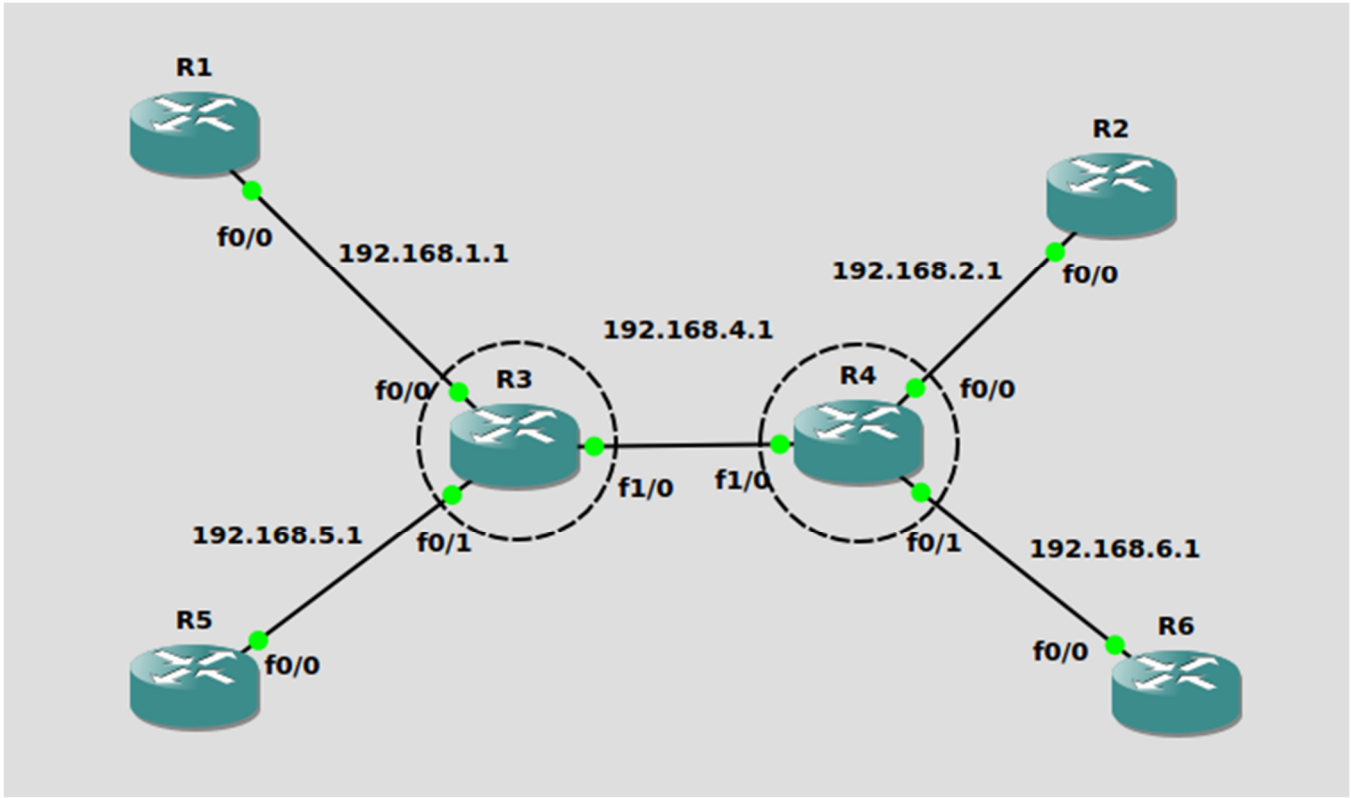
R4(config-if)#ip address 192.168.4.2 255.255.255.0
R4(config-if)#no shutdown
R4(config-if)#exit
R4(config)#ip route 0.0.0.0 0.0.0.0 192.168.4.1
R4(config)#end
R4#wr mem
Building configuration...

*Mar  1 00:07:52.455: %SYS-5-CONFIG_I: Configured from console by console[OK]
R4#
```

Попробуем пропинговать роутер R3 со стороны R4.

```
R4#ping 192.168.4.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.4.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 20/21/24 ms
R4#
```

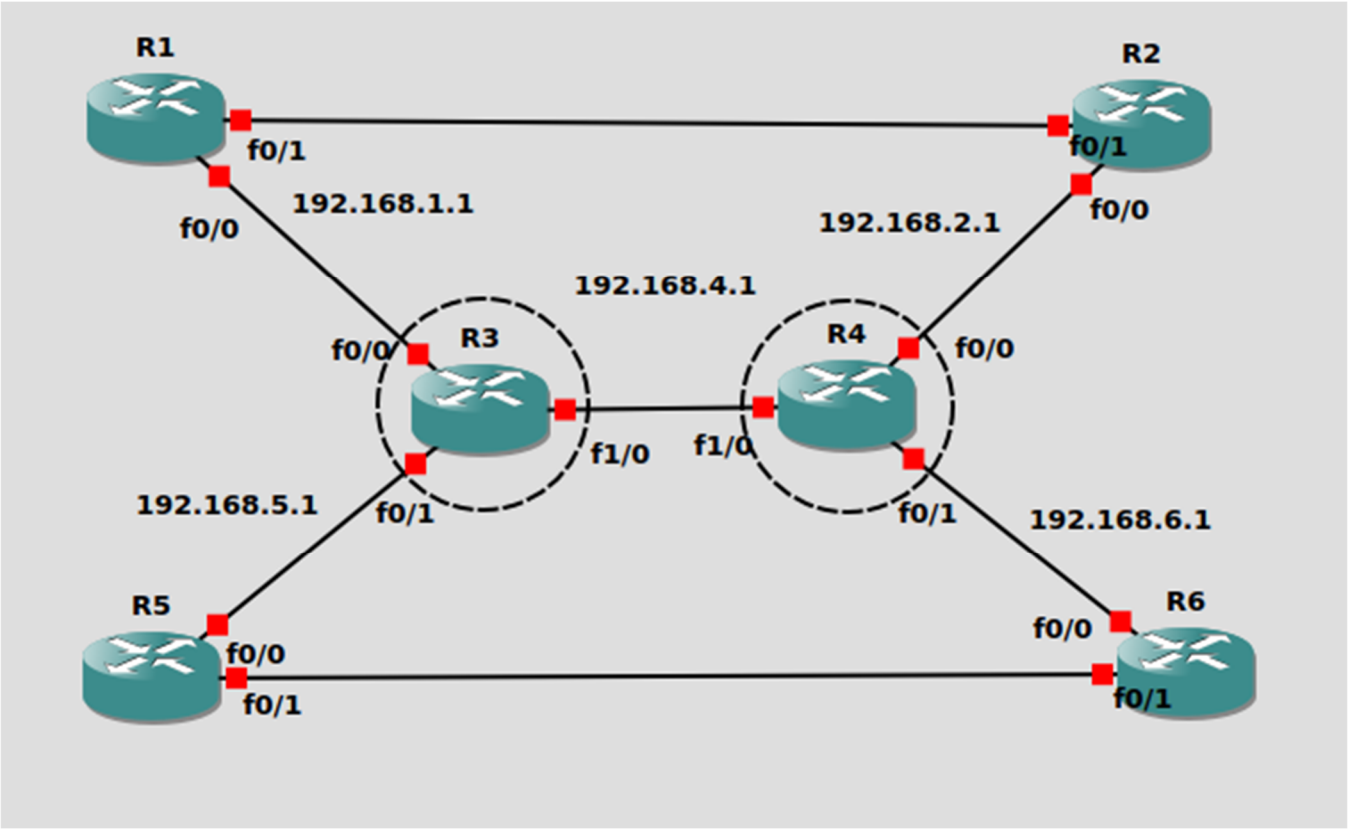


Теперь проверим работу всей сети, попробовав пропинговать роутер R2 из роутера R1.

```
R1#ping 192.168.2.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.2.2, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 268/380/544 ms
R1#
```

Усложним задачу и добавим пути в обход. Визуально это будет выглядеть так (с этого момента на скриншотах часть роутеров может быть выключена):



Теперь нужно настроить связь между этими роутерами по аналогии с центральными.

```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#int fa0/1
R1(config-if)#ip address 192.168.7.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#ex
*Mar  1 00:01:27.931: %LINK-3-UPDOWN: Interface FastEthernet0/1, changed state t
o up
*Mar  1 00:01:28.931: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et0/1, changed state to up
R1(config-if)#exit
```

```
R1(config)#ip route 192.168.2.0 255.255.255.0 192.168.7.2
R1(config)#ip route 192.168.6.0 255.255.255.0 192.168.7.2
R1(config)#end
R1#wr mem
Building configuration...
[OK]
R1#
```

```
to administratively down
R2#conf  t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#int fa0/1
R2(config-if)#ip address 192.168.7.2 255.255.255.0
R2(config-if)#no shutdown
R2(config-if)#exit
```

```
R2(config)#ip route 192.168.1.0 255.255.255.0 192.168.7.1
R2(config)#ip route 192.168.5.0 255.255.255.0 192.168.7.1
R2(config)#end
R2#wr mem
*Mar  1 00:01:04.871: %SYS-5-CONFIG_I: Configured from console by console
R2#wr mem
Building configuration...
[OK]
R2#
```

```
Success rate is 0 percent (0/5)
R2#ping 192.168.7.1

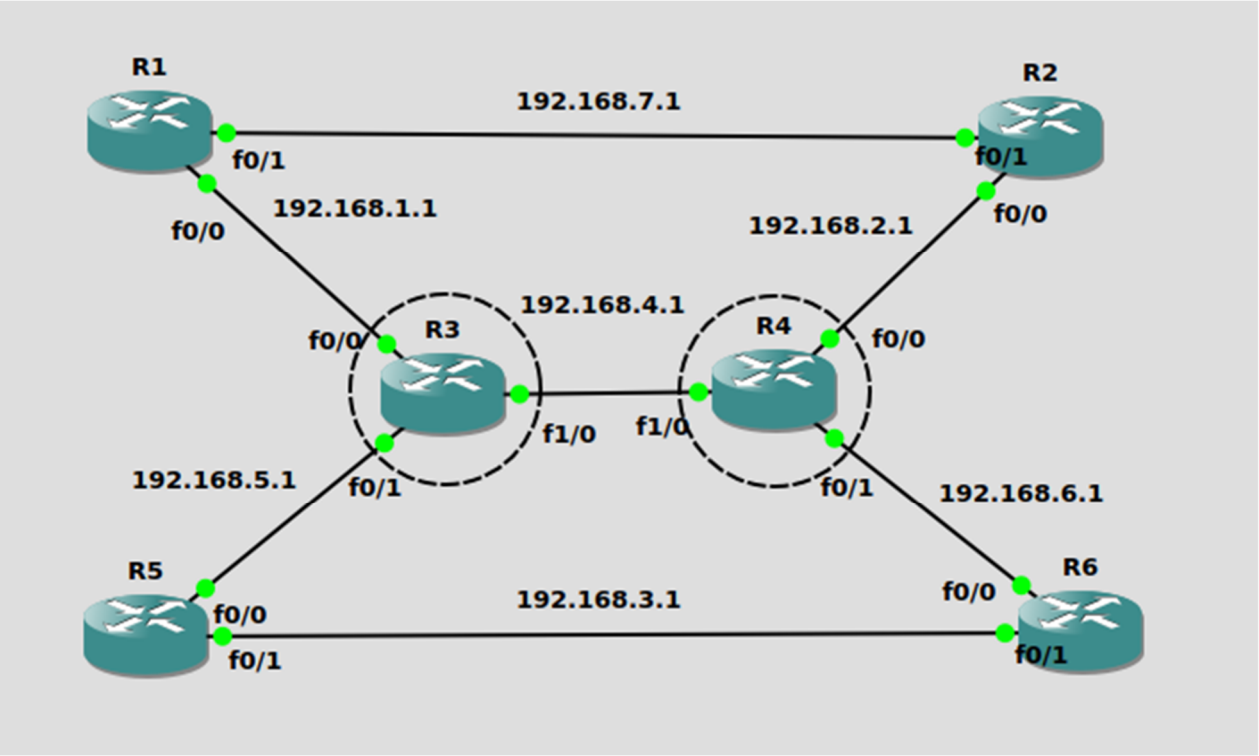
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.7.1, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 20/25/32 ms
R2#
```

Однако, этого будет недостаточно, нужно также настроить центральные роутеры на работу с новыми сетями.

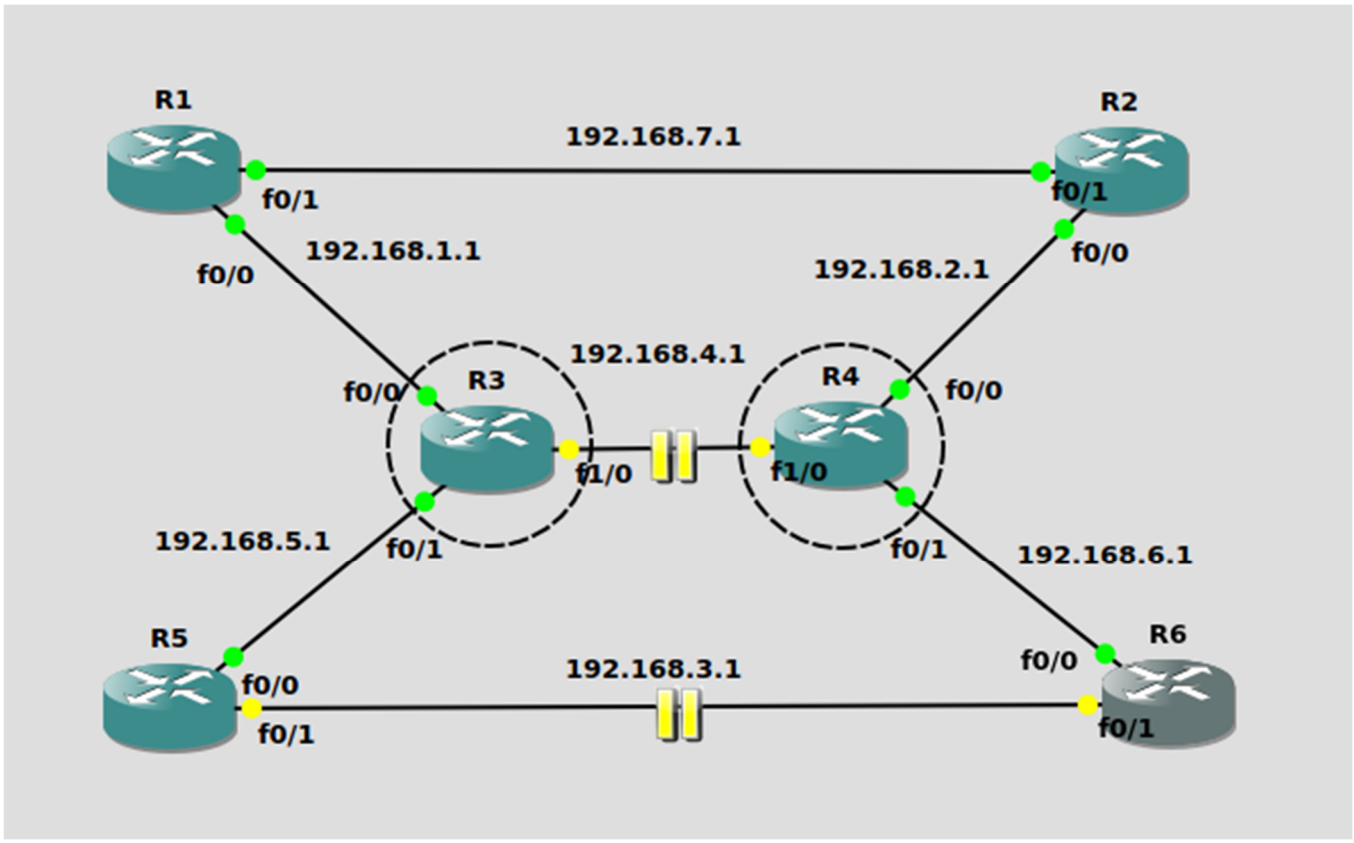
```
R3#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R3(config)#ip route 192.168.2.0 255.255.255.0 192.168.1.2
R3(config)#ip route 192.168.6.0 255.255.255.0 192.168.1.2
R3(config)#end
R3#wr
*Mar  1 00:01:30.723: %SYS-5-CONFIG_I: Configured from console by console
R3#wr mem
Building configuration...
[OK]
R3#
```

```
R4#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R4(config)#ip route 192.168.1.0 255.255.255.0 192.168.2.2
R4(config)#ip route 192.168.5.0 255.255.255.0 192.168.2.2
R4(config)#end
R4#wr
*Mar  1 00:00:36.423: %SYS-5-CONFIG_I: Configured from console by console
R4#wr
Building configuration...
[OK]
R4#
```

Таким же образом произведите настройку роутеров R5 и R6.



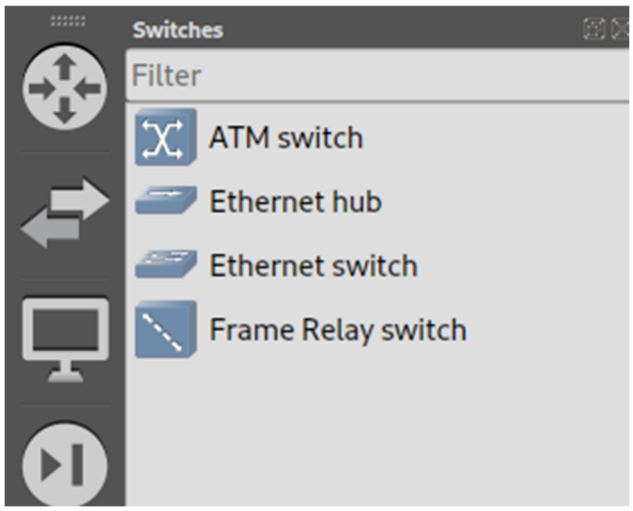
Теперь можно проверить работоспособность обходных узлов, удалив соединение между центральными роутерами (не забудьте проложить маршрут у каждого роутера к своим узлам).

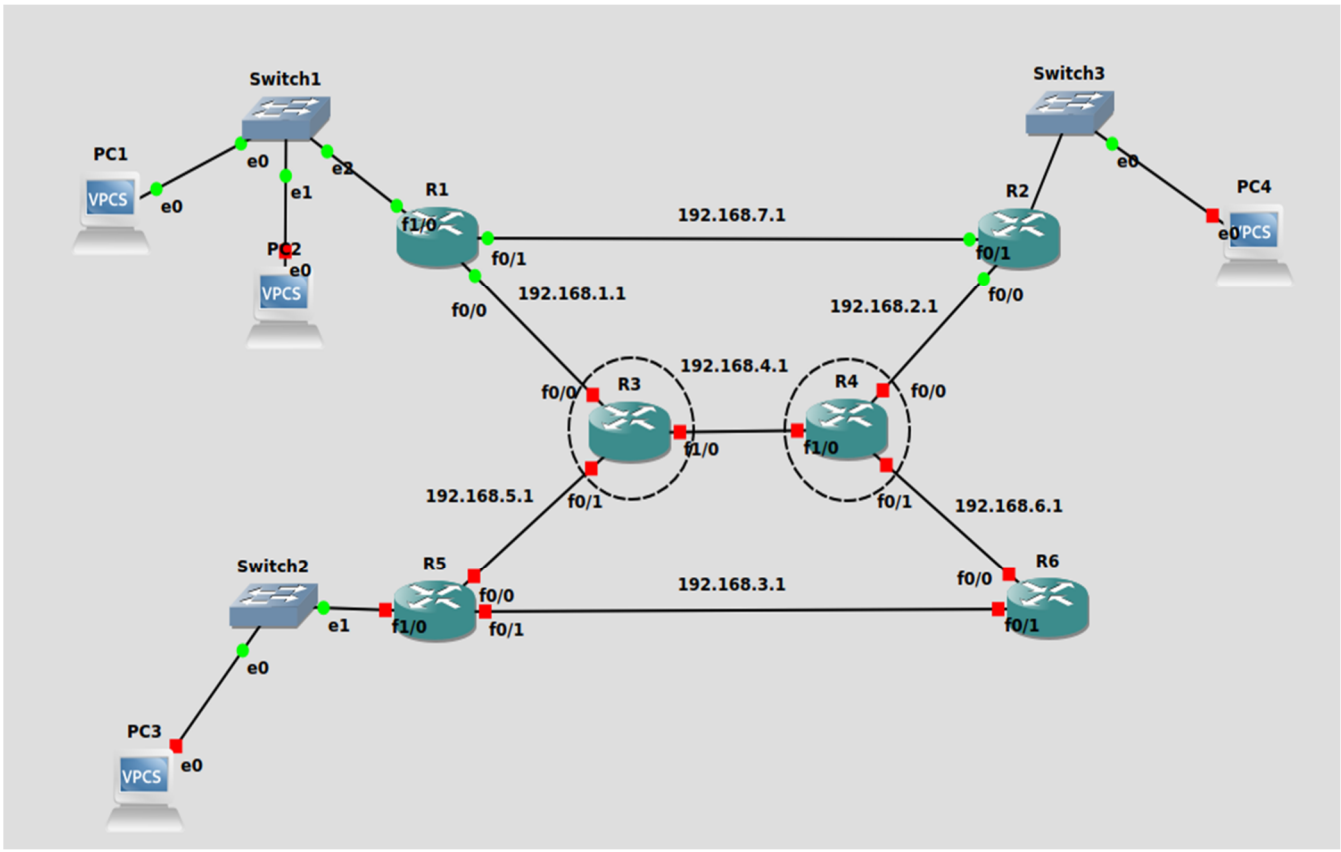



```
et1/0, changed state to up
R6#ping 192.168.1.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 292/632/1696 ms
R6#
```

Для добавления коммутаторов и компьютеров перенесите их также на модель.





Не забудьте сделать порты на роутере.

```
et1/0, changed state to down
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#int fa1/0
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#end
R1#
*Mar  1 00:01:57.999: %SYS-5-CONFIG_I: Configured from console by console
R1#wr
*Mar  1 00:01:58.759: %LINK-3-UPDOWN: Interface FastEthernet1/0, changed state t
o up
*Mar  1 00:01:59.759: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et1/0, changed state to up
R1#wr mem
Building configuration...
[OK]
R1#
```

После этого необходимо прописать маршрут к локальной сети из роутера, к которому в последующем мы будем обращаться.

```
*Mar  1 00:00:12.355: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthern
et1/0, changed state to down
R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#ip route 192.168.10.0 255.255.255.0 192.168.7.1
R2(config)#end
R2#wr mem
Building configuration...

*Mar  1 00:09:39.351: %SYS-5-CONFIG_I: Configured from console by console[OK]
R2#
```

```
PC1> ip 192.168.10.2 192.168.10.1
Checking for duplicate address...
PC1 : 192.168.10.2 255.255.255.0 gateway 192.168.10.1

PC1> ping 192.168.10.1

84 bytes from 192.168.10.1 icmp_seq=1 ttl=255 time=9.448 ms
84 bytes from 192.168.10.1 icmp_seq=2 ttl=255 time=3.980 ms
^C
PC1> ping 192.168.7.1

84 bytes from 192.168.7.1 icmp_seq=1 ttl=255 time=7.325 ms
84 bytes from 192.168.7.1 icmp_seq=2 ttl=255 time=10.080 ms
^C
PC1> ping 192.168.7.2

84 bytes from 192.168.7.2 icmp_seq=1 ttl=254 time=45.102 ms
84 bytes from 192.168.7.2 icmp_seq=2 ttl=254 time=19.898 ms
^C
PC1>
```

Таким образом вы можете самостоятельно настраивать сети, а также их подключения.